

Bianca Valenciani

Implementação e Simulação da Unidade de Processamento de um Processador RISC-V RV32IM Monociclo

São José dos Campos - Brasil

Julho de 2025

Bianca Valenciani

Implementação e Simulação da Unidade de Processamento de um Processador RISC-V RV32IM Monociclo

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Julho de 2025

Resumo

Este relatório apresenta o desenvolvimento completo de um **processador RISC-V de 32 bits**, baseado na configuração **RV32IM**, que implementa o conjunto base de instruções inteiras (**RV32I**) juntamente com uma extensão simplificada para **operações de multiplicação e divisão (M)**. Ao longo do projeto, todos os módulos fundamentais da **unidade de processamento** foram desenvolvidos em **Verilog**, incluindo a **Unidade Lógica e Aritmética (ULA)**, a **unidade de multiplicação/divisão**, o **banco de registradores**, o **extensor de imediato** e os **multiplexadores** necessários para o fluxo correto de dados.

Após a verificação individual de cada módulo, esses componentes foram integrados em um núcleo completo e funcional, com suporte à leitura e escrita em **memória de dados**, controle de fluxo por desvios condicionais e **operações com periféricos MMIO**. A funcionalidade do processador foi validada tanto por **simulação funcional**, quanto por meio de **testes práticos em hardware**, utilizando uma placa **FPGA DE2-115**. Programas de teste em assembly, incluindo um sistema básico de entrada/saída e o cálculo do número de Fibonacci foram executados com sucesso, comprovando o funcionamento correto do pipeline de dados, da unidade de controle e da comunicação com os periféricos.

Palavras-chaves: RISC-V, Arquitetura de Computadores, Datapath, Processador Monociclo, RV32I, RV32M, Conjunto de Instruções, Verilog, Quartus, Simulação, FPGA DE2-115.

Lista de ilustrações

Figura 1 – Formatos de instrução da arquitetura RISC-V	14
Figura 2 – Formatos de instrução da arquitetura RISC-V: Load e Store	15
Figura 3 – Diferença entre Arquitetura de Harvard e de Von Neumman	17
Figura 4 – Formatos de Instrução RISC-V (RV32I/M) utilizados	25
Figura 5 – Diagrama total do Datapath do Processador RV32IM Monociclo	26
Figura 6 – Diagrama 1/3 do Datapath do Processador RV32IM Monociclo	27
Figura 7 – Diagrama 2/3 do Datapath do Processador RV32IM Monociclo	28
Figura 8 – Diagrama 3/3 do Datapath do Processador RV32IM Monociclo	29
Figura 9 – Saída do console da simulação do testbench da ULA, detalhando os casos de teste.	50
Figura 10 – Forma de onda da simulação da ULA (Parte 1): Testes iniciais incluindo AND, OR, ADD, SUB.	51
Figura 11 – Forma de onda da simulação da ULA (Parte 2): Testes incluindo XOR, SLT e SLL.	52
Figura 12 – Forma de onda da simulação da ULA (Parte 3): Testes incluindo SRL, SRA e operação default.	52
Figura 13 – Saída do console da simulação do testbench da unidade de Multiplicação/Divisão.	53
Figura 14 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação MUL.	54
Figura 15 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação DIV.	55
Figura 16 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação REM e Default.	56
Figura 17 – Saída do console da simulação do testbench do Extensor de Imediato.	57
Figura 18 – Forma de onda da simulação do Extensor de Imediato (Parte 1): Testes dos tipos I e S.	58
Figura 19 – Forma de onda da simulação do Extensor de Imediato (Parte 2): Testes dos tipos B e J.	59
Figura 20 – Saída do console da simulação do testbench do Somador PC Target.	59
Figura 21 – Forma de onda da simulação do Somador PC Target.	60
Figura 22 – Saída do console da simulação do testbench do Banco de Registradores.	61
Figura 23 – Forma de onda da simulação do Banco de Registradores (Parte 1): Testes de Reset, Escrita/Leitura, x0.	62
Figura 24 – Forma de onda da simulação do Banco de Registradores (Parte 2): Testes de escrita desabilitada, sobrescrita e leitura simultânea.	63

Figura 25 – Saída do console da simulação do testbench do Contador de Programa.	64
Figura 26 – Forma de onda da simulação do Contador de Programa (Parte 1): Teste de Reset e primeira carga.	64
Figura 27 – Forma de onda da simulação do Contador de Programa (Parte 2): Cargas subsequentes.	65
Figura 28 – Saída do console da simulação do testbench do Somador PC+4.	66
Figura 29 – Forma de onda da simulação do Somador PC+4.	67
Figura 30 – Saída do console da simulação do testbench da UnidadeProcessamento	68
Figura 31 – Forma de onda da simulação da UnidadeProcessamento integrada.	69
Figura 32 – FPGA mostrando o final da sequência inputada pelo usuário.	72

Lista de tabelas

Tabela 1 – Operações Aritméticas e Lógicas (Tipo R)	21
Tabela 2 – Aritmética com Imediatos (Tipo I)	21
Tabela 3 – Acesso à Memória (Tipos I e S)	22
Tabela 4 – Controle de Fluxo (Tipos B e J)	22
Tabela 5 – Multiplicação e Divisão (RV32M Simplificado)	22
Tabela 6 – Codificação das Instruções Tipo R Implementadas.	23
Tabela 7 – Codificação das Instruções Tipo I Implementadas (Imediato e Load).	23
Tabela 8 – Codificação das Instruções Tipo S Implementadas (Store).	24
Tabela 9 – Codificação das Instruções Tipo B Implementadas (Branch Condicional).	24
Tabela 10 – Codificação das Instruções Tipo J e I Implementadas (Saltos e Sistema).	24

Sumário

1	INTRODUÇÃO	9
2	OBJETIVOS	11
2.1	Geral	11
2.2	Específico	11
3	FUNDAMENTAÇÃO TEÓRICA	13
3.1	Arquiteturas RISC vs. CISC	13
3.2	ISA RISC-V	13
3.3	Formatos de Instrução RISC-V	14
3.4	Modos de Endereçamento RISC-V	15
3.5	Arquitetura Harvard vs. Von Neumann	16
3.6	Processadores Monociclo vs. Multiciclo	17
3.7	Componentes do <i>Datapath</i> Monociclo	18
3.8	<i>Memory-Mapped I/O</i> (MMIO)	18
3.9	Linguagem de Descrição de Hardware: Verilog	18
4	DESENVOLVIMENTO	21
4.1	Conjunto de Instruções Implementado	21
4.2	Formatos e Codificação das Instruções	22
4.3	Datapath do Processador	25
4.3.1	Descrição dos Componentes Principais (Referenciando Figura 5)	30
4.3.1.1	Bloco PC e Lógica Associada (Figura 6)	30
4.3.1.2	Banco de Registradores (Figura 7):	31
4.3.1.3	Extensor de Imediato (Figura 7):	31
4.3.1.4	ULA (ALU - Unidade Lógica e Aritmética) (Figura 7)	31
4.3.1.5	MUX1 (aluscr) (Figura 7):	32
4.3.1.6	Unidade de Controle (Figura 7):	32
4.3.1.7	Unidade de Branch (Figura 7):	32
4.3.1.8	Unidade de Multiplicação e Divisão (Figura 7):	33
4.3.1.9	MUX2 (Aluresult) (Figura 8):	33
4.3.1.10	Memória de Dados (Figura 8):	33
4.3.1.11	Decodificador MMIO Saída (Figura 8):	33
4.3.1.12	Display Saída (Figura 8):	33
4.3.1.13	Bloco Entrada (Figura 8):	33
4.3.1.14	Decodificador MMIO (Entrada) (Figura 8):	34

4.3.2	Interfaces MMIO	34
4.3.2.0.1	Dispositivo de Entrada (input_device.v):	34
4.3.2.1	MUX5 (Input_select) (Figura 8):	34
4.3.2.2	MUX3 (memtoreg) (Figura 8):	34
4.3.3	Fluxo de Dados e Sinais de Controle por Tipo de Instrução	34
4.3.4	Implementação dos Módulos da Unidade de Processamento em Verilog	38
4.3.4.1	Contador de Programa (PC) - program_counter.v	38
4.3.4.2	Somador PC+4 - pc_plus_4_adder.v	38
4.3.4.3	Somador de Endereço de Destino para Desvios e Saltos Diretos (PC Target Adder) - pc_target_adder.v	38
4.3.4.4	Somador de Endereço de Destino para Desvios e Saltos Diretos (PC Target Adder)	39
4.3.4.5	Multiplexador da Fonte do Próximo PC (MUX4 - PcSource) - mux_pc_source.v	39
4.3.4.6	Unidade Lógica e Aritmética (ULA) - ulav	40
4.3.4.7	Unidade de Multiplicação e Divisão - div_mult.v	40
4.3.4.8	Extensor de Imediato - extensor.v	41
4.3.4.9	Banco de Registradores - bancoderegistradores.v	41
4.3.4.10	Multiplexadores da Unidade de Processamento	41
4.3.4.11	Módulos de Interface MMIO (input_device.v e Lógica de Display)	42
4.3.4.11.1	Dispositivo de Entrada (input_device.v):	42
4.3.4.11.2	Lógica do Dispositivo de Saída (Display):	42
4.3.4.11.3	Dispositivo de Entrada (input_device.v):	43
4.3.5	Integração da Unidade de Processamento - UnidadeProcessamento.v	43
4.3.5.1	Módulos de Interface com Hardware Físico (FPGA)	46
4.3.5.1.1	Debouncer de Botão (debouncer.v e debouncer_level.v):	46
4.3.5.1.2	Divisor de Clock (clock_divider.v):	46
4.3.5.1.3	Conversor Binário para BCD (binary_to_bcd.v):	47
4.3.5.1.4	Memória de Dados (data_memory.v):	47
5	RESULTADOS OBTIDOS E DISCUSSÕES	49
5.1	Simulação dos Módulos Individuais da Unidade de Processamento	49
5.1.1	Simulação da Unidade Lógica e Aritmética (ULA)	49
5.1.2	Simulação da Unidade de Multiplicação e Divisão	53
5.1.3	Simulação do Extensor de Imediato	56
5.1.4	Simulação do Somador de Endereço de Destino (PC Target Adder)	59
5.1.5	Simulação do Banco de Registradores	61
5.1.6	Simulação do Contador de Programa (PC)	63
5.1.7	Simulação do Somador PC+4	65
5.2	Simulação da Unidade de Processamento Integrada - UnidadePro- cessamento	67
5.2.0.0.1	Assembler RISC-V para Geração de Código de Máquina:	70

5.3	Verificação em Hardware (FPGA)	70
5.3.1	Configuração do Teste	70
5.3.2	Resultados do Teste de I/O	71
5.3.3	Resultados do Teste de Fibonacci	71
5.3.4	Conclusão da Seção	72
5.4	Análise Geral e Comprovação de Funcionamento	72
5.4.1	Verificação Detalhada e Códigos dos Testbenches	73
6	CONSIDERAÇÕES FINAIS	75
	REFERÊNCIAS	77
	APÊNDICES	79
	APÊNDICE A – CÓDIGOS FIBONACCI	81
	APÊNDICE B – CÓDIGOS VERILOG DO PROCESSADOR	83
B.1	Código RISC_V_processor.v	83
B.2	Código toplevel.v	87
B.3	Códigos dos módulos separadamente	90
	APÊNDICE C – CÓDIGOS VERILOG DOS TESTBENCHES	105
C.1	Testbench da ULA (ula_riscv_comprehensive_tb.v)	105
C.2	Testbench da Unidade de Multiplicação/Divisão (div_mult_tb.v)	106
C.3	Testbench da Unidade de Extensão (extensor_tb.v)	109
C.4	Testbench da Unidade de Banco de Registradores (bancoderegistradores_tb.v)	112
C.5	Testbench da Unidade de Mux1 (mux1_tb.v)	115
C.6	Testbench da Unidade de Mux2 (mux2.v)	116
C.7	Testbench da Unidade de PC (program_counter_tb.v)	118
C.8	Testbench da Unidade de PC+4 (pc_plus_4adder_tb.v)	119
C.9	Testbench da Unidade de PC Target Adder (pc_target_adder_tb.v)	120
C.10	Testbench da Unidade de MUX4 (mux_pc_source_tb.v)	120
C.11	Testbench da Unidade de Processamento (UnidadeProcessamento_tb.v)	122
	APÊNDICE D – ASSEMBLER	127
	ANEXOS	131

1 Introdução

A arquitetura de computadores é um dos pilares fundamentais da Engenharia da Computação, pois define a organização interna, o funcionamento e as interações entre os componentes de um sistema computacional. Compreender a arquitetura permite projetar sistemas mais eficientes, seguros e otimizados, além de fornecer a base necessária para o desenvolvimento de *softwares* que exploram melhor os recursos de *hardware* disponíveis.

Nesse contexto, destaca-se o paradigma RISC (*Reduced Instruction Set Computer*)(1), que adota a filosofia de um conjunto reduzido e simplificado de instruções, visando a maior eficiência na execução das operações. Dentre as principais vantagens das arquiteturas RISC estão a simplicidade na implementação do hardware, a regularidade nos formatos das instruções e a facilidade para desenvolvimento de pipelines otimizados.

O RISC-V é uma arquitetura de conjunto de instruções (ISA) baseada nesse paradigma, e tem se consolidado como uma das opções mais promissoras nos últimos anos. Diferentemente de outras ISAs, o RISC-V é aberto, livre de royalties, modular e extensível. Essas características tornam a arquitetura altamente flexível, facilitando sua adoção tanto em ambientes acadêmicos quanto industriais, incluindo aplicações embarcadas, sistemas operacionais, data centers e até processadores de alto desempenho(2).

Este trabalho foca no projeto de um processador baseado na arquitetura RISC-V, capaz de executar o conjunto base de instruções inteiras (RV32I), além da extensão M para operações de multiplicação e divisão, em um *datapath* de ciclo único. Também será incorporado o suporte a entrada e saída por meio de mapeamento de memória (MMIO), incluindo um dispositivo de entrada genérico e um *display* de sete segmentos.

A escolha do RISC-V para este projeto se justifica pelas suas vantagens em relação a outras ISAs educacionais, como o MIPS. Além de mais moderna e ativa, a comunidade RISC-V oferece suporte contínuo, documentação acessível e exemplos práticos, o que torna o processo de aprendizado e implementação mais alinhado com as tendências atuais da computação. Sua modularidade, por exemplo, permite a inclusão gradual de extensões como multiplicação e divisão, operações de ponto flutuante ou suporte à vetorização, conforme a necessidade do projeto(1).

O relatório está estruturado da seguinte forma: o Capítulo 2 apresenta os objetivos geral e específicos que nortearam o projeto. O Capítulo 3 aborda os fundamentos teóricos relevantes, incluindo tópicos adicionais relacionados à implementação em hardware. O 4 discorre todo desenvolvimento e testes do processador além de detalhar o conjunto de instruções adotado, sua codificação, o datapath projetado e a implementação completa em Verilog dos módulos da unidade de processamento. Já o Capítulo 5 descreve os

resultados das simulações funcionais, bem como a validação dos módulos integrados. Finalmente, o Capítulo 6 apresenta as considerações finais, destacando os resultados obtidos com a verificação em FPGA e as dificuldades enfrentadas. Foram utilizadas referências técnicas, manuais da especificação RISC-V e materiais acadêmicos para fundamentar o desenvolvimento e a implementação do processador.

2 Objetivos

Este capítulo apresenta os objetivos geral e específicos que orientaram o desenvolvimento do processador RISC-V ao longo do projeto, do projeto arquitetural à verificação prática em hardware.

2.1 Geral

O objetivo geral deste projeto foi projetar, implementar em Verilog e verificar funcionalmente um processador RISC-V monociclo, capaz de executar o conjunto base de instruções inteiras (RV32I) e uma versão simplificada da extensão de multiplicação e divisão (RV32M), incluindo também suporte a periféricos de entrada e saída por meio de mapeamento em memória (MMIO). Ao final do projeto, o processador foi inteiramente validado por simulação e também em hardware, por meio da execução de programas de teste em uma FPGA.

2.2 Específico

Para atingir o objetivo geral, os seguintes objetivos específicos foram estabelecidos e alcançados ao longo do desenvolvimento:

- Definir o subconjunto de instruções RV32I e RV32M (simplificada) a ser implementado e especificar os formatos de instrução (R, I, S, B, J) e a codificação binária correspondente para cada instrução, considerando os campos `opcode`, `funct3` e `funct7`, quando aplicável.
- Detalhar os modos de endereçamento utilizados nas instruções de acesso à memória e de desvio condicional.
- Projetar o datapath monociclo completo, incluindo a identificação e interligação dos componentes essenciais, como memória de instruções, banco de registradores, ULA, unidades de multiplicação e divisão, extensores de imediato, multiplexadores e módulos de entrada/saída.
- Especificar os sinais de controle necessários à operação coordenada do datapath, conforme os diferentes tipos de instruções da ISA.
- Projetar e integrar as interfaces MMIO para dispositivos físicos de entrada e saída.

- Implementar em Verilog os módulos da unidade de processamento: ULA, banco de registradores, extensor de imediato, unidade de multiplicação e divisão (simplificada), multiplexadores e demais componentes necessários.
- Integrar os módulos desenvolvidos no núcleo de processamento principal (RISCV_processor), responsável pela execução das instruções da ISA.
- Desenvolver testbenches em Verilog para simular e verificar individualmente o funcionamento de cada módulo implementado.
- Simular e validar o funcionamento do processador completo a partir de um testbench integrado, utilizando instruções representativas da ISA definida.
- Analisar formas de onda e saídas simuladas para verificar a conformidade da implementação com o comportamento esperado.
- Integrar o processador com periféricos reais em uma FPGA (DE2-115) e realizar testes práticos com programas de entrada/saída e cálculo de Fibonacci.
- Confirmar a operação correta do processador no ambiente de hardware, incluindo o caminho de dados, a lógica de controle, o acesso à memória e os módulos de I/O.

3 Fundamentação Teórica

3.1 Arquiteturas RISC vs. CISC

As arquiteturas de computadores podem ser amplamente classificadas em duas filosofias principais de projeto de conjunto de instruções: RISC (Reduced Instruction Set Computer) e CISC (Complex Instruction Set Computer).

As arquiteturas **RISC**, como a adotada neste projeto (RISC-V), são baseadas em princípios que visam simplificar o conjunto de instruções, permitindo maior eficiência na execução e um design de hardware mais simples. As principais características dessas arquiteturas incluem a utilização de um número reduzido de instruções de tamanho fixo, um grande número de registradores de propósito geral, execução de a maioria das instruções em um único ciclo de clock (em designs monociclo) e a separação estrita entre instruções de acesso à memória (operações de *load/store*) e instruções de processamento aritmético/lógico, que operam apenas em registradores (3). Essas características facilitam o desenvolvimento de datapaths mais simples, otimizações por parte dos compiladores e a aplicação de técnicas avançadas como *pipelining*, tornando a arquitetura RISC atrativa para sistemas embarcados e aplicações de alto desempenho.

Em contraste, as arquiteturas **CISC** possuem um conjunto de instruções mais vasto e complexo. As instruções CISC podem variar em tamanho, realizar múltiplas operações de baixo nível (como ler da memória, realizar uma operação e escrever na memória, tudo em uma única instrução) e suportar diversos e complexos modos de endereçamento. O objetivo original era facilitar a programação em *assembly* e reduzir o "gap semântico" entre linguagens de alto nível e a máquina. No entanto, essa complexidade leva a um hardware de controle mais intrincado e dificulta a implementação eficiente de técnicas como o *pipelining*. Exemplos clássicos de arquiteturas CISC incluem a família x86 da Intel.

3.2 ISA RISC-V

A RISC-V é uma arquitetura de conjunto de instruções (ISA) aberta e modular, que segue os princípios das arquiteturas RISC clássicas. Sua base é composta por um conjunto essencial de instruções inteiras, chamado de **RV32I** (1), que define operações aritméticas, lógicas, controle de fluxo e acesso à memória em registradores de 32 bits.

Além do conjunto básico, a RISC-V permite extensões modulares. Neste projeto, destacamos a extensão **M**, que adiciona suporte para multiplicação e divisão inteiras, resultando na configuração **RV32IM** (1).

3.3 Formatos de Instrução RISC-V

As instruções da RISC-V são codificadas em formatos diferentes, dependendo da operação a ser realizada. A seguir, são apresentados os principais formatos:

- **Formato R:** Operações entre registradores.
- **Formato I:** Operações com imediato ou carga de memória.
- **Formato S:** Instruções de armazenamento em memória.
- **Formato B:** Instruções de desvio condicional.
- **Formato J:** Desvios incondicionais (*jump*).

As instruções da RISC-V são codificadas em formatos diferentes, dependendo da operação a ser realizada. Os principais formatos são R, I, S, B e J, ilustrados nas Figuras 1 e 2. Existe também o formato U (Upper Immediate), utilizado por instruções como LUI (Load Upper Immediate) e AUIPC (Add Upper Immediate to PC), que carregam um imediato de 20 bits nos bits mais significativos de um registrador. Embora parte da ISA base RV32I, as instruções de formato U não foram incluídas no escopo de implementação deste projeto para manter o foco no conjunto RV32IM simplificado inicialmente proposto. A estrutura detalhada de todos os formatos pode ser consultada na especificação oficial (1).

A Figura 1 ilustra a estrutura genérica dos formatos R, I, S e U, conforme apresentado por (4).

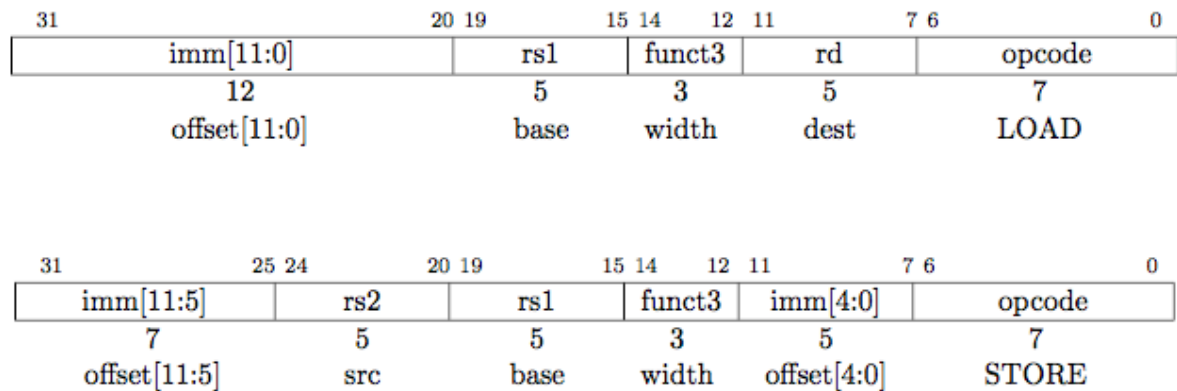
Figura 1 – Formatos de instrução da arquitetura RISC-V

31	25 24	20 19	15 14	12 11	7 6	0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[31:12]					rd	opcode	U-type

Fonte: Grupo iMobilis, DECOM/UFOP(4)

Para as operações de acesso à memória, como carga (Load) e armazenamento (Store), os formatos I e S, respectivamente, são utilizados. Estes formatos são detalhados na Figura 2, que ilustra como os campos de imediato são organizados para formar o endereço de memória (4).

Figura 2 – Formatos de instrução da arquitetura RISC-V: Load e Store



Fonte: Grupo iMobilis, DECOM/UFOP(4)

A estrutura detalhada desses formatos pode ser consultada na especificação oficial (1) ou em guias como (2).

3.4 Modos de Endereçamento RISC-V

Os modos de endereçamento definem como o processador determina o endereço dos operandos para uma instrução. O RISC-V, sendo uma arquitetura de carga/armazenamento (*load/store*), utiliza modos de endereçamento relativamente simples, principalmente para acesso à memória e desvios. Os modos relevantes para as instruções implementadas neste projeto incluem:

- **Endereçamento por Registrador (*Register Direct*):** Os operandos estão diretamente contidos em registradores especificados na instrução. É o caso das instruções aritméticas e lógicas do tipo R (ex: `add rd, rs1, rs2`), onde `rs1` e `rs2` são as fontes.
- **Endereçamento Imediato (*Immediate*):** Um dos operandos é um valor constante (imediato) embutido na própria instrução. Exemplos incluem `addi rd, rs1, imm`, onde `imm` é o operando imediato.
- **Endereçamento Base + Deslocamento:** Utilizado pelas instruções de acesso à memória `lw` e `sw`. O endereço efetivo da memória é calculado somando o conteúdo de um registrador base (especificado por `rs1`) com um valor de deslocamento (imediato) presente na instrução. Por exemplo, em `lw rd, offset(rs1)`, o endereço é `Regs[rs1] + sign_extend(offset)`.
- **Endereçamento Relativo ao PC (*PC-Relative*):** Usado pelas instruções de desvio condicional (tipo B, como `beq`). O endereço de destino do desvio é calculado

somando o valor atual do Contador de Programa (PC) com um *offset* imediato (estendido com sinal e multiplicado por 2, pois os imediatos de desvio são em unidades de meia-palavra) contido na instrução.

- **Endereçamento Indireto por Registrador com Deslocamento (*Register-Indirect with Offset for Jumps*):** A instrução `jalr` (*Jump and Link Register*) utiliza este modo. O novo PC é calculado somando o conteúdo de um registrador base (`rs1`) com um *offset* imediato de 12 bits (estendido com sinal).

A especificação completa dos modos de endereçamento pode ser encontrada em (1).

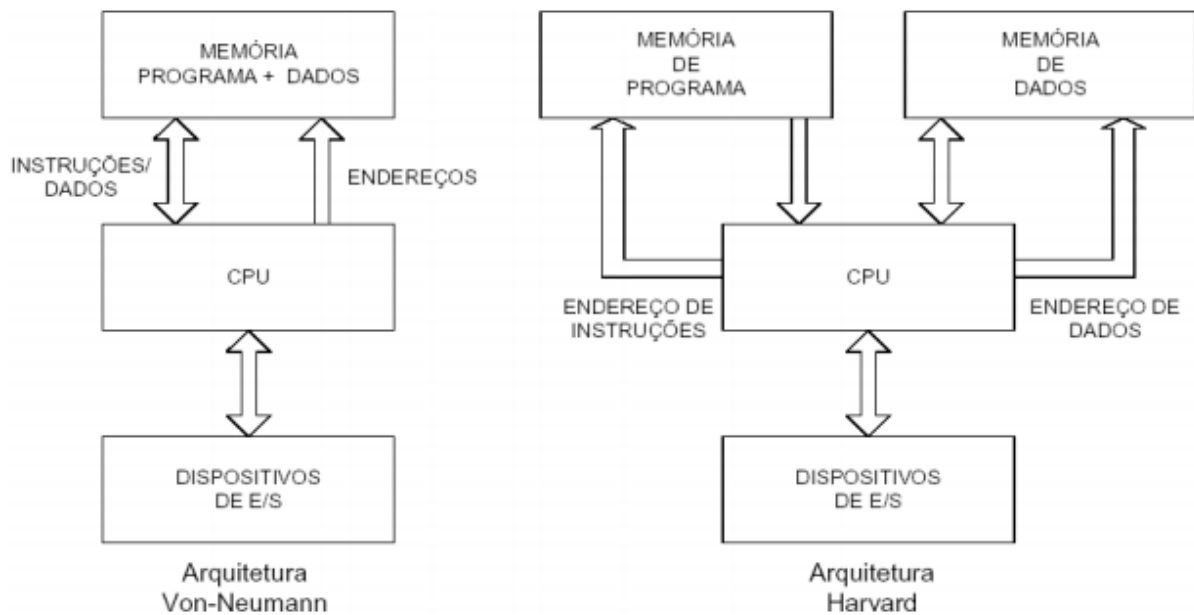
3.5 Arquitetura Harvard vs. Von Neumann

Duas arquiteturas fundamentais de organização de memória em sistemas computacionais são a de Von Neumann e a de Harvard.

A **arquitetura de Von Neumann**, proposta por John von Neumann, utiliza um único espaço de endereçamento e um único barramento para dados e instruções. Isso significa que a CPU busca instruções e dados da mesma unidade de memória através do mesmo caminho. A principal vantagem é a simplicidade no design da memória e do barramento, e a flexibilidade de usar a memória para dados ou instruções conforme necessário. No entanto, o compartilhamento do barramento cria um gargalo, conhecido como "gargalo de Von Neumann", pois instruções e dados não podem ser acessados simultaneamente, limitando a taxa de transferência (3).

A **arquitetura de Harvard**, por outro lado, utiliza espaços de endereçamento e barramentos fisicamente separados para instruções e dados. Isso permite que a busca de uma instrução ocorra em paralelo com o acesso (leitura ou escrita) a um dado na memória, aumentando a vazão e o desempenho, especialmente em sistemas com *pipeline*. É comumente encontrada em microcontroladores e processadores de sinais digitais (DSPs) (5). O processador projetado neste trabalho, ao possuir uma Memória de Instrução e uma Memória de Dados distintas no seu datapath (Figura 3), alinha-se conceitualmente com os benefícios da arquitetura de Harvard, permitindo a busca da próxima instrução enquanto a instrução atual acessa a memória de dados, embora em um design monociclo esses paralelismos não sejam explorados em sua totalidade como em um pipeline.

Figura 3 – Diferença entre Arquitetura de Harvard e de Von Neumann



Fonte: (6)

3.6 Processadores Monociclo vs. Multiciclo

A execução de uma instrução em um processador pode ser realizada em um único ciclo de clock (monociclo) ou em múltiplos ciclos de clock (multiciclo).

Um **processador monociclo**, como o projetado neste trabalho, executa cada instrução completa em um único ciclo de clock. A duração do ciclo de clock é determinada pelo caminho crítico da instrução mais longa (geralmente uma instrução de acesso à memória). A principal vantagem é a simplicidade do design da unidade de controle. No entanto, como todas as instruções levam o mesmo tempo para executar, instruções mais simples (como uma adição) acabam subutilizando o ciclo de clock, o que pode levar a um desempenho geral inferior comparado a outras abordagens (3).

Um **processador multiciclo** divide a execução de cada instrução em múltiplas etapas (ex: busca, decodificação, execução, acesso à memória, escrita), onde cada etapa leva um ciclo de clock. Isso permite que instruções diferentes levem um número diferente de ciclos para completar, e o ciclo de clock pode ser menor, ditado pela etapa mais longa. Além disso, unidades funcionais (como ULA e memória) podem ser reutilizadas em diferentes etapas para diferentes instruções, potencialmente reduzindo a quantidade de hardware. Embora mais complexo de controlar, um design multiciclo pode oferecer melhor desempenho e ser um passo intermediário para arquiteturas com *pipeline* (5). A escolha por um design monociclo neste projeto foi motivada pela simplicidade inicial de implementação e compreensão dos fluxos de dados fundamentais.

3.7 Componentes do *Datapath* Monociclo

O processador implementado neste projeto segue um *datapath* de monociclo, onde cada instrução é executada em um único ciclo de *clock*. Os principais componentes deste *datapath* incluem:

- **PC (*Program Counter*)**: Mantém o endereço da próxima instrução a ser executada.
- **Memória de Instrução**: Armazena as instruções do programa.
- **Banco de Registradores**: Contém 32 registradores de propósito geral.
- **Extensor de Sinal**: Expande valores imediatos para 32 bits, mantendo o sinal.
- **ULA (*Unidade Lógica e Aritmética*)**: Executa operações aritméticas e lógicas.
- **Memória de Dados**: Armazena e recupera valores da memória durante execuções de *load/store*.
- **Unidade de Controle**: Gera sinais de controle baseados na instrução atual.
- **MUXes**: Seletores que definem o fluxo de dados conforme os sinais de controle.

Esses componentes são conectados para formar um caminho de dados capaz de processar qualquer instrução definida na ISA base e extensões implementadas (3) (5).

3.8 *Memory-Mapped I/O* (MMIO)

Memory-Mapped I/O (MMIO) é uma técnica utilizada para permitir que dispositivos periféricos sejam acessados como se fossem posições de memória. Em vez de usar instruções específicas de entrada e saída, o processador interage com os periféricos por meio de operações de leitura e escrita em endereços previamente reservados para esse fim.

No contexto da arquitetura RISC-V, isso significa que regiões da memória de dados são atribuídas a componentes como temporizadores, controladores de teclado, *displays*, entre outros, facilitando o *design* e a integração com o *hardware* externo (1).

A aplicação de MMIO no contexto RISC-V segue os princípios gerais de acesso à memória, sendo uma técnica comum em diversas arquiteturas (3)(5).

3.9 Linguagem de Descrição de Hardware: Verilog

Para a implementação do processador neste projeto, foi utilizada a linguagem de descrição de hardware (HDL) Verilog. Verilog é uma linguagem padronizada (IEEE 1364)

amplamente utilizada na indústria e academia para modelar, simular e sintetizar circuitos digitais, desde componentes simples até sistemas complexos em um chip (SoCs) (7).

A linguagem permite descrever a estrutura de um circuito (como os módulos são interconectados) e seu comportamento (como as saídas são geradas em função das entradas e do estado interno). Módulos em Verilog possuem portas de entrada e saída bem definidas, e a lógica interna pode ser descrita em diferentes níveis de abstração, incluindo comportamental (usando blocos `always`, `if-else`, `case`), fluxo de dados (usando atribuições contínuas `assign`) e estrutural (instanciando outros módulos). Para este projeto, a descrição comportamental e de fluxo de dados foram predominantemente utilizadas para implementar os componentes da unidade de processamento. A simulação funcional do código Verilog, realizada com ferramentas como ModelSim, é uma etapa crucial para verificar a correção do design antes da síntese para hardware físico, como o FPGA.

4 Desenvolvimento

4.1 Conjunto de Instruções Implementado

Este capítulo detalha o projeto e a implementação da unidade de processamento do processador RISC-V RV32IM. Inicialmente, são reapresentados o conjunto de instruções, seus formatos e a arquitetura do datapath concebidos no Ponto de Checagem 1 (PC1), que servem como especificação para a implementação em hardware. Em seguida, é descrita a implementação em Verilog dos principais módulos da unidade de processamento e sua subsequente integração. As instruções foram organizadas em grupos conforme sua funcionalidade e tipo de codificação(1)(2). A Tabela 1 apresenta as operações aritméticas e lógicas do tipo R que operam entre registradores.

Tabela 1 – Operações Aritméticas e Lógicas (Tipo R)

Instrução	Função	Formato	Tipo
add	Soma entre registradores	$rd \leftarrow rs1 + rs2$	R
sub	Subtração entre registradores	$rd \leftarrow rs1 - rs2$	R
and	E lógico entre registradores	$rd \leftarrow rs1 \& rs2$	R
or	OU lógico entre registradores	$rd \leftarrow rs1 \mid rs2$	R
xor	OU exclusivo entre registradores	$rd \leftarrow rs1 \oplus rs2$	R
sll	Deslocamento lógico à esquerda	$rd \leftarrow rs1 \ll rs2$	R
srl	Deslocamento lógico à direita	$rd \leftarrow rs1 \gg rs2$	R
sra	Deslocamento aritmético à direita	$rd \leftarrow rs1 \ggg rs2$	R
slt	menor que (signed)	$rd \leftarrow rs1 < rs2$	R

Fonte: A autora

As operações aritméticas e lógicas que utilizam um valor imediato como um dos operandos são do tipo I, conforme detalhado na Tabela 2.

Tabela 2 – Aritmética com Imediatos (Tipo I)

Instrução	Função	Formato	Tipo
addi	Soma com imediato	$rd \leftarrow rs1 + imm$	I
andi	E lógico com imediato	$rd \leftarrow rs1 \& imm$	I
ori	OU lógico com imediato	$rd \leftarrow rs1 \mid imm$	I
xori	OU exclusivo com imediato	$rd \leftarrow rs1 \oplus imm$	I
slti	Define menor que (signed) com imediato	$rd \leftarrow (rs1 < imm)$	I

Fonte: A autora

O acesso à memória para carregar (load) e armazenar (store) palavras de 32 bits é

realizado por instruções dos tipos I e S, respectivamente, conforme mostrado na Tabela 3.

Tabela 3 – Acesso à Memória (Tipos I e S)

Instrução	Função	Formato	Tipo
lw	Carrega palavra da memória	$rd \leftarrow \text{Mem}[rs1 + \text{offset}]$	I
sw	Armazena palavra na memória	$\text{Mem}[rs1 + \text{offset}] \leftarrow rs2$	S

Fonte: A autora

As instruções de controle de fluxo, responsáveis por desvios condicionais (tipo B) e saltos incondicionais (tipo J e I para ‘jalr’), são apresentadas na Tabela 4.

Tabela 4 – Controle de Fluxo (Tipos B e J)

Instrução	Função	Formato	Tipo
beq	Desvia se igual	$\text{if } (rs1 == rs2) \text{ PC} += \text{offset}$	B
bne	Desvia se diferente	$\text{if } (rs1 != rs2) \text{ PC} += \text{offset}$	B
blt	Desvia se menor (signed)	$\text{if } (rs1 < rs2) \text{ PC} += \text{offset}$	B
bge	Desvia se maior ou igual (signed)	$\text{if } (rs1 \geq rs2) \text{ PC} += \text{offset}$	B
jal	Salto e link	$rd \leftarrow \text{PC}+4; \text{PC} += \text{offset}$	J
jalr	Salto indireto e link	$rd \leftarrow \text{PC}+4; \text{PC} \leftarrow rs1 + \text{offset}$	I

Fonte: A autora

A extensão M simplificada implementada neste processador adiciona suporte para operações de multiplicação e divisão inteiras assinadas, detalhadas na Tabela 5. Estas são instruções do tipo R.

Tabela 5 – Multiplicação e Divisão (RV32M Simplificado)

Instrução	Função	Formato	Tipo
mul	Multiplicação (signed)	$rd \leftarrow rs1 * rs2$	R
div	Divisão (signed)	$rd \leftarrow rs1 / rs2$	R
rem	Resto da divisão (signed)	Resto da divisão	R

Fonte: A autora

4.2 Formatos e Codificação das Instruções

A arquitetura RISC-V adota diferentes formatos de instrução, projetados para acomodar os operandos e campos necessários a cada tipo específico de operação. No processador desenvolvido neste projeto, foram implementadas instruções que utilizam os formatos R, I, S, B e J. Cada formato possui uma estrutura distinta, que define a disposição dos campos como registradores-fonte e destino, imediatos, e códigos de operação. As tabelas 6 a 10 ilustram a organização desses formatos.

As tabelas a seguir detalham a codificação binária específica (campos **opcode**, **funct3** e **funct7**, quando aplicável) para cada instrução suportada pelo processador projetado, agrupadas por formato. Todos os valores binários são mostrados com o bit mais significativo à esquerda. Os opcodes, funct3 e funct7 aqui apresentados seguem a codificação padrão definida na especificação oficial da ISA RISC-V (1). O cartão de referência (8) também foi uma fonte útil para consulta rápida desses valores.

A Tabela 6 detalha a codificação das instruções do tipo R implementadas, incluindo as da extensão M simplificada.

Tabela 6 – Codificação das Instruções Tipo R Implementadas.

Instrução	Formato	Opcode	Funct3	Funct7
add	R	0110011	000	0000000
sub	R	0110011	000	0100000
sll	R	0110011	001	0000000
slt	R	0110011	010	0000000
xor	R	0110011	100	0000000
srl	R	0110011	101	0000000
sra	R	0110011	101	0100000
or	R	0110011	110	0000000
and	R	0110011	111	0000000
<i>Extensão M (Simplificada)</i>				
mul	R	0110011	000	0000001
div	R	0110011	100	0000001
rem	R	0110011	110	0000001

Fonte: A autora

Para as instruções do tipo I, que envolvem operações com imediatos e a instrução de carga ‘lw’, a codificação é apresentada na Tabela 7.

Tabela 7 – Codificação das Instruções Tipo I Implementadas (Imediato e Load).

Instrução	Formato	Opcode	Funct3
<i>Operações com Imediato</i>			
addi	I	0010011	000
slti	I	0010011	010
xori	I	0010011	100
ori	I	0010011	110
andi	I	0010011	111
<i>Acesso à Memória (Load)</i>			
lw	I	0000011	010

Fonte: A autora

A instrução de armazenamento ‘sw’, do tipo S, possui a codificação mostrada na

Tabela 8.

Tabela 8 – Codificação das Instruções Tipo S Implementadas (Store).

Instrução	Formato	Opcode	Funct3
sw	S	0100011	010

Fonte: A autora

As instruções de desvio condicional, do tipo B, são codificadas conforme a Tabela 9.

Tabela 9 – Codificação das Instruções Tipo B Implementadas (Branch Condicional).

Instrução	Formato	Opcode	Funct3
beq	B	1100011	000
bne	B	1100011	001
blt	B	1100011	100
bge	B	1100011	101

Fonte: A autora

Finalmente, a Tabela 10 apresenta a codificação para as instruções de salto (‘jal’, ‘jalr’) e as instruções de sistema (‘ecall’, ‘nop’) não serão implementadas nessa etapa.

Tabela 10 – Codificação das Instruções Tipo J e I Implementadas (Saltos e Sistema).

Instrução	Formato	Opcode	Funct3	Imm[11:0] ou Comentário
jal	J	1101111	N/A	Campo Imm codifica offset
jalr	I	1100111	000	Campo Imm codifica offset

Fonte: A autora

Além disso, a figura 4 ilustra a composição completa de cada formato no projeto aqui proposto.

Figura 4 – Formatos de Instrução RISC-V (RV32I/M) utilizados

Formato R: add, sub, and, or, xor, sll, srl, sra, slt, mul, div, rem

funct7	rs2	rs1	funct3	rd	opcode
[31–25]	[24–20]	[19–15]	[14–12]	[11–7]	[6–0]

Formato I: addi, slti, xori, ori, andi, lw, jalr

imm[11:0]	rs1	funct3	rd	opcode
[31–20]	[19–15]	[14–12]	[11–7]	[6–0]

Formato S: sw

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
[31–25]	[24–20]	[19–15]	[14–12]	[11–7]	[6–0]

Formato B: beq, bne, blt, bge

imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
[31 30–25]	[24–20]	[19–15]	[14–12]	[11–8 7]	[6–0]

Formato J: jal

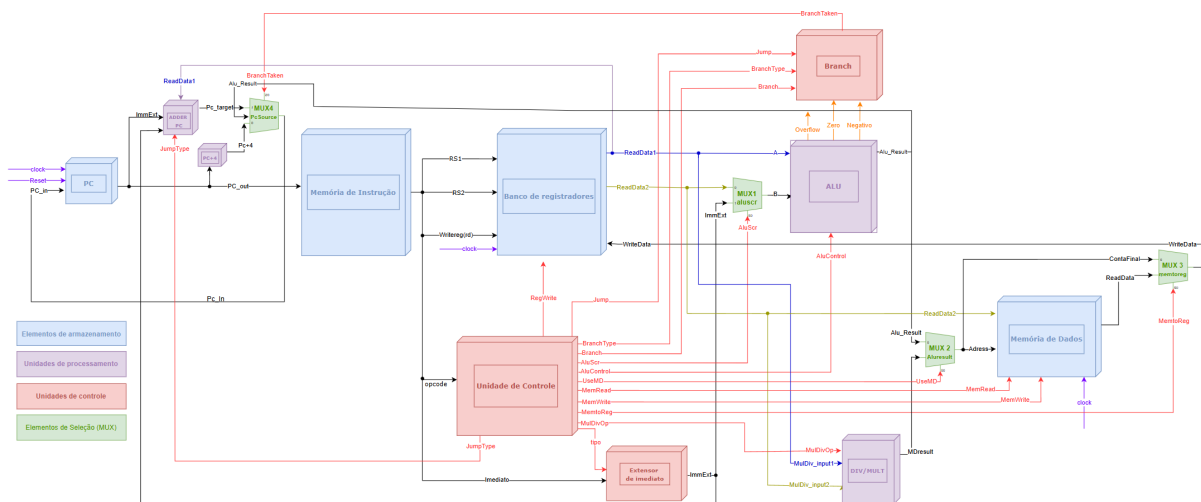
imm[20 10:1 11 19:12]	rd	opcode
[31 30–21 20 19–12]	[11–7]	[6–0]

4.3 Datapath do Processador

A implementação do processador RISC-V proposta neste projeto segue uma arquitetura mononúcleo com suporte ao conjunto de instruções RV32IM reduzido, incluindo acesso à memória, controle de fluxo, operações aritméticas, lógicas e de multiplicação/divisão. O *datapath* projetado integra os principais blocos funcionais necessários para execução dessas instruções, como unidades de controle, memória, banco de registradores, unidade lógica e aritmética (ALU), extensores de imediato, módulos de entrada e saída (MMIO), além de elementos de seleção como multiplexadores (MUX). A Figura 5 apresenta o diagrama completo do *datapath* implementado, com destaque para os caminhos de dados e sinais de controle. A unidade foi projetada para as operações simplificadas da extensão M, inspirando-se em implementações abertas como (9).

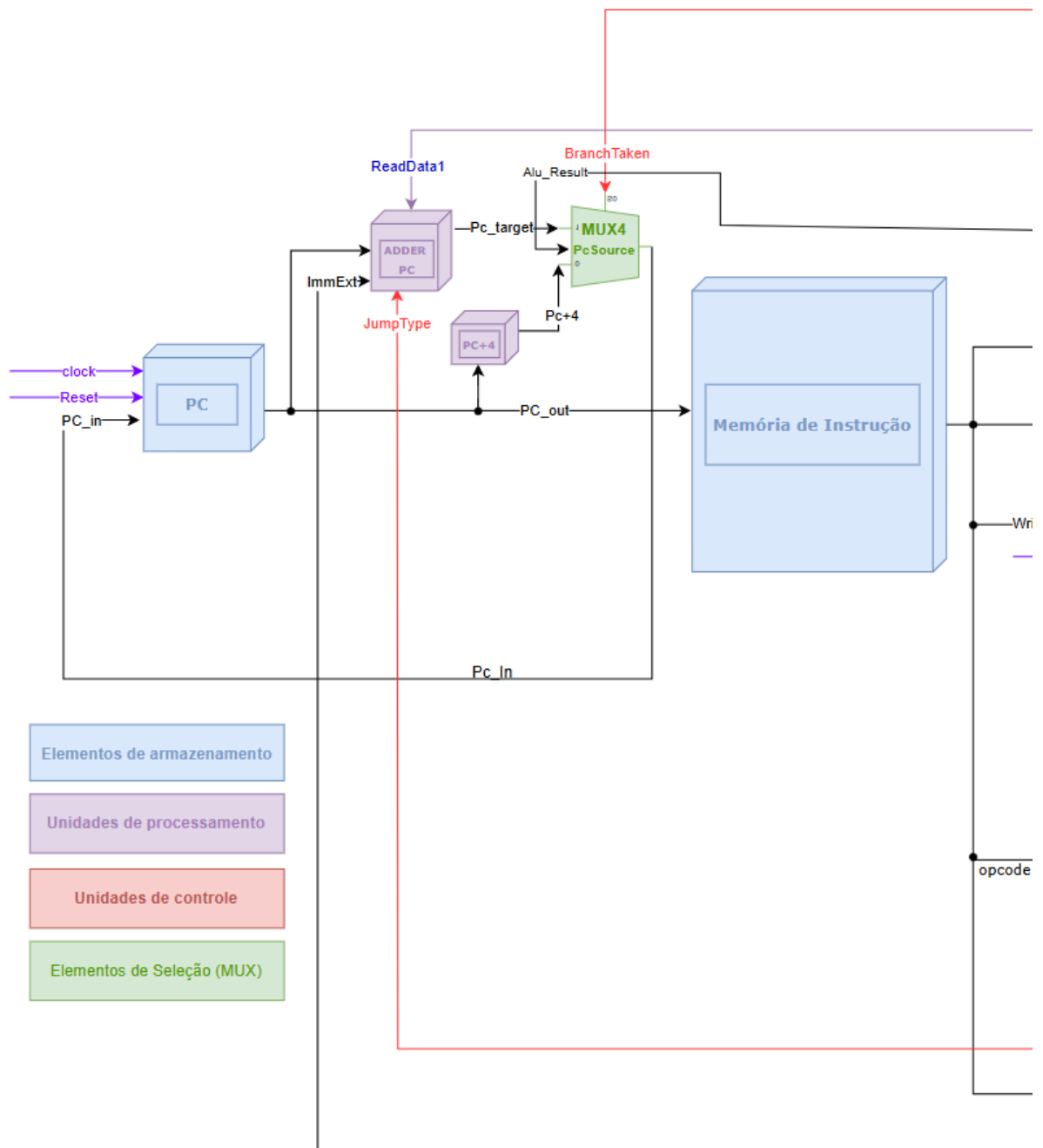
Para facilitar a visualização e compreensão, o datapath completo da Figura 5 foi dividido em três partes. A Figura 6 ilustra a seção inicial do processador, abrangendo o Contador de Programa (PC) e a lógica de busca de instrução, bem como a conexão com a Unidade de Controle e o Banco de Registradores. A Figura 7 foca na unidade de execução principal, incluindo o Banco de Registradores, a Unidade Lógica e Aritmética (ULA), a unidade de multiplicação/divisão e o Extensor de Imediato. Por fim, a Figura 8 detalha a interface com a Memória de Dados e os módulos de Entrada/Saída Mapeados em Memória (MMIO). A descrição detalhada dos componentes e do fluxo de dados nas subseções seguintes fará referência a essas figuras segmentadas quando apropriado, além

da visão geral da Figura 5.



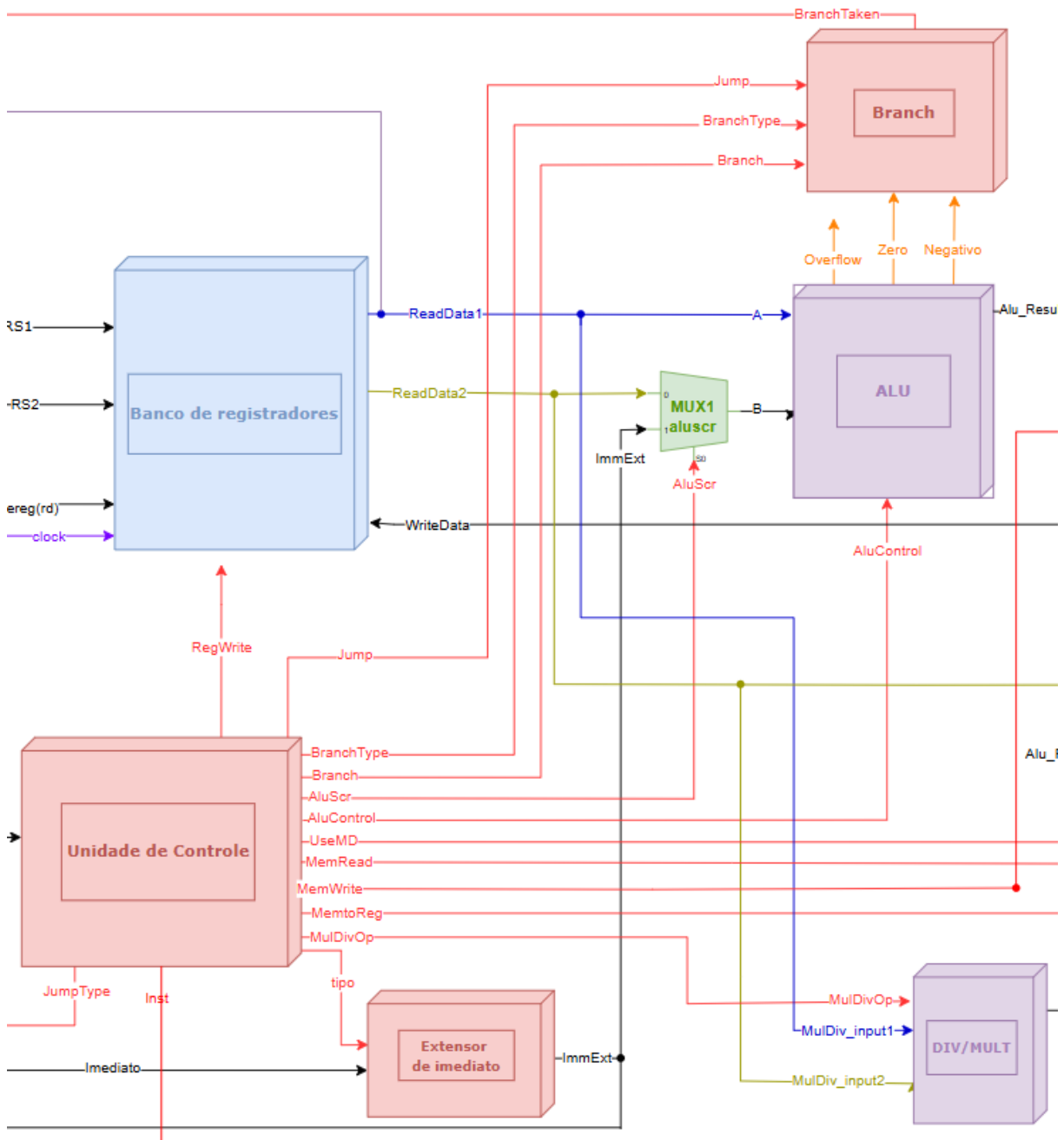
Fonte: A autora

Figura 6 – Diagrama 1/3 do Datapath do Processador RV32IM Monociclo



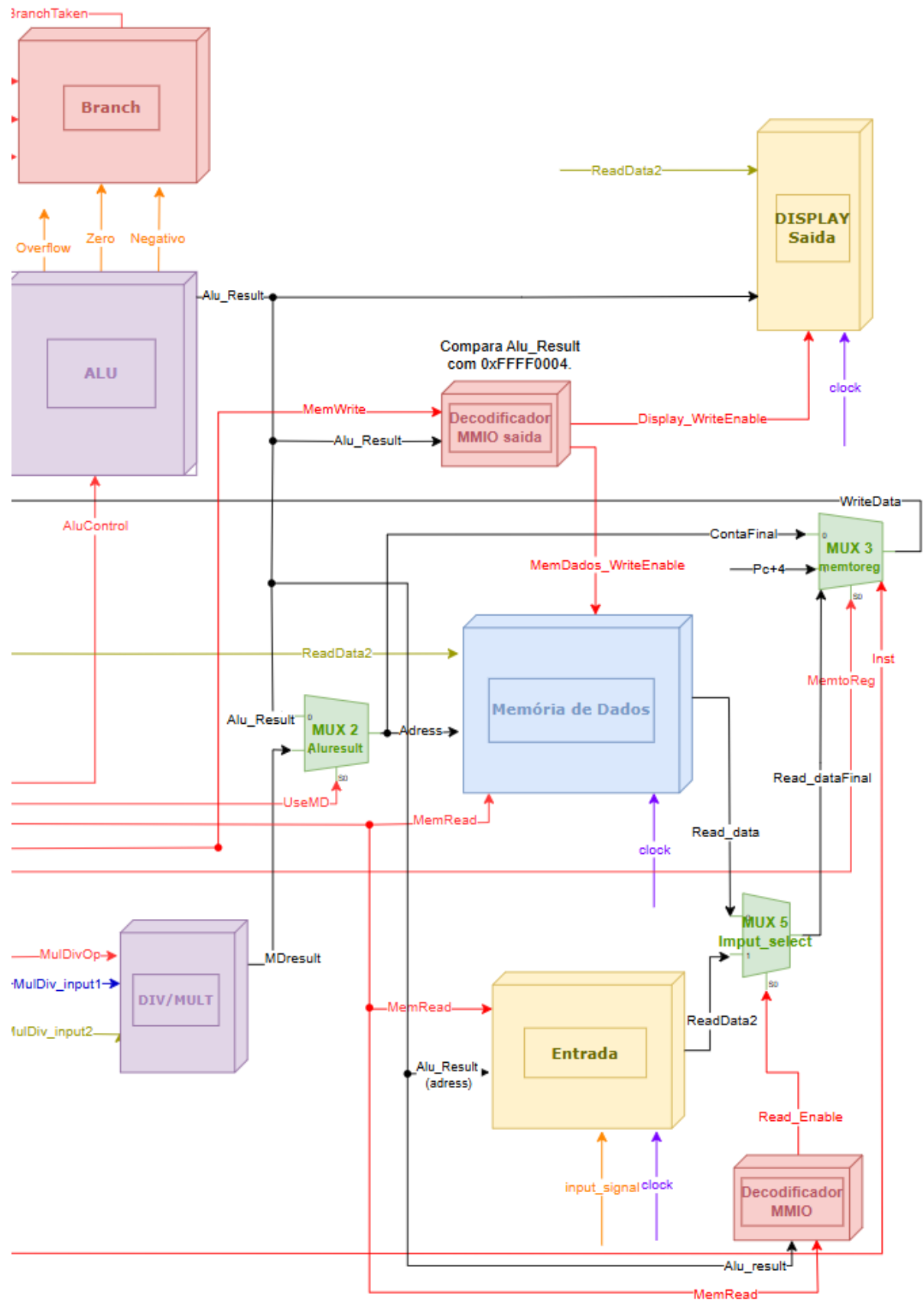
Fonte: A autora

Figura 7 – Diagrama 2/3 do Datapath do Processador RV32IM Monociclo



Fonte: A autora

Figura 8 – Diagrama 3/3 do Datapath do Processador RV32IM Monociclo



Fonte: A autora

4.3.1 Descrição dos Componentes Principais (Referenciando Figura 5)

A arquitetura do processador RV32IM monociclo projetado, ilustrada na Figura 5 (e em detalhe nas Figuras 6, 7 e 8), é composta pelos seguintes módulos principais:

4.3.1.1 Bloco PC e Lógica Associada (Figura 6)

- **PC (Contador de Programa):** Registrador de 32 bits que armazena o endereço da instrução corrente a ser buscada na Memória de Instrução. É um componente sequencial, atualizado na borda de subida do sinal de *clock*. Recebe o próximo endereço a ser carregado (*PC_in*, proveniente da saída do MUX4) e um sinal de *Reset* que, quando ativo, inicializa o PC com um endereço predefinido (geralmente o início do programa), desde que *enable* ativo, que é a negação de *'pc_stall'*, cuja função é parar o PC para a entrada do usuário antes do *'enter'*. A saída do PC (*PC_out*) é utilizada para a próxima instrução.
- **Somador PC+4:** Unidade combinacional que calcula o endereço da próxima instrução em um fluxo sequencial. Recebe *PC_out* como entrada e gera como saída *PC_out + 4*. Este resultado (*Pc+4*) é uma das fontes de entrada para o MUX4 (para execução normal) e também para o MUX3 (para salvar o endereço de retorno nas instruções *jal* e *jalr*).
- **Somador de Endereço de Destino para Desvios/Saltos Diretos (PC Target Adder):** Bloco combinacional (referido como "ADDER PC" na Figura 6) que calcula o endereço de destino para instruções de desvio condicional (Tipo B) e saltos incondicionais diretos (Tipo J, como *jal*). Ele soma o valor atual do PC (*PC_out*) com o valor do imediato estendido com sinal (*ImmExt*), que representa o deslocamento relativo. A saída deste somador é o *Pc_target*.
- **MUX4 (PcSource):** Multiplexador de múltiplas entradas que seleciona qual será o próximo valor a ser carregado no PC (*PC_in*). Suas entradas principais são:
 1. O endereço sequencial *Pc+4* (vindo do Somador PC+4).
 2. O endereço de destino calculado *Pc_target* (vindo do Somador PC Target Adder, para desvios e *jal*).
 3. O resultado da ULA, *Alu_Result* (para a instrução *jalr*, que calcula *rs1 + imm*). (Esta entrada pode não estar explicitamente detalhada na Figura 6, mas é essencial para a funcionalidade completa do JALR no datapath da Figura 3).

A seleção no MUX4 é controlada por sinais gerados pela Unidade de Controle e pela Unidade de Branch, como *BranchTaken* (para desvios condicionais) e *Jump* (para saltos incondicionais como *jal* e *jalr*), para decidir entre execução sequencial, desvio tomado, ou salto.

4.3.1.2 Banco de Registradores (Figura 7):

Armazena os 32 registradores de propósito geral (x0-x31). Recebe os endereços dos registradores a serem lidos (rs1, rs2) e escritos (rd) da instrução. Recebe o dado a ser escrito (*WriteData* do MUX3) e o sinal de habilitação de escrita (*RegWrite* da Unidade de Controle), além do *clock*. Fornece os dados lidos (*ReadData1*, *ReadData2*) para a ULA, Unidade de *Branch*, Memória de Dados (para sw) e Display (para escrita MMIO).

4.3.1.3 Extensor de Imediato (Figura 7):

Recebe o campo imediato da instrução e o estende (geralmente com sinal) para 32 bits (ImmExt). O valor estendido é usado no cálculo de endereço (ULA), no cálculo de desvio (Somador PC Target) e como operando para instruções tipo I (via MUX1).

4.3.1.4 ULA (ALU - Unidade Lógica e Aritmética) (Figura 7)

A Unidade Lógica e Aritmética (ULA) é o componente central de execução do processador, responsável por realizar uma variedade de operações sobre os dados fornecidos. Conforme ilustrado na Figura 7, a ULA recebe dois operandos de 32 bits: o primeiro (*OperandA*) vindo diretamente do Banco de Registradores (*ReadData1*), e o segundo (*OperandB*) vindo da saída do MUX1 (*alusr*), que seleciona entre *ReadData2* (para operações Tipo R) ou o imediato estendido *ImmExt* (para operações Tipo I, S e B).

As operações executadas pela ULA são determinadas pelo sinal de controle de múltiplos bits *AluControl*, gerado pela Unidade de Controle com base na instrução decodificada. As funcionalidades implementadas pela ULA neste processador incluem:

- **Operações Aritméticas:** Adição (*add*, *addi*) e Subtração (*sub*).
- **Operações Lógicas:** AND (*and*, *andi*), OR (*or*, *ori*), e XOR (*xor*, *xori*).
- **Operações de Comparação:** "Set Less Than" com sinal (*slt*, *slti*), que produz 1 se o primeiro operando for menor que o segundo (considerando números em complemento de dois), e 0 caso contrário.
- **Operações de Deslocamento:** Deslocamento lógico à esquerda (*sll*), deslocamento lógico à direita (*srl*), e deslocamento aritmético à direita (*sra*). A quantidade de deslocamento para estas operações (no caso de instruções Tipo R) é fornecida pelos bits inferiores do segundo operando.
- **Cálculo de Endereço para Acesso à Memória:** Para as instruções *lw* (Load Word) e *sw* (Store Word), a ULA é utilizada para calcular o endereço efetivo da memória somando o conteúdo de um registrador base (*ReadData1*) com o offset imediato estendido (*ImmExt* vindo do MUX1).

- **Cálculo de Endereço para a Instrução `jalr` (Jump and Link Register):** A ULA também é fundamental para a instrução `jalr`. Neste caso, ela soma o conteúdo do registrador base especificado por `rs1` (`ReadData1`) com o offset imediato de 12 bits estendido com sinal (`ImmExt` vindo do `MUX1`). O resultado desta soma, `Alu_Result`, representa o endereço de destino para o qual o PC será atualizado. Conforme a especificação RISC-V, o bit menos significativo deste endereço calculado é zerado antes de ser carregado no PC.

A saída principal da ULA é o `Alu_Result` de 32 bits, que representa o resultado da operação executada. Este `Alu_Result` tem múltiplos destinos no datapath:

- É uma das entradas do `MUX2` (`Aluresult`), para ser potencialmente selecionado como o dado a ser escrito de volta no Banco de Registradores (via `MUX3`).
- Serve como endereço para a Memória de Dados nas instruções `lw` e `sw`.
- Para a instrução `jalr`, o `Alu_Result` (após o zeramento do LSB) é o endereço de destino que será selecionado pelo `MUX4` (`PcSource`) para atualizar o PC.

Além do resultado principal, a ULA gera flags de status, como a `ZeroFlag` (ativa se `Alu_Result` for zero), `OverflowFlag` e `NegativeFlag`. A `ZeroFlag` é utilizada pela Unidade de Branch para determinar a condição de desvios como `beq` e `bne` (onde `bne` usa a negação da `ZeroFlag` após uma subtração).

4.3.1.5 MUX1 (`aluscr`) (Figura 7):

Seleciona o segundo operando para a ULA (entrada B). Escolhe entre `ReadData2` (vindo do Banco de Registradores, para operações *R-Type*) e `ImmExt` (vindo do Extensor, para operações *I-Type*, *S-Type*, *B-Type*). É controlado pelo sinal `AluScr` da Unidade de Controle.

4.3.1.6 Unidade de Controle (Figura 7):

Decodifica a instrução (recebendo opcode e, implicitamente, `funct3/funct7`) e gera todos os sinais de controle necessários para coordenar os outros componentes. Suas saídas incluem `RegWrite`, `Jump`, `BranchType`, `Branch`, `AluScr`, `AluControl`, `UseMD`, `MemRead`, `MemWrite`, `MemtoReg`, `MulDivOp`, `JumpType`, e `Inst` (segundo bit de controle para `MUX3`).

4.3.1.7 Unidade de Branch (Figura 7):

Compara `ReadData1` e `ReadData2` com base no `BranchType` (`beq`, `bne`, `blt`, `bge`) vindo da Unidade de Controle. Usa o resultado da comparação e o sinal `Branch` (da

Unidade de Controle) para determinar se o desvio deve ser tomado ($BranchTaken = Branch \text{ AND } ConditionMet$). A saída $BranchTaken$ influencia a seleção no MUX4.

4.3.1.8 Unidade de Multiplicação e Divisão (Figura 7):

Realiza as operações de multiplicação (mul), divisão inteira (div) e resto (rem) especificadas pela extensão M simplificada. Recebe operandos ($MulDiv_input1$, $MulDiv_input2$ - que devem ser conectados a $ReadData1$ e $ReadData2$) e o tipo de operação ($MulDivOp$ da Unidade de Controle). A saída é $MDresult$.

4.3.1.9 MUX2 (Aluresult) (Figura 8):

Seleciona entre o resultado da ULA (Alu_Result) e o resultado da Unidade ($MDresult$). É controlado pelo sinal $UseMD$ da Unidade de Controle. Sua saída é $ContaFinal$. (Que podeseer integrado ao MUX3).

4.3.1.10 Memória de Dados (Figura 8):

Armazena os dados do programa. Recebe o endereço ($Adress$ vindo de Alu_Result), o dado a ser escrito ($WriteData$ vindo de $ReadData2$), um sinal de habilitação de escrita ($MemDados_WriteEnable$ do Decodificador MMIO Saída), um sinal de habilitação de leitura ($MemRead$ da Unidade de Controle, usado internamente para habilitar a saída) e o $clock$. Fornece o dado lido ($Read_data$) para o MUX5.

4.3.1.11 Decodificador MMIO Saída (Figura 8):

Lógica combinacional que verifica se um endereço de escrita (Alu_Result) corresponde ao endereço mapeado do $Display$ (2040) quando $MemWrite$ (original da Unidade de Controle) está ativo. Gera $Display_WriteEnable$ (ativo somente para o endereço do display) e $MemDados_WriteEnable$ (ativo para outros endereços quando $MemWrite$ está ativo).

4.3.1.12 Display Saída (Figura 8):

Representa a saída ($display$ 7 segmentos). Recebe o dado ($ReadData2$), a habilitação de escrita específica ($Display_WriteEnable$) e o $clock$ para registrar o valor a ser exibido. ($DISPLAY_ADDRESS = 2044$)

4.3.1.13 Bloco Entrada (Figura 8):

Representa a entrada mapeada em memória. Recebe o endereço (Alu_Result), o sinal $MemRead$ (para saber se uma leitura está ocorrendo neste endereço - via Decodificador MMIO), o sinal de entrada externo ($input_signal$) e o $clock$. Sua saída ($ReadData2$

no diagrama, idealmente *Input_ReadData*) contém o valor lido do periférico quando selecionado.

4.3.1.14 Decodificador MMIO (Entrada) (Figura 8):

Lógica combinacional que verifica se um endereço de leitura (*Alu_Result*) corresponde ao endereço mapeado do Bloco Entrada (0xFFFF0000) quando *MemRead* (da Unidade de Controle) está ativo. Gera o sinal *SelecDado* para o MUX5.

4.3.2 Interfaces MMIO

4.3.2.0.1 Dispositivo de Entrada (*input_device.v*):

O módulo *input_device.v* foi desenvolvido para simular, via interface MMIO, um periférico de entrada baseado em switches físicos. .

4.3.2.1 MUX5 (*Input_select*) (Figura 8):

Seleciona a fonte do dado lido que irá para o MUX3. Escolhe entre *Read_data* (vindo da Memória de Dados principal) e *ReadData2* (vindo do Bloco Entrada). É controlado por *SelecDado*. Sua saída é *Read_dataFinal*.

4.3.2.2 MUX3 (*memtoreg*) (Figura 8):

Multiplexador final que seleciona qual valor será escrito de volta no Banco de Registradores. Recebe *ContaFinal* (resultado da ULA ou , via MUX2), *Pc+4* (para salvar endereço de retorno de *jal/jalr*) e *Read_dataFinal* (dado lido da memória ou MMIO, via MUX5). É controlado pelos sinais *MemtoReg* e *Inst* (vindos da Unidade de Controle). Sua saída é *WriteData*.

4.3.3 Fluxo de Dados e Sinais de Controle por Tipo de Instrução

1. *add rd, rs1, rs2* (Tipo R - ULA)

- Instrução buscada (Figura 6).
- Unidade de Controle (Figura 7) decodifica, ativa *RegWrite*=1, *AluSrc*=0, *MemtoReg*=0, *Inst*=0 (seleciona *ContaFinal* no MUX3), *UseMD*=0 (seleciona *Alu_Result* no MUX2), e *AluControl* para operação ADD.
- Banco de Registradores lê *rs1* (*ReadData1*) e *rs2* (*ReadData2*).
- *ReadData1* vai para ULA (A). *ReadData2* passa pelo MUX1 (selecionado por *AluSrc*=0) para ULA (B).
- ULA calcula A+B, resultado vai para *Alu_Result*.

- *Alu_Result* passa pelo MUX2 (*UseMD=0*) para *ContaFinal*.
- *ContaFinal* passa pelo MUX3 (*MemtoReg=0*, *Inst=0*) para *WriteData*.
- *WriteData* é escrito no registrador rd do Banco de Registradores (habilitado por *RegWrite=1*).
- PC é atualizado para $PC+4$ (MUX4 seleciona saída do somador $PC+4$).

2. lw rd, offset(rs1) (Tipo I - Load)

- Instrução buscada (Figura 6).
- Unidade de Controle (Figura 7) decodifica, ativa *RegWrite=1*, *AluSrc=1*, *MemRead=1*, *MemtoReg=0*, *Inst=1* (seleciona *Read_dataFinal* no MUX3).
- Banco de Registradores lê rs1 (*ReadData1*).
- Extensor estende *offset* para *ImmExt*.
- *ReadData1* vai para ULA (A). *ImmExt* passa pelo MUX1 (*AluSrc=1*) para ULA (B).
- ULA calcula A+B (endereço), resultado vai para *Alu_Result*.
- *Alu_Result* vai como *Adress* para Memória de Dados (e Decodificadores MMIO) (Figura 8).
- Unidade de Controle ativa *MemRead=1*.
- Decodificador MMIO (Entrada) verifica endereço; se não for o endereço de entrada, *SelecDado=0*.
- Memória de Dados lê no *Adress* e coloca resultado em *Read_data*.
- *Read_data* vai para MUX5. MUX5 seleciona (*SelecDado=0*) *Read_data* para *Read_dataFinal*.
- *Read_dataFinal* passa pelo MUX3 (*MemtoReg=0*, *Inst=1*) para *WriteData*.
- *WriteData* é escrito no registrador rd (habilitado por *RegWrite=1*).
- PC é atualizado para $PC+4$.

3. sw rs2, offset(rs1) (Tipo S - Store)

- Instrução buscada (Figura 6).
- Unidade de Controle (Figura 7) decodifica, ativa *AluSrc=1*, *MemWrite=1*, *RegWrite=0*.
- Banco de Registradores lê rs1 (*ReadData1*) e rs2 (*ReadData2*).
- Extensor estende *offset* para *ImmExt*.
- *ReadData1* vai para ULA (A). *ImmExt* passa pelo MUX1 (*AluSrc=1*) para ULA (B).

- ULA calcula $A+B$ (endereço), resultado vai para *Alu_Result*.
- *Alu_Result* vai como *Adress* para Memória de Dados e Decodificador MMIO Saída (Figura 8).
- *ReadData2* é direcionado para a entrada *WriteData* da Memória de Dados (e *DataIn* do *Display*).
- Unidade de Controle ativa *MemWrite*=1.
- Decodificador MMIO Saída verifica *Alu_Result* e *MemWrite*: Se não for endereço do display, ativa *MemDados_WriteEnable*=1. Se for, ativa *Display_WriteEnable*=1.
- Memória de Dados (se *MemDados_WriteEnable*=1) escreve *ReadData2* no endereço *Alu_Result*. *Display* (se *Display_WriteEnable*=1) captura *ReadData2*.
- Nenhum dado é escrito no Banco de Registradores (*RegWrite*=0).
- PC é atualizado para $PC+4$.

4. *beq* rs1, rs2, offset (Tipo B - Branch)

- Instrução buscada (Figura 6).
- Unidade de Controle (Figura 7) decodifica, ativa *Branch*=1, *RegWrite*=0, *AluSrc*=0 (se ULA for usada para comparar) ou *BranchType* apropriado (se Unidade de *Branch* separada for usada).
- Banco de Registradores lê rs1 (*ReadData1*) e rs2 (*ReadData2*).
- Extensor estende *offset*. Somador PC Target calcula $PC_out + ImmExt \rightarrow Pc_target$.
- Se Unidade de *Branch* for usada: ela compara *ReadData1* e *ReadData2* baseado em *BranchType* e gera sinal de condição. $BranchTaken = Branch \text{ AND } ConditionMet$.
- Se ULA for usada: ULA subtrai (*AluControl*=SUB), flag *Zero* é verificada. $BranchTaken = Branch \text{ AND } Zero$.
- MUX4 (Figura 6): Se *BranchTaken*=1, seleciona *Pc_target*. Se *BranchTaken*=0, seleciona $PC+4$.
- PC é atualizado com a saída do MUX4.

5. *jal* rd, offset (Tipo J - Jump and Link)

- Instrução buscada (Figura 6).
- Unidade de Controle (Figura 7) decodifica, ativa *RegWrite*=1, *Jump*=1 (para MUX4), *MemtoReg*=1, *Inst*=0 (seleciona $PC+4$ no MUX3).
- Extensor estende offset (formato J). Somador PC Target calcula $PC_out + ImmExt \rightarrow Pc_target$.

- MUX4 (Figura 6): Seleciona Pc_target (controlado por $Jump=1$).
- PC é atualizado com Pc_target .
- Somador $PC+4$ calcula $PC_out + 4$.
- $PC+4$ vai para MUX3. MUX3 seleciona ($MemtoReg=1$, $Inst=0$) $Pc+4$ para $WriteData$.
- $WriteData$ (contendo $PC+4$) é escrito no registrador rd (habilitado por $RegWrite=1$).

6. Acesso MMIO de Leitura (ex: `lw rd, 0(t1)` com $t1=Endereço\ Entrada$)

- Fluxo similar ao `lw` normal até o cálculo do endereço Alu_Result (Figura 7, 8).
- Unidade de Controle ativa $MemRead=1$.
- Alu_Result vai para Decodificador MMIO (Entrada).
- Decodificador MMIO (Entrada) detecta o endereço e $MemRead=1$, ativa $SelecDado=1$.
- Bloco Entrada coloca $input_signal$ na sua saída ($ReadData2$ no diagrama).
- MUX5 seleciona ($SelecDado=1$) a saída do Bloco Entrada para $Read_dataFinal$.
- $Read_dataFinal$ passa pelo MUX3 ($MemtoReg=0$, $Inst=1$) para $WriteData$.
- $WriteData$ é escrito em rd (habilitado por $RegWrite=1$).
- PC é atualizado para $PC+4$.

7. Acesso MMIO de Escrita (ex: `sw rs2, 0(t1)` com $t1=Endereço\ Display$)

- Fluxo similar ao `sw` normal até o cálculo do endereço Alu_Result e leitura de rs2 ($ReadData2$) (Figura 7, 8).
- Unidade de Controle ativa $MemWrite=1$.
- Alu_Result e $MemWrite$ vão para Decodificador MMIO Saída.
- Decodificador MMIO Saída detecta o endereço do *display* e $MemWrite=1$, ativa $Display_WriteEnable=1$ e $MemDados_WriteEnable=0$.
- $ReadData2$ vai para a entrada de dados do Display.
- *Display* captura $ReadData2$ no pulso de *clock* (habilitado por $Display_WriteEnable=1$).
- Memória de Dados não escreve ($MemDados_WriteEnable=0$).
- Nenhuma escrita no Banco de Registradores ($RegWrite=0$).
- PC é atualizado para $PC+4$.

4.3.4 Implementação dos Módulos da Unidade de Processamento em Verilog

Com base na arquitetura do datapath definida (Seção 4.3), os principais componentes da unidade de processamento foram implementados em linguagem de descrição de hardware Verilog. Cada módulo foi desenvolvido em um arquivo separado para promover a modularidade e facilitar os testes individuais. A seguir, descreve-se a implementação de cada um desses blocos.

4.3.4.1 Contador de Programa (PC) - `program_counter.v`

O Contador de Programa (PC), descrito funcionalmente na Seção 4.3.1.1, é um registrador de 32 bits que armazena o endereço da instrução corrente a ser buscada na memória. Sua principal função é ser atualizado a cada ciclo de clock com o endereço da próxima instrução, seja ela a instrução sequencial seguinte ou o destino de um desvio ou salto. O módulo Verilog, apresentado no Código B.3, implementa essa funcionalidade. Ele possui entradas para o clock, um sinal de reset assíncrono (ativo alto), um sinal *'enable'*, e a entrada de 32 bits `pc_in`, que fornece o próximo endereço a ser carregado. A saída `pc_out` disponibiliza o valor atual do PC. Durante o reset, o PC é inicializado com um endereço padrão (ex: 32'h00000000). Em cada borda de subida do clock, se o reset não estiver ativo, `pc_out` recebe o valor de `pc_in` B.3. O *enable* funciona como um sinal de permissão vindo do botão de enter, para que o usuário consiga corretamente inserir suas entradas antes do funcionamento da CPU.

4.3.4.2 Somador PC+4 - `pc_plus_4_adder.v`

Para a execução sequencial de instruções, é necessário calcular o endereço da próxima instrução, que é o endereço da instrução atual mais quatro bytes (considerando instruções de 32 bits). O módulo `pc_plus_4_adder.v` (Código B.4) realiza essa simples operação aritmética. Trata-se de um bloco puramente combinacional que recebe o valor atual do PC (`current_pc`) como entrada e produz a saída `pc_plus_4`, que é `current_pc + 4`. Esta saída é utilizada como uma das fontes para o multiplexador que seleciona o próximo PC (MUX4) e também como o valor de retorno para as instruções de salto e link (`jal`, `jalr`), sendo uma entrada para o MUX3. A implementação utiliza uma atribuição contínua (`assign`).

4.3.4.3 Somador de Endereço de Destino para Desvios e Saltos Diretos (PC Target Adder) - `pc_target_adder.v`

O Somador de Endereço de Destino, também referido como "ADDER PC" e descrito funcionalmente na Seção 4.3.4.4, é responsável por calcular o endereço alvo para as instruções de desvio condicional (Tipo B) e para o salto incondicional direto (`jal` - Tipo J). Este endereço é determinado somando o valor atual do Contador de Programa (`PC_out`)

com o valor do imediato estendido com sinal (*ImmExt*), que representa o deslocamento relativo ao PC.

O módulo Verilog para este somador, `pc_target_adder.v` (Código B.5), implementa uma simples adição de 32 bits. É um bloco puramente combinacional que recebe duas entradas de 32 bits, `in_pc_out` e `in_imm_extended`, e produz uma saída de 32 bits, `out_pc_target`. A operação é realizada através de uma atribuição contínua (`assign`).

4.3.4.4 Somador de Endereço de Destino para Desvios e Saltos Diretos (PC Target Adder)

O Somador de Endereço de Destino, visualizado como o bloco "ADDER PC" na Figura 6 (detalhe da lógica do PC), é um componente combinacional dedicado ao cálculo do endereço alvo para instruções de desvio condicional (Tipo B, como `beq`) e saltos incondicionais diretos (Tipo J, como `jal`).

Para essas instruções, o endereço de destino é relativo ao valor atual do Contador de Programa (PC). Portanto, a função deste somador é adicionar o valor atual do PC (`PC_out`) ao valor do imediato estendido com sinal (*ImmExt*) extraído da instrução. O *ImmExt*, neste contexto, representa o deslocamento (offset) a ser aplicado ao PC. A operação realizada é:

$$Pc_target = PC_out + ImmExt_{B/J} \quad (4.1)$$

A saída deste somador, denominada `Pc_target`, é uma das entradas do MUX4 (`PcSource`). O MUX4 então decidirá, com base nos sinais de controle (como `BranchTaken` ou `Jump`), se este `Pc_target` calculado ou o `PC+4` (para fluxo sequencial) será o próximo valor a ser carregado no Contador de Programa.

A implementação em Verilog para este somador é um circuito somador de 32 bits padrão. É importante notar que o cálculo do endereço de destino para a instrução `jalr` (que envolve um registrador base e um imediato) é realizado pela Unidade Lógica e Aritmética (ULA) principal, e seu resultado é selecionado através de um caminho diferente no MUX4, conforme descrito nas seções subsequentes da ULA e do MUX4. O sinal `JumpType` indicado na Figura 6 entrando no "ADDER PC" pode, neste contexto, ser interpretado como um sinal que habilita ou influencia a seleção do *ImmExt* apropriado (Tipo B ou J) para este somador, ou é um remanescente de um design alternativo; no datapath completo, este somador foca no cálculo `PC_out + ImmExt`.

4.3.4.5 Multiplexador da Fonte do Próximo PC (MUX4 - `PcSource`) - `mux_pc_source.v`

O MUX4 (`PcSource`), descrito funcionalmente na Seção 4.3.1.1 e ilustrado na Figura 6, é um componente combinacional crucial que seleciona o endereço da próxima instrução a ser carregada no Contador de Programa (PC). Ele determina se o processador continuará a execução sequencial, tomará um desvio condicional, ou realizará um salto incondicional.

O módulo Verilog, apresentado no Código B.6, implementa um multiplexador de 3 entradas de dados principais de 32 bits:

- `in_pc_plus_4`: O endereço da próxima instrução sequencial (PC atual + 4).
- `in_pc_target_branch_jal`: O endereço de destino calculado para desvios condicionais (Tipo B) e para a instrução `jal` (Tipo J), geralmente PC atual + offset.
- `in_alu_result_jalr`: O endereço de destino calculado pela ULA para a instrução `jalr` (`rs1 + offset`).

A seleção entre essas fontes é governada pelo sinal de entrada de 2 bits `sel_pc_src`, que seria gerado pela Unidade de Controle com base na instrução decodificada e no resultado de condições de desvio. Uma instrução `case` dentro de um bloco `always @(*)` é utilizada para direcionar a entrada selecionada para a saída `out_pc_in`. Um comportamento padrão foi definido para o caso de uma combinação de seleção não utilizada, encaminhando `in_pc_plus_4` para a saída como medida de segurança.

4.3.4.6 Unidade Lógica e Aritmética (ULA) - `ulav`

A ULA, descrita funcionalmente na Seção 4.3.1.4, é responsável por realizar as operações aritméticas (adição, subtração), lógicas (AND, OR, XOR), de comparação (SLT) e de deslocamento (SLL, SRL, SRA). O módulo Verilog, apresentado no Código B.7, recebe dois operandos de 32 bits (`operand_a`, `operand_b`) e um sinal de controle de 4 bits (`alu_control`) que determina a operação a ser executada. A saída principal é o resultado de 32 bits (`alu_result`), além de uma `zero_flag`. A lógica interna utiliza um bloco `always @(*)` combinacional com uma instrução `case` para selecionar a operação com base no `alu_control`. Operações como SLT utilizam a capacidade do Verilog de realizar comparações com sinal (`$signed()`), e os deslocamentos consideram os 5 bits inferiores do segundo operando para determinar a quantidade do deslocamento, conforme a especificação RISC-V para instruções tipo R. O código está no Apêndice B.7

4.3.4.7 Unidade de Multiplicação e Divisão - `div_mult.v`

Esta unidade, referida na Seção 4.3.1.8, implementa as operações simplificadas da extensão M: multiplicação (`mul`), divisão inteira assinada (`div`) e resto de divisão assinada (`rem`). O módulo Verilog (Código B.8) recebe dois operandos de 32 bits e um sinal de controle `mul_div_op` para selecionar a operação. A saída é o resultado de 32 bits. As operações são implementadas utilizando os operadores nativos do Verilog (`*`, `/`, `%`), que são sintetizáveis. Para a multiplicação, apenas os 32 bits inferiores do produto são retornados, conforme a definição da instrução `mul`. O tratamento de casos especiais como divisão por

zero e overflow da divisão segue as definições da especificação RISC-V, resultando em valores específicos na saída.

4.3.4.8 Extensor de Imediato - `extensor.v`

O Extensor de Imediato (Seção 4.3.1.3) é crucial para converter os campos de imediato de diversos formatos de instrução para um valor de 32 bits, aplicando a extensão de sinal quando necessário. Conforme o Código B.9, o módulo recebe a instrução completa de 32 bits e um sinal `imm_type` que indica o formato do imediato (I, S, B, J). A lógica combinacional interna, implementada com um bloco `always @(*)` e uma instrução `case`, seleciona os bits apropriados da instrução, os reorganiza conforme o formato e aplica a extensão de sinal replicando o bit mais significativo do imediato original para preencher os bits superiores da saída de 32 bits.

4.3.4.9 Banco de Registradores - `bancoderegistradores.v`

O Banco de Registradores (Seção 4.3.1.2), detalhado no Código B.10, armazena os 32 registradores de propósito geral do RISC-V. Ele possui duas portas de leitura combinacionais e uma porta de escrita síncrona. As leituras são implementadas com atribuições contínuas (`assign`) e um operador ternário para garantir que o acesso ao registrador `x0` (endereço 0) sempre retorne zero. Esta abordagem combinacional permite que os dados lidos estejam disponíveis no mesmo ciclo em que os endereços de leitura são fornecidos. A escrita é controlada por um bloco `always @(posedge clock or posedge reset)`, ocorrendo apenas na borda de subida do clock, se o sinal de habilitação `reg_write` estiver ativo e o registrador de destino não for `x0`.

4.3.4.10 Multiplexadores da Unidade de Processamento

Dois multiplexadores principais foram implementados para direcionar os dados dentro da unidade de processamento central, conforme descrito nas Seções 4.3.1.5 e 4.3.1.9.

O primeiro, `Mux1.v` (Código B.11), seleciona o segundo operando para a ULA (`AluSrc` como sinal de seleção), escolhendo entre o dado lido do banco de registradores (`ReadData2`) ou o imediato estendido (`ImmExt`).

O segundo, `Mux2.v` (Código B.12), seleciona qual resultado será candidato à escrita no banco de registradores (`UseMD` como sinal de seleção), escolhendo entre o resultado da ULA (`AluResult`) ou da unidade de multiplicação/divisão (`MDResult`). Ambos são implementados com lógica combinacional simples utilizando atribuições contínuas e o operador ternário.

4.3.4.11 Módulos de Interface MMIO (`input_device.v` e Lógica de Display)

Para implementar a funcionalidade de Entrada/Saída Mapeada em Memória (MMIO), foram desenvolvidos os blocos que interagem com os periféricos físicos e respondem aos acessos de memória do processador.

4.3.4.11.1 Dispositivo de Entrada (`input_device.v`):

Este módulo, apresentado no Código B.13, serve como a interface para os switches de entrada da placa FPGA. Sua função é capturar o valor dos 18 switches (`physical_switches_i`) e armazená-lo em um registrador interno (`data_out_o`). Essa captura não é contínua; ela ocorre apenas na borda de subida do clock quando um sinal de pulso limpo, proveniente de um debouncer conectado a um botão físico (`'enter_button_i'`), está ativo. Isso garante que o valor lido pelo processador seja estável e corresponda ao valor presente nos switches no momento em que o usuário pressiona "Enter". A saída `data_out_o` é então conectada ao caminho de dados MMIO do processador para ser lida pela instrução `lw`.

4.3.4.11.2 Lógica do Dispositivo de Saída (Display):

A lógica para o dispositivo de saída não foi encapsulada em um módulo separado, mas implementada diretamente no `toplevel.v` para maior clareza na conexão com os pinos físicos. Conforme detalhado no Código B.2 (ver Apêndice B), esta lógica consiste em três partes principais:

1. **Registrador de Saída:** Um registrador de 32 bits (`display_value_reg`) atua como a "memória" do display. Ele captura o valor vindo do processador (`data_from_cpu_to_display`) somente quando o sinal de habilitação de escrita do display (`we_from_cpu_to_display`) está ativo. Este sinal é a materialização da instrução `sw` executada pela CPU no endereço MMIO do display.
2. **Conversor BCD:** O valor armazenado no registrador de saída é continuamente enviado para o módulo `binary_to_bcd` (descrito na Seção 4.3.5.1), que o converte para 8 dígitos BCD.
3. **Drivers de Display:** O resultado BCD é então dividido em dígitos de 4 bits, e cada dígito é convertido para o padrão de 7 segmentos através de uma função local (`bcd_to_7seg`) e atribuído diretamente às portas de saída físicas da placa (`HEX0` a `HEX7`). Para a placa DE2-115, que possui barramentos de dados independentes para cada display, a multiplexação não é necessária.

Essa estrutura garante que o valor enviado pela CPU seja armazenado de forma estável e continuamente exibido nos displays de 7 segmentos.

4.3.4.11.3 Dispositivo de Entrada (`input_device.v`):

O módulo `input_device.v` foi desenvolvido para simular, via interface MMIO, um periférico de entrada baseado em switches físicos. Este componente permite a leitura de um valor de 32 bits a partir das chaves físicas da placa, controlada por um botão de "Enter" e sincronizada com o clock do sistema. Quando o botão de entrada (`enter_button_i`) é pressionado, o valor presente nos switches (`physical_switches_i`) é capturado e armazenado no registrador interno `captured_data`. Simultaneamente, o sinal de status `data_ready` é ativado, indicando que novos dados estão disponíveis. Esse sinal permanece ativo até que a CPU realize uma leitura do periférico (indicada por `cpu_read_en`), momento em que o sinal `data_ready` é automaticamente reiniciado. O dado disponível para a CPU é exposto pela saída `data_for_cpu_o`. Essa lógica garante uma comunicação segura e eficiente entre a CPU e o periférico de entrada, respeitando o modelo de sincronização típico em sistemas embarcados com leitura de estado. O código do módulo está no apêndice de códigos [B.13](#).

4.3.5 Integração da Unidade de Processamento - `UnidadeProcessamento.v`

Os módulos Verilog descritos anteriormente foram integrados para formar o núcleo da unidade de processamento, denominado `UnidadeProcessamento`, conforme apresentado no Código [4.1](#). Este módulo encapsula a principal lógica de execução de dados do processador.

O `UnidadeProcessamento` instancia o Banco de Registradores, o Extensor de Imediato, a ULA, a Unidade DIV/MULT e os multiplexadores `mux_alu_operand_b` e `mux_writeback_source1`. As interconexões entre esses submódulos seguem diretamente o datapath ilustrado nas Figuras [7](#) e [8](#). Por exemplo, as saídas `rf_read_data1` e `rf_read_data2` do Banco de Registradores alimentam as entradas da ULA (com `rf_read_data2` passando pelo `mux_alu_operand_b` juntamente com o `extended_immediate`) e da Unidade DIV/MULT. O resultado da ULA (`ula_result`) e da Unidade DIV/MULT (`div_mult_result`) são direcionados ao `mux_writeback_source1`, cuja saída (`core_result_out`) representa o dado que, no processador completo, seria uma das fontes para o MUX final de escrita no Banco de Registradores (MUX3 na Figura [8](#)).

As entradas de controle para o `processing_core` (`reg_write_en`, `imm_type_sel`, `alu_src_sel`, `alu_control_op`, `mul_div_op_sel`, `use_md_sel`) seriam normalmente fornecidas pela Unidade de Controle principal, baseada na instrução decodificada. Para fins de teste isolado deste núcleo, esses sinais são fornecidos pelo testbench. A entrada `instruction` fornece os bits da instrução necessários para o Extensor de Imediato e para os endereços de leitura/escrita do Banco de Registradores. As saídas principais do núcleo são o resultado da operação (`core_result_out`) e os dados lidos dos registradores (`reg_read_data1`, `reg_read_data2`) e a flag zero da ULA (`alu_zero_flag`), que seriam

utilizados por outros componentes do processador completo (como a Unidade de Branch ou a Memória de Dados para stores).

```

1 // Modulo do Nucleo de Processamento Central
2 module UnidadeProcessamento (
3     // Entradas de Controle (seriam da Unidade de Controle)
4     input wire      clk,
5     input wire      reset,
6     input wire      reg_write_en,    // RegWrite
7     input wire [2:0] imm_type_sel,    // ImmType
8     input wire      alu_src_sel,      // AluSrc
9     input wire [3:0] alu_control_op,  // AluControl
10    input wire [1:0] mul_div_op_sel,   // MulDivOp
11    input wire      use_md_sel,        // UseMD
12
13    // Entrada da Instrucao
14    input wire [31:0] instruction,
15
16    // Saida principal deste nucleo (para o MUX3 no datapath completo)
17    output wire [31:0] core_result_out, // Equivalente ao ContaFinal
18
19    // Saidas para Unidade de Branch (se nao integrada na ULA) e outros
20    output wire [31:0] reg_read_data1, // Para Branch, Store Data
21    output wire [31:0] reg_read_data2, // Para Branch
22    output wire      alu_zero_flag     // Para Branch (beq)
23 );
24
25 // Sinais internos (fios)
26 wire [31:0] extended_immediate;
27 wire [31:0] rf_read_data1;
28 wire [31:0] rf_read_data2;
29 wire [31:0] alu_operand_b;
30 wire [31:0] ula_result;
31 // wire      ula_zero_flag; // Ja eh uma saida do modulo
32 wire [31:0] div_mult_result;
33
34 // Instanciacao do Extensor de Imediato
35 immediate_extender imm_ext_unit (
36     .instruction      (instruction),
37     .imm_type         (imm_type_sel),
38     .imm_extended     (extended_immediate)
39 );
40
41 // Instanciacao do Banco de Registradores
42 // O write_data vira de um MUX externo (MUX3) no datapath completo.
43
44 register_file reg_file_unit (
45     .clock            (clk),

```

```

46     .reset      (reset),
47     .reg_write   (reg_write_en),
48     .read_addr1  (instruction[19:15]), // rs1 address
49     .read_addr2  (instruction[24:20]), // rs2 address
50     .write_addr   (instruction[11:7]), // rd address
51     .write_data   (core_result_out),   // TEMPORARIO para teste
52     .read_data1   (rf_read_data1),
53     .read_data2   (rf_read_data2)
54 );
55
56 // Instanciacao do MUX1 (aluscr)
57 mux_alu_operand_b mux1_alu_op_b (
58     .in_read_data2      (rf_read_data2),
59     .in_imm_extended     (extended_immediate),
60     .sel_alu_src         (alu_src_sel),
61     .out_alu_operand_b  (alu_operand_b)
62 );
63
64 // Instanciacao da ULA
65 ula_riscv_alu_unit (
66     .operand_a      (rf_read_data1),
67     .operand_b      (alu_operand_b),
68     .alu_control     (alu_control_op),
69     .alu_result      (ula_result),
70     .zero_flag       (alu_zero_flag) // Conectando a saida da ULA a
        saida do core
71 );
72
73 // Instanciacao da Unidade DIV/MULT
74 div_mult_unit div_mult_module (
75     .operand1      (rf_read_data1),
76     .operand2      (rf_read_data2),
77     .mul_div_op     (mul_div_op_sel),
78     .md_result      (div_mult_result)
79 );
80
81 // Instanciacao do MUX2 (Aluresult)
82 mux_writeback_source1 mux2_wb_src1 (
83     .in_alu_result      (ula_result),
84     .in_md_result       (div_mult_result),
85     .sel_use_md         (use_md_sel),
86     .out_conta_final    (core_result_out)
87 );
88
89 // Atribuindo saidas do core
90 assign reg_read_data1 = rf_read_data1;
91 assign reg_read_data2 = rf_read_data2;

```

```
92  
93 endmodule
```

Listing 4.1 – Código Verilog do Núcleo de Processamento Integrado (UnidadeProcessamento.v)

4.3.5.1 Módulos de Interface com Hardware Físico (FPGA)

Para integrar o processador RISC-V com os periféricos da placa FPGA DE2-115 e garantir uma interação robusta e observável, foram desenvolvidos módulos de interface específicos. Estes blocos de hardware não fazem parte do núcleo do processador em si, mas ficam no módulo `toplevel.v` e servem como a ponte entre a lógica síncrona da CPU e o mundo físico assíncrono e de alta frequência.

4.3.5.1.1 Debouncer de Botão (`debouncer.v` e `debouncer_level.v`):

Botões mecânicos, como os pinos **KEY** da placa, sofrem de um fenômeno chamado *bouncing*, onde um único pressionamento gera múltiplos pulsos elétricos rápidos. Para filtrar este ruído e gerar um sinal de controle limpo, foram implementados dois tipos de debouncer. O primeiro, `debouncer.v`, gera um pulso único de um ciclo de clock para cada pressionamento, ideal para eventos como o "Enter" do dispositivo de entrada. O segundo, `debouncer_level.v`, gera um sinal de nível estável que permanece ativo enquanto o botão é pressionado, sendo a solução adequada para o sinal de **reset** do sistema. Ambos os módulos utilizam um contador interno para garantir que o estado do botão permaneça estável por um período predefinido (ex: 20 ms) antes de validar a mudança de estado. Código no apêndice [B.14](#).

4.3.5.1.2 Divisor de Clock (`clock_divider.v`):

O processador, sendo um design monociclo, executa uma instrução a cada pulso de clock. Com o clock de 50 MHz da placa, a execução de um programa inteiro ocorre em microssegundos, tornando a observação visual do seu funcionamento em tempo real impossível. Para fins de depuração e demonstração em hardware, foi implementado um divisor de clock (Código [B.15](#)). Este módulo utiliza um contador para gerar um sinal de habilitação de clock (`clk_enable` ou `tick`) a uma frequência muito mais baixa (ex: 1 Hz). Este sinal de habilitação é então utilizado para permitir a atualização dos registradores síncronos do processador (PC, Banco de Registradores, etc.) apenas uma vez por segundo, efetivamente fazendo com que o processador execute em "câmera lenta" e permitindo a visualização do PC mudando nos LEDs.

4.3.5.1.3 Conversor Binário para BCD (`binary_to_bcd.v`):

Os displays de 7 segmentos da placa são projetados para exibir dígitos decimais (0-9). No entanto, o resultado dos cálculos do processador é um número binário de 32 bits. Para exibir este valor, é necessário primeiro convertê-lo para o formato BCD (Binary-Coded Decimal), onde cada grupo de 4 bits representa um dígito decimal. Para esta tarefa, foi implementado um conversor combinacional (Código B.16) que utiliza o algoritmo "Double Dabble" (*shift and add 3*). Este módulo recebe um valor binário de 32 bits e gera uma saída de 32 bits contendo os 8 dígitos BCD correspondentes, que são então enviados para a lógica final de acionamento dos displays.

4.3.5.1.4 Memória de Dados (`data_memory.v`):

A Memória de Dados, descrita no módulo `data_memory.v` (B.17), é responsável por armazenar e recuperar os dados utilizados durante a execução dos programas no processador. Ela foi implementada como uma RAM de escrita e leitura síncronas, com capacidade de 256 palavras de 32 bits, representadas por um array de registradores em Verilog. A escrita ocorre de forma síncrona na borda de subida do sinal `clk_write`, sendo habilitada apenas quando o sinal de controle `MemWrite` está ativo. A leitura também é síncrona, sendo realizada na borda de subida do clock `clk_read` quando o sinal `MemRead` está ativo. O dado é transferido para a saída `read_data` com um ciclo de latência, refletindo o conteúdo armazenado no endereço indexado por `address[9:2]`. A divisão do endereço em bits altos e baixos considera o alinhamento por palavra (4 bytes), garantindo acesso correto à memória.

5 Resultados Obtidos e Discussões

Este capítulo apresenta os resultados obtidos a partir da simulação funcional dos módulos Verilog desenvolvidos para a unidade de processamento do processador RISC-V RV32IM. O objetivo principal desta etapa foi verificar a corretude da implementação de cada componente individualmente e, posteriormente, a integração básica de alguns desses componentes, utilizando testbenches específicos e a análise de formas de onda geradas por um simulador Verilog (ModelSim e/ou Waveform).

O datapath projetado foi concebido para atender aos requisitos de execução de todo o conjunto de instruções definido no Capítulo 4 (especificamente nas Tabelas 1 a 10).

Conforme descrito na Seção 4.3.3, foram incluídos todos os caminhos de dados, unidades funcionais (ULA, Unidade), elementos de armazenamento (PC, Banco de Registradores, Memórias), multiplexadores e unidades de controle (Unidade de Controle principal, Decodificadores MMIO) necessários para o fluxo correto de informações e a execução das operações aritméticas, lógicas, de acesso à memória, desvios, saltos e interações com os periféricos MMIO. A análise detalhada do fluxo de dados para cada tipo de instrução, apresentada na seção anterior, demonstra como os componentes interagem e como os sinais de controle orquestram as operações, indicando que o *design* é capaz de executar as funcionalidades propostas.

5.1 Simulação dos Módulos Individuais da Unidade de Processamento

Para garantir a funcionalidade correta de cada bloco antes da integração, foram desenvolvidos e executados testbenches individuais para os principais componentes da unidade de processamento. A seguir, são apresentados os resultados para os módulos principais. Os demais testbenches podem ser vistos no Apêndice C.

5.1.1 Simulação da Unidade Lógica e Aritmética (ULA)

A verificação funcional da Unidade Lógica e Aritmética (ULA) foi realizada através de um testbench abrangente em Verilog (`ula_riscv_comprehensive_tb.v`). Este testbench aplicou uma série sistemática de operandos de entrada (`operand_a`, `operand_b`) e sinais de controle (`alu_control`) para cobrir todas as operações implementadas: ADD, SUB, AND, OR, XOR, SLT (Set Less Than, signed), SLL (Shift Left Logical), SRL (Shift Right Logical) e SRA (Shift Right Arithmetic). Foram incluídos casos de teste com valores positivos, negativos, zero, e casos limite para as operações de deslocamento e aritméticas.

A Figura 9 apresenta a saída textual do console gerada durante a execução da simulação, onde cada linha detalha as entradas aplicadas, o código de controle da operação, o resultado obtido pela ULA e o estado da flag zero.

Figura 9 – Saída do console da simulação do testbench da ULA, detalhando os casos de teste.

```
# Loading work.ula_tb
VSIM 11>run -all
# Iniciando Testes da ULA...
# Time: 0 | A: 00000005 | B: 0000000a | Ctrl: 0000 | Result: 00000000 | Zero: 1
# Time: 10000 | A: 00000000 | B: 00000000 | Ctrl: 0000 | Result: 00000000 | Zero: 1
# Time: 20000 | A: ffffffff | B: 0000000a | Ctrl: 0000 | Result: 0000000a | Zero: 0
# Time: 30000 | A: ffffffff | B: ffffffff | Ctrl: 0000 | Result: ffffffff | Zero: 0
# Time: 40000 | A: 7fffffff | B: 00000001 | Ctrl: 0000 | Result: 00000001 | Zero: 0
# Time: 50000 | A: 80000000 | B: ffffffff | Ctrl: 0000 | Result: 80000000 | Zero: 0
# Time: 60000 | A: 0000000a | B: 00000005 | Ctrl: 0001 | Result: 0000000f | Zero: 0
# Time: 70000 | A: 00000005 | B: 0000000a | Ctrl: 0001 | Result: 0000000f | Zero: 0
# Time: 80000 | A: 00000007 | B: 00000007 | Ctrl: 0001 | Result: 00000007 | Zero: 0
# Time: 90000 | A: ffffffff | B: 0000000a | Ctrl: 0001 | Result: ffffffff | Zero: 0
# Time: 100000 | A: 80000000 | B: 00000001 | Ctrl: 0001 | Result: 80000001 | Zero: 0
# Time: 110000 | A: ffff0000 | B: 00ffff00 | Ctrl: 0010 | Result: 00feff00 | Zero: 0
# Time: 120000 | A: aaaaaaaaaa | B: 55555555 | Ctrl: 0010 | Result: ffffffff | Zero: 0
# Time: 130000 | A: f0f0f0f0 | B: 0f0f0f0f | Ctrl: 0011 | Result: elelelel | Zero: 0
# Time: 140000 | A: 00000000 | B: 00000000 | Ctrl: 0011 | Result: 00000000 | Zero: 1
# Time: 150000 | A: a5a5a5a5 | B: 5a5a5a5a | Ctrl: 0100 | Result: ffffffff | Zero: 0
# Time: 160000 | A: a5a5a5a5 | B: a5a5a5a5 | Ctrl: 0100 | Result: 00000000 | Zero: 1
# Time: 170000 | A: 00000005 | B: 0000000a | Ctrl: 0101 | Result: 00000001 | Zero: 0
# Time: 180000 | A: 0000000a | B: 00000005 | Ctrl: 0101 | Result: 00000000 | Zero: 1
# Time: 190000 | A: 00000005 | B: 00000005 | Ctrl: 0101 | Result: 00000000 | Zero: 1
# Time: 200000 | A: ffffffff | B: ffffffff | Ctrl: 0101 | Result: 00000001 | Zero: 0
# Time: 210000 | A: ffffffff | B: ffffffff | Ctrl: 0101 | Result: 00000000 | Zero: 1
# Time: 220000 | A: ffffffff | B: 00000005 | Ctrl: 0101 | Result: 00000001 | Zero: 0
# Time: 230000 | A: 00000005 | B: ffffffff | Ctrl: 0101 | Result: 00000000 | Zero: 1
# Time: 240000 | A: 0000000f | B: 00000002 | Ctrl: 0110 | Result: 0000003c | Zero: 0
# Time: 250000 | A: 80000000 | B: 00000001 | Ctrl: 0110 | Result: 00000000 | Zero: 1
# Time: 260000 | A: ffffffff | B: 00000000 | Ctrl: 0110 | Result: ffffffff | Zero: 0
# Time: 270000 | A: 000000f0 | B: 00000004 | Ctrl: 0111 | Result: 0000000f | Zero: 0
# Time: 280000 | A: f0000000 | B: 00000004 | Ctrl: 0111 | Result: 0f000000 | Zero: 0
# Time: 290000 | A: 00000001 | B: 00000001 | Ctrl: 0111 | Result: 00000000 | Zero: 1
# Time: 300000 | A: 000000f0 | B: 00000004 | Ctrl: 1000 | Result: 0000000f | Zero: 0
# Time: 310000 | A: f0000000 | B: 00000004 | Ctrl: 1000 | Result: ff000000 | Zero: 0
# Time: 320000 | A: ffffffff | B: 00000001 | Ctrl: 1000 | Result: ffffffff | Zero: 0
# Time: 330000 | A: 12345678 | B: 9abcdef0 | Ctrl: 1111 | Result: 00000000 | Zero: 1
# Testes finalizados.
# ** Note: $stop : C:/Users/User/Desktop/ULA/SIMULATION/test_ULA.v(88)
# Time: 340 ns Iteration: 0 Instance: /ula_tb
# Break in Module ula_tb at C:/Users/User/Desktop/ULA/SIMULATION/test_ULA.v line 88
VSIM 12>
```

Fonte: Simulação realizada pela autora em ModelSim.

Para uma análise visual detalhada do comportamento temporal dos sinais, as formas de onda da simulação foram capturadas e são apresentadas nas Figuras 10, 11 e 12.

A Figura 10 ilustra os primeiros casos de teste. Por exemplo:

- No tempo aproximado de 20 ns, com `alu_control = 0000` (AND), `operand_a = 0` e `operand_b = 0`, o `alu_result` é corretamente 0 e a `zero_flag` é ativada (St1).
- Em 40 ns, para `alu_control = 0000` (AND), `operand_a = 0xFFFFFFFF` (-1) e `operand_b = 0x00000001` (1), o resultado é `0x00000001` (1), e `zero_flag = 0`.
- Em 60 ns, para `alu_control = 0001` (OR), `operand_a = 0` e `operand_b = 0x0000000a` (10), o resultado é `0x0000000a` (10), e `zero_flag = 0`.

- Em 120 ns, para `alu_control` = 0010 (ADD), `operand_a` = 0xAAAAAAAA e `operand_b` = 0x55555555, o resultado é 0xFFFFFFFF (-1).
- Em 150 ns, para `alu_control` = 0011 (SUB), `operand_a` = 5 e `operand_b` = 10, o resultado é -5 (0xFFFFFFFFB).

Figura 10 – Forma de onda da simulação da ULA (Parte 1): Testes iniciais incluindo AND, OR, ADD, SUB.



Fonte: Simulação realizada pela autora em ModelSim.

A Figura 11 continua a sequência de testes, focando nas operações lógicas e de comparação.

- Em 180 ns, para `alu_control` = 0101 (SLT), `operand_a` = -10 (0xFFFFFFFF6) e `operand_b` = -5 (0xFFFFFFFFB), como $-10 < -5$, o `alu_result` é 1.
- Em 210 ns, para `alu_control` = 0101 (SLT), `operand_a` = -5 (0xFFFFFFFFB) e `operand_b` = -10 (0xFFFFFFFF6), como -5 não é menor que -10, o `alu_result` é 0.
- Em 240 ns, para `alu_control` = 0110 (SLL), `operand_a` = 15 (0xF) e `operand_b` (quantidade de shift) = 2, o resultado é 60 (0x3C).

Finalmente, a Figura 12 mostra os testes para as operações de deslocamento restantes e um caso de controle default.

- Em 270 ns, para `alu_control` = 0111 (SRL), `operand_a` = 240 (0xF0) e `operand_b` (quantidade de shift) = 4, o resultado é 15 (0xF).
- Em 300 ns, para `alu_control` = 1000 (SRA), `operand_a` = -2 (0xFFFFFFFFFE) e `operand_b` (quantidade de shift) = 1, o resultado é -1 (0xFFFFFFFFF), demonstrando a preservação do bit de sinal.

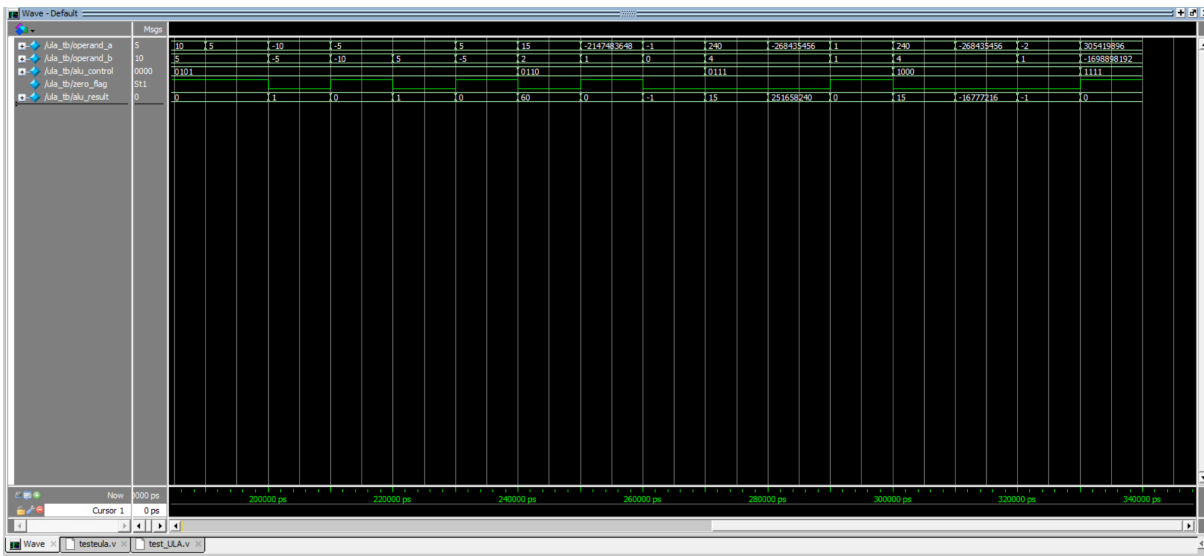
Figura 11 – Forma de onda da simulação da ULA (Parte 2): Testes incluindo XOR, SLT e SLL.



Fonte: Simulação realizada pela autora em ModelSim.

- Em 330 ns, com um `alu_control` = 1111 (default/não mapeado), o `alu_result` é 0 e a `zero_flag` é ativada, conforme o comportamento padrão definido na ULA.

Figura 12 – Forma de onda da simulação da ULA (Parte 3): Testes incluindo SRL, SRA e operação default.



Fonte: Simulação realizada pela autora em ModelSim.

A correspondência entre os resultados exibidos no console (Figura 9) e o comportamento temporal observado nas formas de onda (Figuras 10 a 12) confirma que a ULA implementada executa corretamente todas as operações especificadas, incluindo a correta

sinalização da `zero_flag`. Os diversos casos de teste aplicados fornecem um bom nível de confiança na funcionalidade deste componente.

5.1.2 Simulação da Unidade de Multiplicação e Divisão

A unidade responsável pelas operações de multiplicação (`mul`), divisão inteira (`div`) e resto de divisão (`rem`), denominada `div_mult`, foi submetida a um conjunto exaustivo de testes através do testbench `div_mult_tb.v`. Este testbench foi projetado para verificar a corretude funcional para diversas combinações de operandos, incluindo valores positivos, negativos, zero, e casos especiais como divisão por zero e overflow da divisão, conforme definido pela especificação RISC-V.

A Figura 13 apresenta a saída consolidada do console gerada durante a simulação. Cada linha indica o tipo de operação, os operandos aplicados, o resultado esperado calculado pelo testbench e o resultado efetivamente produzido pela unidade `div_mult`, juntamente com uma indicação de sucesso para cada um dos 33 casos de teste executados.

Figura 13 – Saída do console da simulação do testbench da unidade de Multiplicação/Divisão.

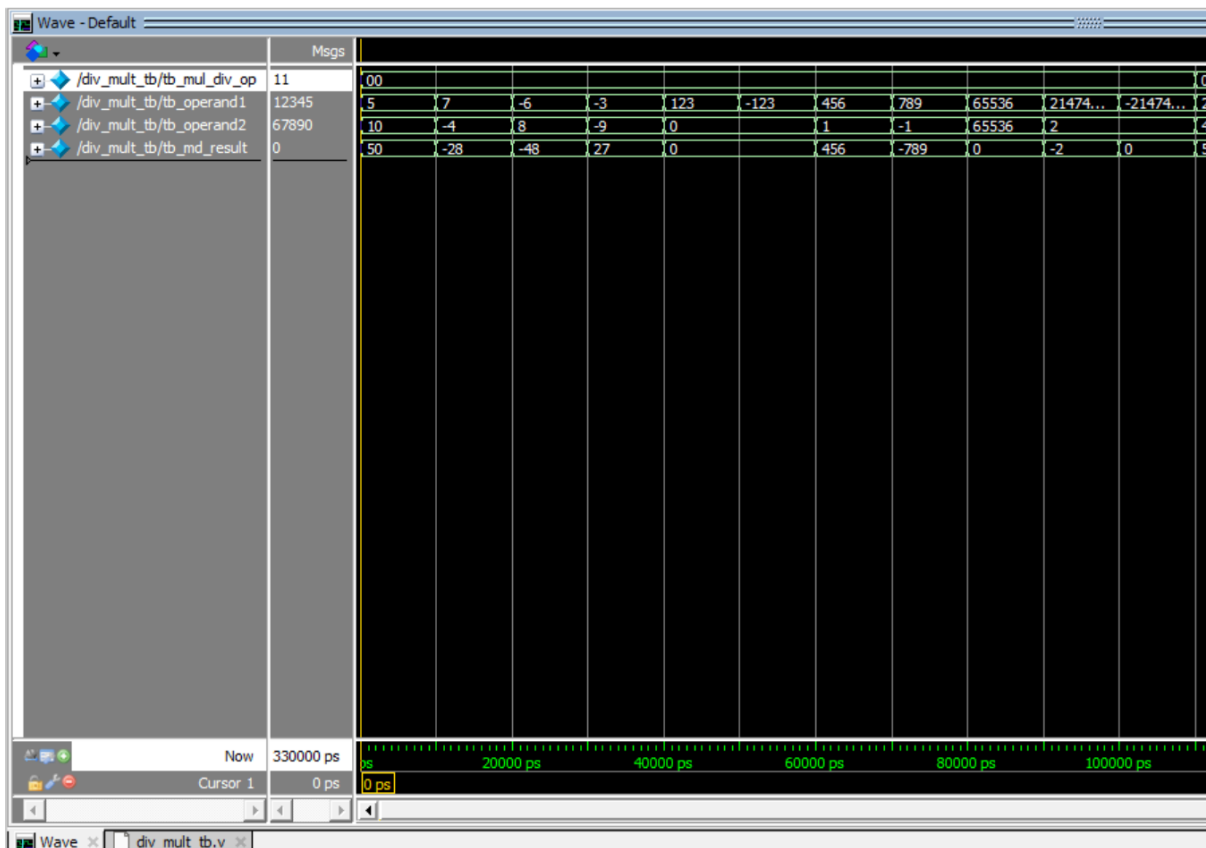
```
#
# --- TESTES DE MULTIPLICAÇÃO (MUL) ---
# [ 1] SUCESSO: MUL: 5 * 10. Operands: 5 (00000005), 10 (0000000a). Op: 00. Resultado: 50 (00000032)
# [ 2] SUCESSO: MUL: 7 * -4. Operands: 7 (00000007), -4 (fffffffc). Op: 00. Resultado: -28 (ffffffe4)
# [ 3] SUCESSO: MUL: -6 * 8. Operands: -6 (fffffffa), 8 (00000008). Op: 00. Resultado: -48 (ffffffa0)
# [ 4] SUCESSO: MUL: -3 * -9. Operands: -3 (fffffffd), -9 (ffffff97). Op: 00. Resultado: 27 (0000001b)
# [ 5] SUCESSO: MUL: 123 * 0. Operands: 123 (0000007b), 0 (00000000). Op: 00. Resultado: 0 (00000000)
# [ 6] SUCESSO: MUL: -123 * 0. Operands: -123 (ffffff85), 0 (00000000). Op: 00. Resultado: 0 (00000000)
# [ 7] SUCESSO: MUL: 456 * 1. Operands: 456 (000001c8), 1 (00000001). Op: 00. Resultado: 456 (000001c8)
# [ 8] SUCESSO: MUL: 789 * -1. Operands: 789 (00000315), -1 (fffffffd). Op: 00. Resultado: -789 (fffffc0b)
# [ 9] SUCESSO: MUL: 65536 * 65536 (low 32b). Operands: 65536 (00010000), 65536 (00010000). Op: 00. Resultado: 0 (00000000)
# [10] SUCESSO: MUL: MAX_INT_S * 2 (low 32b). Operands: 2147483647 (7fffffff), 2 (00000002). Op: 00. Resultado: -2 (fffffffe)
# [11] SUCESSO: MUL: MIN_INT_S * 2 (low 32b). Operands: -2147483648 (80000000), 2 (00000002). Op: 00. Resultado: 0 (00000000)
#
# --- TESTES DE DIVISÃO (DIV) ---
# [12] SUCESSO: DIV: 20 / 4. Operands: 20 (00000014), 4 (00000004). Op: 01. Resultado: 5 (00000005)
# [13] SUCESSO: DIV: 10 / 3. Operands: 10 (0000000a), 3 (00000003). Op: 01. Resultado: 3 (00000003)
# [14] SUCESSO: DIV: -10 / 3. Operands: -10 (ffffffe6), 3 (00000003). Op: 01. Resultado: -3 (fffffffd)
# [15] SUCESSO: DIV: 10 / -3. Operands: 10 (0000000a), -3 (fffffffd). Op: 01. Resultado: -3 (fffffffd)
# [16] SUCESSO: DIV: -10 / -3. Operands: -10 (ffffffe6), -3 (fffffffd). Op: 01. Resultado: 3 (00000003)
# [17] SUCESSO: DIV: 0 / 7. Operands: 0 (00000000), 7 (00000007). Op: 01. Resultado: 0 (00000000)
# [18] SUCESSO: DIV: 123 / 1. Operands: 123 (0000007b), 1 (00000001). Op: 01. Resultado: 123 (0000007b)
# [19] SUCESSO: DIV: 123 / -1. Operands: 123 (0000007b), -1 (fffffffd). Op: 01. Resultado: -123 (ffffff85)
# [20] SUCESSO: DIV: 7 / 0. Operands: 7 (00000007), 0 (00000000). Op: 01. Resultado: -1 (fffffffd)
# [21] SUCESSO: DIV: -7 / 0. Operands: -7 (ffffff89), 0 (00000000). Op: 01. Resultado: -1 (fffffffd)
# [22] SUCESSO: DIV: MIN_INT_S / -1. Operands: -2147483648 (80000000), -1 (fffffffd). Op: 01. Resultado: -2147483648 (80000000)
#
# --- TESTES DE RESTO (REM) ---
# [23] SUCESSO: REM: 10 % 3. Operands: 10 (0000000a), 3 (00000003). Op: 10. Resultado: 1 (00000001)
# [24] SUCESSO: REM: -10 % 3. Operands: -10 (ffffffe6), 3 (00000003). Op: 10. Resultado: -1 (fffffffd)
# [25] SUCESSO: REM: 10 % -3. Operands: 10 (0000000a), -3 (fffffffd). Op: 10. Resultado: 1 (00000001)
# [26] SUCESSO: REM: -10 % -3. Operands: -10 (ffffffe6), -3 (fffffffd). Op: 10. Resultado: -1 (fffffffd)
# [27] SUCESSO: REM: 0 % 7. Operands: 0 (00000000), 7 (00000007). Op: 10. Resultado: 0 (00000000)
# [28] SUCESSO: REM: 123 % 1. Operands: 123 (0000007b), 1 (00000001). Op: 10. Resultado: 0 (00000000)
# [29] SUCESSO: REM: 123 % -1. Operands: 123 (0000007b), -1 (fffffffd). Op: 10. Resultado: 0 (00000000)
# [30] SUCESSO: REM: 7 % 0. Operands: 7 (00000007), 0 (00000000). Op: 10. Resultado: 7 (00000007)
# [31] SUCESSO: REM: -7 % 0. Operands: -7 (ffffff89), 0 (00000000). Op: 10. Resultado: -7 (ffffff89)
# [32] SUCESSO: REM: MIN_INT_S % -1. Operands: -2147483648 (80000000), -1 (fffffffd). Op: 10. Resultado: 0 (00000000)
#
# --- TESTE DE OPERAÇÃO DEFAULT/INVÁLIDA ---
# [33] SUCESSO: DEFAULT OP: 12345, 67890. Operands: 12345 (00003039), 67890 (00010932). Op: 11. Resultado: 0 (00000000)
#
# Testes finalizados.
# TODOS OS 33 TESTES DA div_mult PASSARAM!
```

Fonte: Simulação realizada pela autora em ModelSim.

As formas de onda correspondentes a esta simulação foram segmentadas para melhor visualização e análise. A Figura 14 foca nos testes da operação de multiplicação (`tb_mul_div_op = 2'b00`). Observa-se, por exemplo, que para a entrada `tb_operand1 = 5` e `tb_operand2 = 10` (entre 0 ps e 10 ns na forma de onda), a saída `tb_md_result` é corretamente 50. Similarmente, para `tb_operand1 = 7` e `tb_operand2 = -4` (entre 10 ns e 20 ns), o resultado é -28. Os testes também cobrem casos de multiplicação por zero e com

operandos que gerariam overflow em um resultado de 64 bits, onde a unidade corretamente retorna os 32 bits inferiores do produto, como no caso de $65536 * 65536$ resultando em '0' para `tb_md_result`.

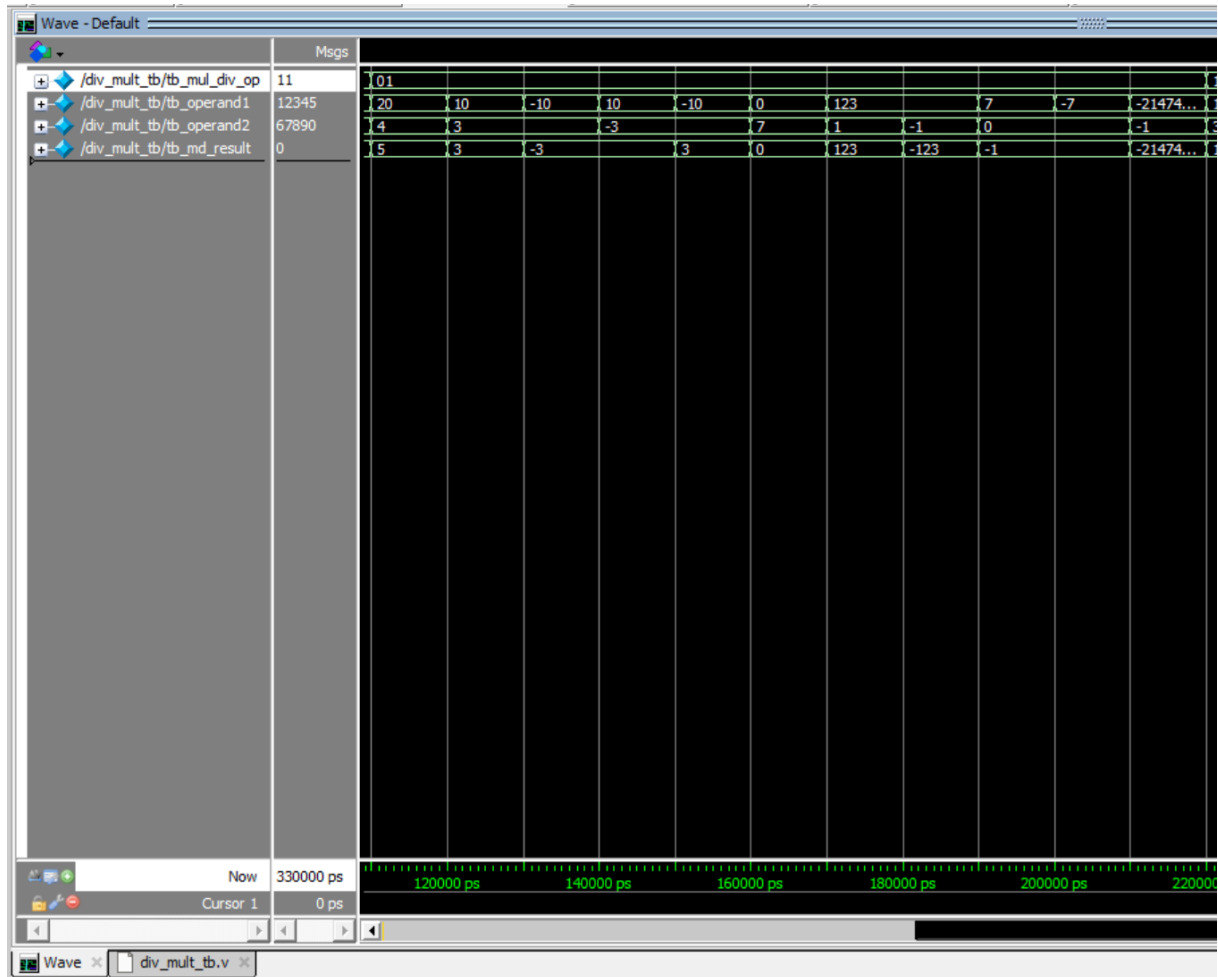
Figura 14 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação MUL.



Fonte: Simulação realizada pela autora em ModelSim.

A Figura 15 ilustra os testes para a operação de divisão (`tb_mul_div_op = 2'b01`). Por exemplo, a divisão de `tb_operand1 = 20` por `tb_operand2 = 4` (entre 110 ns e 120 ns) resulta em `tb_md_result = 5`. Casos com truncamento, como $10 / 3$, resultam em 3. Os casos especiais de divisão por zero (ex: $7 / 0$ entre 190 ns e 200 ns) corretamente produzem -1 (todos os bits '1'), e o caso de overflow `MIN_INT_S / -1` (entre 210 ns e 220 ns) resulta no próprio `MIN_INT_S`, conforme a especificação RISC-V.

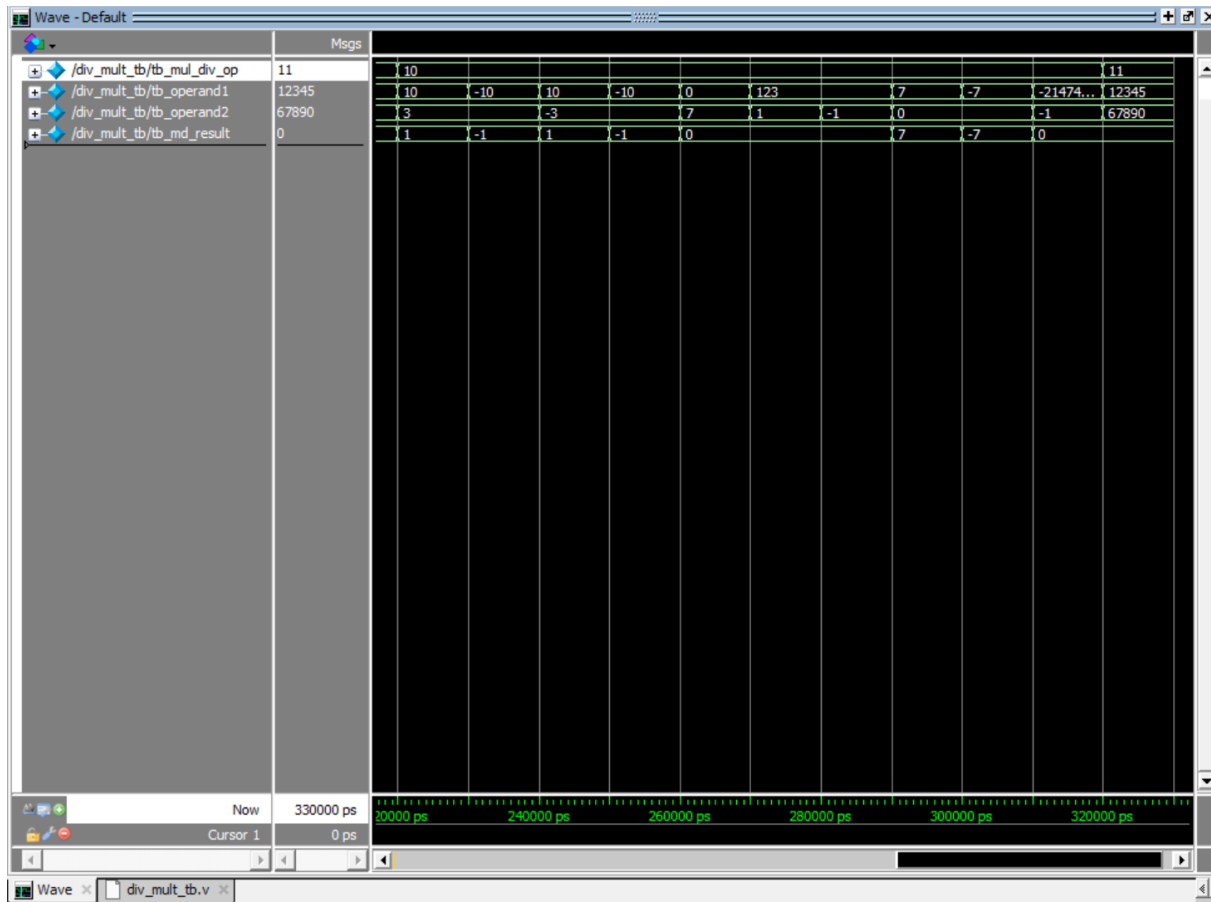
Figura 15 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação DIV.



Fonte: Simulação realizada pela autora em ModelSim.

Finalmente, a Figura 16 apresenta os resultados para a operação de resto (`tb_mul_div)(_op = 2'b10`). A operação $10 \% 3$ (entre 220 ns e 230 ns) resulta em 1. Testes com operandos negativos, como $-10 \% 3$, resultam em -1, seguindo a regra de que o sinal do resto é igual ao do dividendo. Os casos de resto da divisão por zero (ex: $7 \% 0$ entre 290 ns e 300 ns) corretamente retornam o dividendo (7), e o caso de overflow da divisão ($\text{MIN_INT_S} \% -1$ entre 310 ns e 320 ns) resulta em 0, conforme especificado. A última transição na forma de onda (após 320 ns) mostra a operação default/inválida (`tb_mul_div_op = 2'b11`) resultando em 0.

Figura 16 – Forma de onda da simulação da Unidade DIV/MULT: Testes da operação REM e Default.



Fonte: Simulação realizada pela autora em ModelSim.

A saída do console (Figura 13), que reporta "TODOS OS 33 TESTES DA div_mult PASSARAM!", juntamente com a análise detalhada das formas de onda (Figuras 14, 15 e 16), confirmam que a unidade de multiplicação e divisão foi implementada corretamente e se comporta conforme o esperado para todas as operações e casos de teste definidos, alinhando-se com as especificações da arquitetura RISC-V para as instruções `mul`, `div` e `rem`.

5.1.3 Simulação do Extensor de Imediato

O módulo `extensor.v`, responsável por gerar o valor de 32 bits estendido com sinal a partir dos campos de imediato presentes na instrução, foi verificado utilizando o testbench `extensor_tb.v`. Este testbench aplicou instruções representativas dos tipos I, S, B e J, com valores de imediato positivos e negativos, para garantir que a lógica de extração, rearranjo (para tipos B e J) e extensão de sinal estivesse correta. A decodificação do tipo de imediato a ser estendido é realizada internamente pelo módulo `extensor` com base no `opcode` da instrução de entrada.

A Figura 17 exibe a saída do console da simulação, onde cada linha representa um caso de teste, indicando o tipo de operação simulada, a instrução de entrada em hexadecimal, e o valor do imediato estendido resultante, tanto em hexadecimal quanto em decimal. Todos os 10 casos de teste definidos foram executados com sucesso, conforme indicado pela mensagem final "TODOS OS 10 TESTES DO EXTENSOR PASSARAM!".

Figura 17 – Saída do console da simulação do testbench do Extensor de Imediato.

```
# --- TESTES TIPO I ---
VSM13> run -all
# [ 1] SUCESSO:                Tipo I (ADDI): Imediato +5. Instrução: 00500093 -> Extendido: 00000005 ( 5)
# [ 2] SUCESSO:                Tipo I (LW): Imediato -4. Instrução: ffc12083 -> Extendido: ffffffff (-4)
# [ 3] SUCESSO:                Tipo I (JALR): Imediato +2047. Instrução: 7ff08067 -> Extendido: 000007ff ( 2047)
# [ 4] SUCESSO:                Tipo I (JALR): Imediato -2048. Instrução: 80008067 -> Extendido: ffffffff (-2048)
#
# --- TESTES TIPO S ---
# [ 5] SUCESSO:                Tipo S (SW): Imediato +8. Instrução: 00512423 -> Extendido: 00000008 ( 8)
# [ 6] SUCESSO:                Tipo S (SW): Imediato -8. Instrução: fe512c23 -> Extendido: ffffffff (-8)
#
# --- TESTES TIPO B ---
# [ 7] SUCESSO:                Tipo B (BEQ): Offset +20. Instrução: 00208a63 -> Extendido: 00000014 ( 20)
# [ 8] SUCESSO:                Tipo B (BGE): Offset -20. Instrução: fe20d6e3 -> Extendido: ffffffff (-20)
#
# --- TESTES TIPO J ---
# [ 9] SUCESSO:                Tipo J (JAL): Offset +1024. Instrução: 400000ef -> Extendido: 00000400 ( 1024)
# [10] SUCESSO:                Tipo J (JAL): Offset -1024. Instrução: c01ff0ef -> Extendido: fffffc00 (-1024)
#
# Testes do Extensor de Imediato finalizados.
# TODOS OS 10 TESTES DO EXTENSOR PASSARAM!
# ** Note: $finish : C:/Users/User/Desktop/EXTENSOR/testbench/extensor_tb.v(161)
# Time: 110 ns Iteration: 0 Instance: /extensor_tb
# 1
```

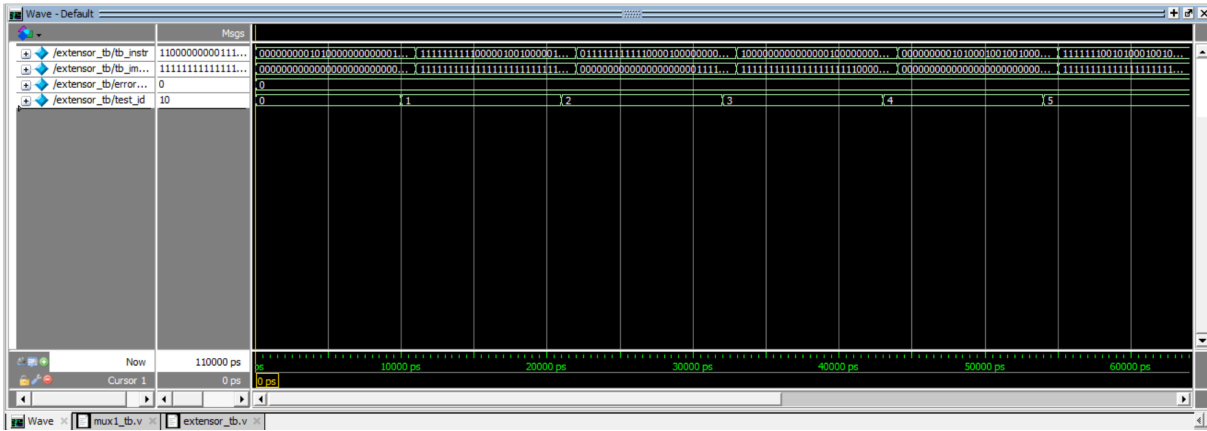
Fonte: Simulação realizada pela autora em ModelSim.

As formas de onda correspondentes, apresentadas nas Figuras 18 e 19, ilustram o comportamento temporal dos sinais. Observa-se que para cada `tb_instr` aplicada, a saída `tb_imm_ext` é atualizada corretamente após um pequeno atraso combinacional.

Na Figura 18, durante os primeiros 60 ns, são testados os formatos Tipo I e Tipo S.

- Por exemplo, entre 0 ns e 10 ns, a instrução `0x00500093` (representando `ADDI x1, x0, 5`) resulta em `tb_imm_ext = 0x00000005` (5 decimal).
- Entre 10 ns e 20 ns, a instrução `0xFFFF0093` (`ADDI x1, x0, -1`) produz `tb_imm_ext = 0xFFFFFFFF` (-1 decimal), demonstrando a correta extensão de sinal para o Tipo I.
- De forma similar, para o Tipo S, entre 40 ns e 50 ns, a instrução `0x00512423` (representando `SW` com imediato +8) gera `tb_imm_ext = 0x00000008` (8 decimal).

Figura 18 – Forma de onda da simulação do Extensor de Imediato (Parte 1): Testes dos tipos I e S.



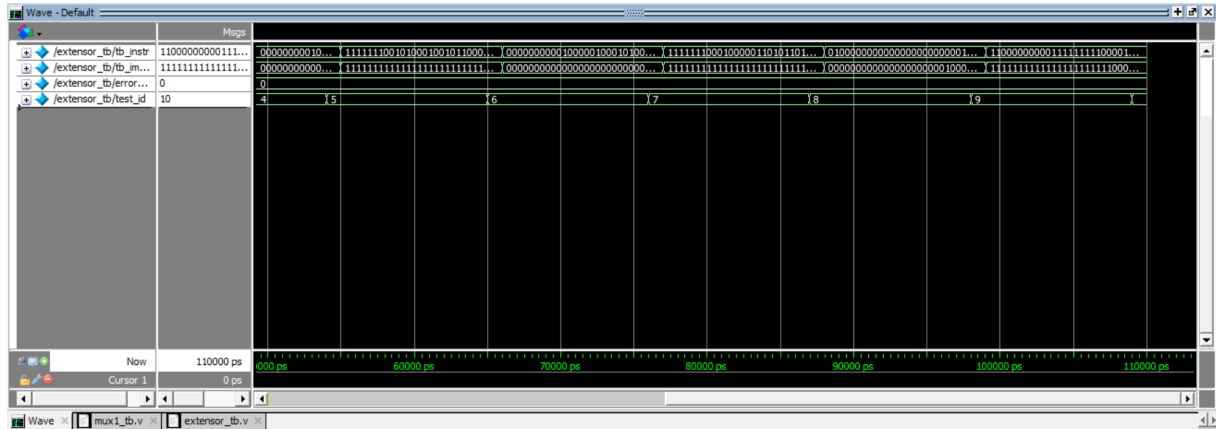
Fonte: Simulação realizada pela autora em ModelSim.

A Figura 19 continua a simulação, mostrando os testes para os formatos Tipo B e Tipo J, que possuem uma codificação de imediato mais complexa.

- Entre 60 ns e 70 ns, a instrução 0x00208a63 (representando BEQ com offset +20) produz `tb_imm_ext = 0x00000014` (20 decimal).
- Para um offset negativo no Tipo B, entre 70 ns e 80 ns, a instrução 0xFE2006E3 (BEQ com offset -20) resulta corretamente em `tb_imm_ext = 0xFFFFFEC` (-20 decimal).
- Os testes para o Tipo J, como o JAL com offset +1024 (instrução 0x400000EF entre 80 ns e 90 ns), geram `tb_imm_ext = 0x00000400` (1024 decimal), e para o offset -1024 (instrução 0xC01FF0EF entre 90 ns e 100 ns) resultam em `tb_imm_ext = 0xFFFFFC00` (-1024 decimal).

O último teste (após 100 ns) com uma instrução de tipo inválido para o extensor (um tipo R, 0xDEADBEEF no console, embora a waveform possa mostrar a última instrução válida para Tipo J se não houve transição para um opcode R-type) resultou em um imediato de 0x0 ou x (indefinido) conforme o comportamento default especificado no módulo, que é o esperado.

Figura 19 – Forma de onda da simulação do Extensor de Imediato (Parte 2): Testes dos tipos B e J.



Fonte: Simulação realizada pela autora em ModelSim.

A combinação da saída do console, que indica sucesso em todos os casos, com a análise visual das formas de onda confirma que o módulo Extensor de Imediato está corretamente decodificando os diferentes formatos de instrução baseados no `opcode` e gerando os valores de imediato estendidos de 32 bits conforme a especificação RISC-V.

5.1.4 Simulação do Somador de Endereço de Destino (PC Target Adder)

O módulo `pc_target_adder.v` é um somador combinacional dedicado a calcular o endereço de destino para instruções de desvio e saltos diretos, somando o valor atual do PC (`in_pc_out`) com um offset imediato estendido (`in_imm_extended`). A sua funcionalidade foi verificada utilizando o testbench `pc_target_adder_tb.v`.

A Figura 20 apresenta a saída do console da simulação, que demonstra o sucesso de todos os cinco casos de teste aplicados. Para cada teste, são mostrados os valores de entrada para o PC e o imediato, e o resultado **Target** (saída do somador) obtido, que correspondeu ao valor esperado.

Figura 20 – Saída do console da simulação do testbench do Somador PC Target.

```

VSIM 24> run -all
# Iniciando Testbench para pc_target_adder...
# [11000 ns][ 1] SUCESSO: PC=1000h, Imm=+20d. PC=00001000, Imm=00000014 -> Target: 00001014
# [22000 ns][ 2] SUCESSO: PC=2040h, Imm=-20d. PC=00002040, Imm=ffffffec -> Target: 0000202c
# [33000 ns][ 3] SUCESSO: PC=0, Imm=+1024d. PC=00000000, Imm=00000400 -> Target: 00000400
# [44000 ns][ 4] SUCESSO: PC=8000h, Imm=-1024d. PC=00008000, Imm=fffffc00 -> Target: 00007c00
# [55000 ns][ 5] SUCESSO: PC=FFFFF000h, Imm=+4096d (Wrap around). PC=fffff000, Imm=00001000 -> Target: 00000000
#
# Testes do pc_target_adder finalizados.
# TODOS OS 5 TESTES DO PC_TARGET_ADDER PASSARAM!

```

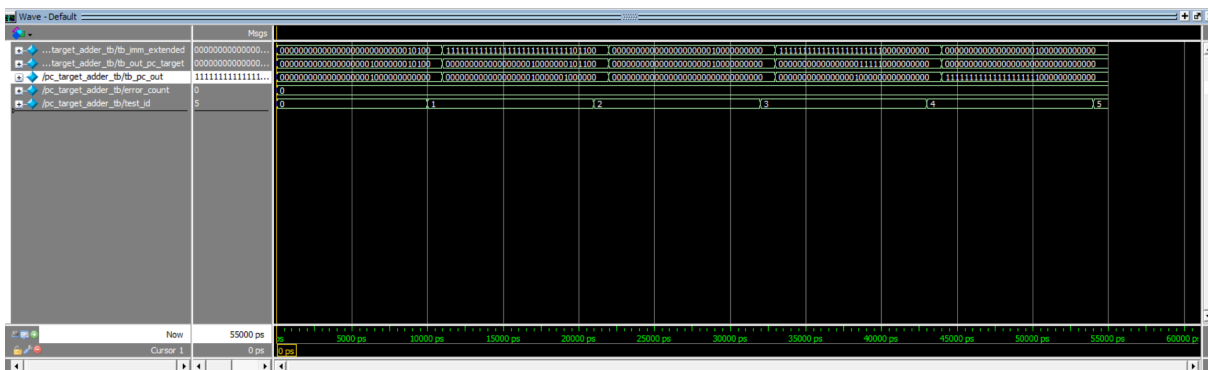
Fonte: Simulação realizada pela autora em ModelSim.

A Figura 21 ilustra as formas de onda da simulação deste somador. Observa-se o comportamento puramente combinacional do módulo, onde a saída `pc_target_adder_tb/tb_out_pc_target` é atualizada em resposta às mudanças nas entradas `pc_target_adder_tb/tb_pc_out` e `pc_target_adder_tb/tb_imm_extended`.

- No primeiro teste (entre 0 ns e 11 ns), com `tb_pc_out = 0x00001000` e `tb_imm_extended = 0x00000014` (20 decimal), a saída `tb_out_pc_target` é corretamente `0x00001014`.
- No segundo teste (entre 11 ns e 22 ns), com `tb_pc_out = 0x00002040` e `tb_imm_extended = 0xFFFFFEC` (-20 decimal), a saída é `0x0000202C`.
- O terceiro teste (entre 22 ns e 33 ns) demonstra a soma de `tb_pc_out = 0x0` com `tb_imm_extended = 0x00000400` (1024 decimal), resultando em `0x00000400`.
- O quarto teste (entre 33 ns e 44 ns) com `tb_pc_out = 0x00008000` e `tb_imm_extended = 0xFFFFFC00` (-1024 decimal) produz `0x00007C00`.
- Finalmente, o quinto teste (entre 44 ns e 55 ns) verifica o comportamento de "wrap-around" da aritmética de 32 bits: `tb_pc_out = 0xFFFFF000` somado com `tb_imm_extended = 0x00001000` (4096 decimal) resulta em `0x00000000`.

Todas as saídas observadas nas formas de onda estão em concordância com os resultados esperados e com a saída do console.

Figura 21 – Forma de onda da simulação do Somador PC Target.



Fonte: Simulação realizada pela autora em ModelSim.

Os resultados da simulação confirmam que o módulo `pc_target_adder` executa corretamente a adição de 32 bits necessária para calcular os endereços de destino para desvios e saltos diretos, tratando adequadamente valores positivos, negativos e situações de overflow (wrap-around) da aritmética de 32 bits.

5.1.5 Simulação do Banco de Registradores

O módulo `bancoderegistradores.v`, que implementa os 32 registradores de propósito geral de 32 bits da arquitetura RISC-V, foi submetido a uma série de testes utilizando o testbench `bancoderegistradores_tb.v`. O objetivo foi validar as operações de leitura e escrita, a funcionalidade do sinal de reset, a regra especial do registrador `x0` (que sempre lê zero e não pode ser escrito) e a correta operação do sinal de habilitação de escrita (`reg_write`).

A Figura 22 apresenta a saída do console da simulação, que resume os resultados dos 10 casos de teste executados, indicando sucesso em todas as verificações.

Figura 22 – Saída do console da simulação do testbench do Banco de Registradores.

```

Transcript
VSI13> run -all
# Iniciando Testbench para bancoderegistradores...
#
# --- Teste de Reset ---
# [22000 ns][ 1] SUCESSO Leitura:          Leitura de x0 e outro reg's Reset. Addr1= 1->D1=00000000, Addr2= 2->D2=00000000
# [23000 ns][ 2] SUCESSO Leitura:          Leitura de x0 e outro reg's Reset. Addr1= 0->D1=00000000, Addr2=15->D2=00000000
#
# --- Teste de Escrita e Leitura Simples ---
# [27000 ns][ 3] SUCESSO Leitura:          Ler x1 e x2's escrita, x2 n'fo alterado. Addr1= 1->D1=aaaaaaaa, Addr2= 2->D2=00000000
# [37000 ns][ 4] SUCESSO Leitura:          Ler x1 e x2's escrita em x2. Addr1= 1->D1=aaaaaaaa, Addr2= 2->D2=bbbbbbbb
#
# --- Teste de Leitura de x0 ---
# [38000 ns][ 5] SUCESSO Leitura:          Ler x0 (deve ser 0) e x1. Addr1= 0->D1=00000000, Addr2= 1->D2=aaaaaaaa
#
# --- Teste de Tentativa de Escrita em x0 ---
# [47000 ns][ 6] SUCESSO Leitura:          Ler x0's tentativa de escrita (deve ser 0). Addr1= 0->D1=00000000, Addr2= 1->D2=aaaaaaaa
#
# --- Teste com RegWrite Desabilitado ---
# [57000 ns][ 7] SUCESSO Leitura:          Ler x3's tentativa de escrita com RegWrite=0 (x3 deve ser 0). Addr1= 3->D1=00000000, Addr2= 1->D2=aaaaaaaa
#
# --- Teste de Sobreescrita de Registrador ---
# [67000 ns][ 8] SUCESSO Leitura:          Ler x1 sobrescrito e x2. Addr1= 1->D1=11112222, Addr2= 2->D2=bbbbbbbb
#
# --- Teste de Leitura Simultanea ---
# [87000 ns][ 9] SUCESSO Leitura:          Leitura de x5 e x10. Addr1= 5->D1=05050505, Addr2=10->D2=10101010
# [88000 ns][10] SUCESSO Leitura:          Leitura de x1 e x10. Addr1= 1->D1=11112222, Addr2=10->D2=10101010
#
# Testes do Banco de Registradores finalizados.
# TODOS OS 10 TESTES DO BANCO DE REGISTRADORES PASSARAM!
# ** Note: $finish      : C:/Users/User/Desktop/BANCODEREG/test/bancoderegistradores_tb.v(149)
Project: bancoderegistradores Now: 88 ns Delta: 0 sim:/bancoderegistradores_tb/#INITIAL#61
  
```

Fonte: Simulação realizada pela autora em ModelSim.

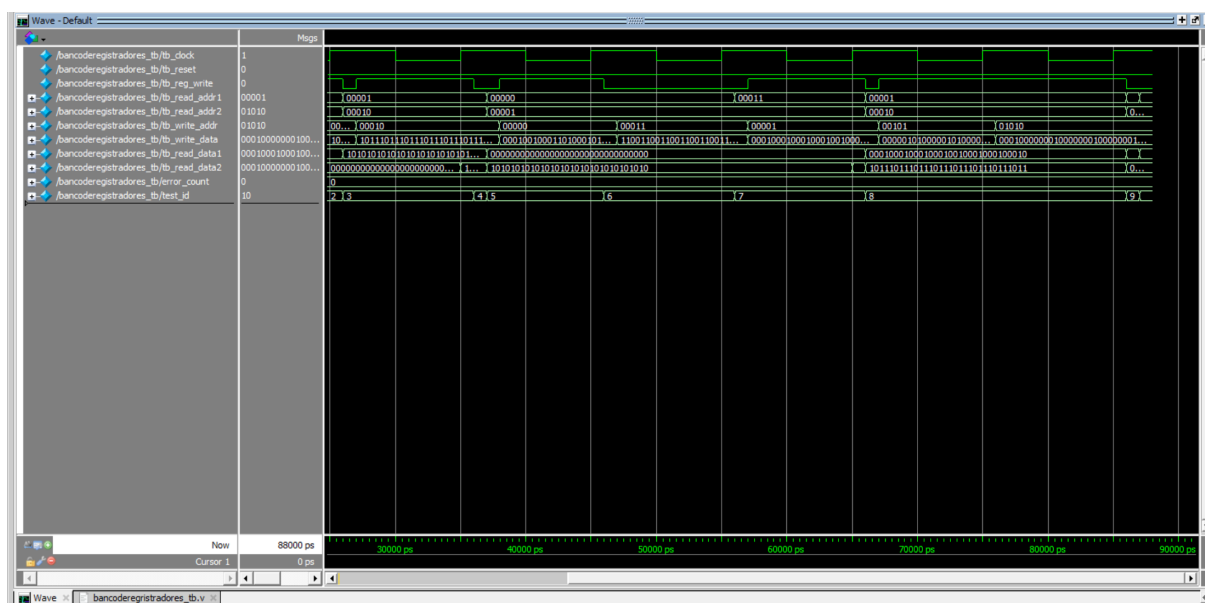
As formas de onda da simulação, ilustradas nas Figuras 23 e 24, fornecem uma visão detalhada do comportamento temporal dos sinais do banco de registradores.

Na Figura 23, podemos observar:

- **Reset (0 ns a 20 ns):** O sinal `tb_reset` é mantido em nível alto, e na borda de subida do clock seguinte à sua desativação (em 20 ns), a leitura dos registradores (ex: `tb_read_addr1=1`, `tb_read_addr2=2`) através de `tb_read_data1` e `tb_read_data2` mostra corretamente o valor `0x00000000`, confirmando a inicialização.
- **Escrita e Leitura Simples (20 ns a 40 ns):**
 - Em 25 ns (após uma borda de clock com `tb_reg_write=1`, `tb_write_addr=1`, `tb_write_data=0xAAAAAAAA`), a subsequente leitura de `x1` (via `tb_read_addr1=1`) mostra `0xAAAAAAAA` em `tb_read_data1`.

x10; x1 e x10). As saídas `tb_read_data1` e `tb_read_data2` refletem corretamente os conteúdos dos endereços de leitura aplicados, demonstrando o funcionamento independente das duas portas de leitura.

Figura 24 – Forma de onda da simulação do Banco de Registradores (Parte 2): Testes de escrita desabilitada, sobrescrita e leitura simultânea.



Fonte: Simulação realizada pela autora em ModelSim.

Os resultados abrangentes obtidos, tanto na saída do console (Figura 22) quanto nas formas de onda (Figuras 23 e 24), indicam que o módulo do banco de registradores opera conforme especificado. Ele lida corretamente com o reset, as operações de leitura e escrita (incluindo a proteção de `x0` e a dependência do sinal `reg_write`), e permite o acesso simultâneo através de suas duas portas de leitura.

5.1.6 Simulação do Contador de Programa (PC)

O Contador de Programa (`program_counter.v`) é um registrador fundamental que armazena o endereço da instrução a ser executada. Seu testbench, `program_counter_tb.v`, foi projetado para verificar a inicialização correta durante o reset e a capacidade de carregar novos valores de `pc_in` na borda de subida do clock.

A Figura 25 apresenta a saída do console da simulação, que detalha cada etapa do teste e confirma que os valores de saída (`PC_Out`) corresponderam aos esperados após o reset e após cada carga de um novo valor de `pc_in`. Todos os testes foram concluídos com sucesso.

Figura 25 – Saída do console da simulação do testbench do Contador de Programa.

```

VSIM 6> run -all
# Iniciando Testbench para Program Counter...
# [20000 ns] Reset: PC_Out = 00000000 (Esperado 00000000)
# [26000 ns] Carga 1: PC_In=00000004, PC_Out=00000004
# [36000 ns] Carga 2: PC_In=00000008, PC_Out=00000008
# [46000 ns] Carga 3: PC_In=1234abcd, PC_Out=1234abcd
# Testes do Program Counter finalizados.
# ** Note: $finish      : C:/Users/User/Desktop/PC/teste/program_counter_tb.v(41)
#   Time: 46 ns  Iteration: 0  Instance: /program_counter_tb

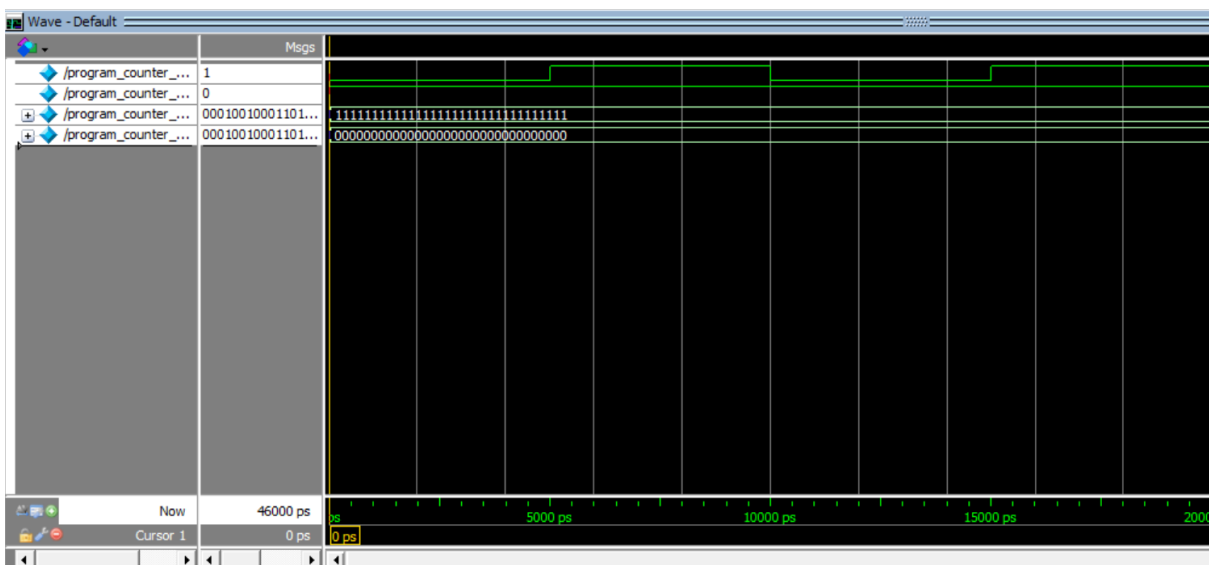
```

Fonte: Simulação realizada pela autora em ModelSim.

As formas de onda correspondentes à simulação do Contador de Programa são mostradas nas Figuras 26 e 27. Na Figura 26, observa-se o comportamento inicial:

- **Reset (0 ns a 20 ns):** O sinal `/program_counter_tb/reset_tb` é mantido em nível alto (1). Durante este período, a saída `/program_counter_tb/uut/pc_out` (ou `/program_counter_tb/pc_out_tb`) permanece em `0x00000000`, conforme esperado pela lógica de reset. A entrada `pc_in_tb` é ignorada.
- **Primeira Carga (após 20 ns):** Logo após `reset_tb` ir para 0 (em 20 ns), no próximo pulso de subida do clock (aproximadamente em 25 ns, considerando que o clock tem período de 10ns e pode ter iniciado em 0 ou 5ns), o PC (`pc_out_tb`) é carregado com o valor de `pc_in_tb`, que foi configurado para `0x00000004`. A forma de onda confirma que `pc_out_tb` assume este valor.

Figura 26 – Forma de onda da simulação do Contador de Programa (Parte 1): Teste de Reset e primeira carga.



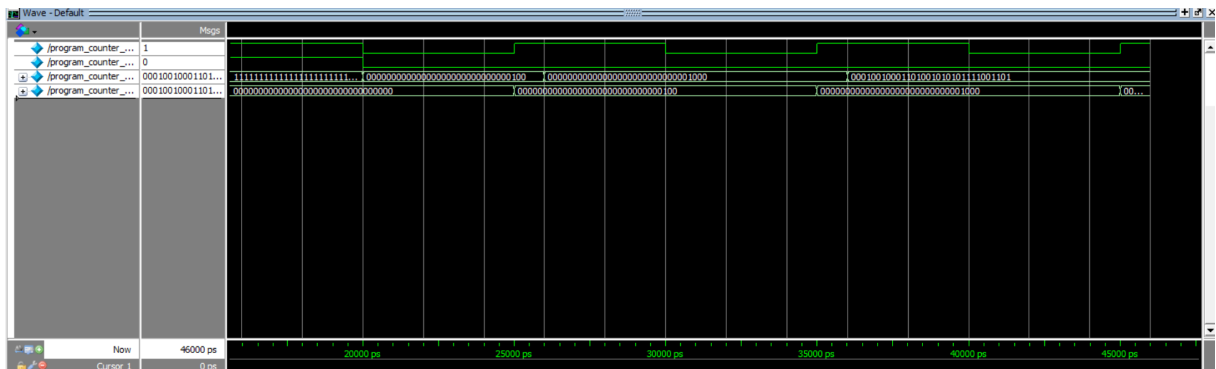
Fonte: Simulação realizada pela autora em ModelSim.

A Figura 27 continua a simulação, mostrando cargas subsequentes:

- **Segunda Carga (após 25 ns):** Após a primeira carga, `pc_in_tb` é alterado para `0x00000008`. Na borda de subida do clock seguinte (aproximadamente em 35 ns), `pc_out_tb` é atualizado para `0x00000008`.
- **Terceira Carga (após 35 ns):** Similarmente, `pc_in_tb` é configurado para `0x1234ABCD`. Na próxima borda de subida do clock (aproximadamente em 45 ns), `pc_out_tb` reflete este novo valor.

As transições na forma de onda para `pc_out_tb` ocorrem precisamente nas bordas de subida do `clock` (quando `reset_tb` está baixo), e o valor carregado é sempre o que estava presente em `pc_in_tb` no momento da borda.

Figura 27 – Forma de onda da simulação do Contador de Programa (Parte 2): Cargas subsequentes.



Fonte: Simulação realizada pela autora em ModelSim.

Os resultados da simulação, validados tanto pela saída do console (Figura 25) quanto pela análise das formas de onda (Figuras 26 e 27), demonstram que o módulo do Contador de Programa funciona corretamente, inicializando com o reset e atualizando seu valor de saída (`pc_out`) com a entrada (`pc_in`) de forma síncrona com o clock.

5.1.7 Simulação do Somador PC+4

O módulo `pc_plus_4_adder.v` é um componente combinacional simples cuja função é calcular o endereço da próxima instrução em uma sequência linear, adicionando 4 ao valor atual do Contador de Programa (PC). O testbench `pc_plus_4_adder_tb.v` foi utilizado para verificar esta funcionalidade, aplicando diferentes valores de entrada para `current_pc` e observando a saída `pc_plus_4`.

A Figura 28 apresenta a saída do console da simulação, detalhando os valores de entrada, a saída esperada e a saída obtida para cada caso de teste. Todos os testes foram concluídos com sucesso, indicando que o somador está operando corretamente.

Figura 28 – Saída do console da simulação do testbench do Somador PC+4.

```
# Iniciando Testbench para Program Counter...
# [20000 ns] Reset: PC_Out = 00000000 (Esperado 00000000)
# [26000 ns] Carga 1: PC_In=00000004, PC_Out=00000004
# [36000 ns] Carga 2: PC_In=00000008, PC_Out=00000008
# [46000 ns] Carga 3: PC_In=1234abcd, PC_Out=1234abcd
# Testes do Program Counter finalizados.
# ** Note: $finish      : C:/Users/User/Desktop/PC/teste/program_counter_tb.v(41)
#   Time: 46 ns  Iteration: 0  Instance: /program_counter_tb
```

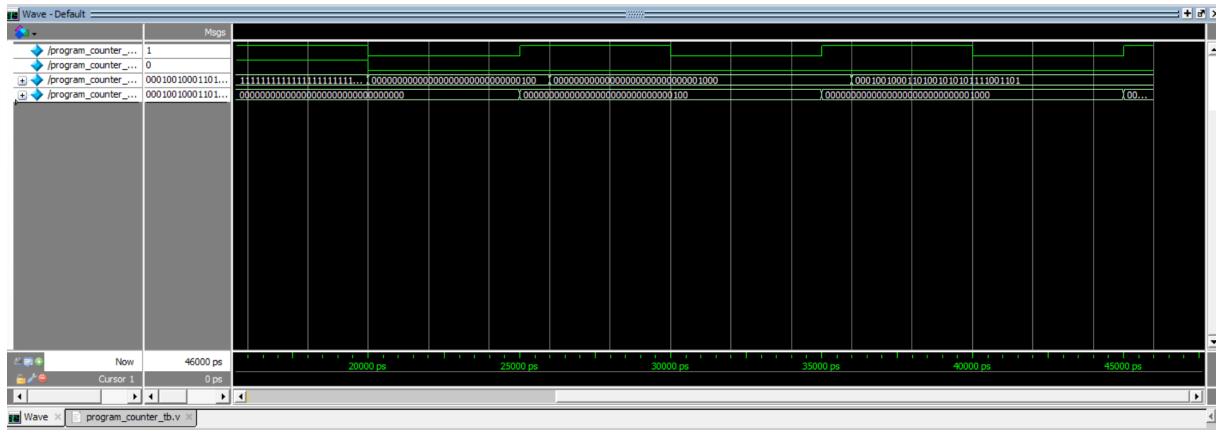
Fonte: Simulação realizada pela autora em ModelSim.

As formas de onda da simulação, ilustradas na Figura 29, demonstram o comportamento combinacional do somador.

- No primeiro intervalo (ex: 0 ns a 10 ns), com `/pc_plus_4_adder_tb/current_pc_tb = 0x00000000`, a saída `/pc_plus_4_adder_tb/pc_plus_4_tb` é imediatamente `0x00000004`.
- Quando `current_pc_tb` muda para `0x00000004` (entre 10 ns e 20 ns), a saída `pc_plus_4_tb` atualiza para `0x00000008`.
- Em um caso próximo ao limite superior (entre 20 ns e 30 ns), com `current_pc_tb = 0xFFFFFFFF8`, a saída é corretamente `0xFFFFFFFFC`.
- Finalmente, o teste de "wrap-around" (entre 30 ns e 40 ns), com `current_pc_tb = 0xFFFFFFFFC`, demonstra que a adição de 4 resulta em `0x00000000` devido ao overflow de 32 bits, que é o comportamento esperado para a aritmética de endereços.

Em todos os casos, a saída `pc_plus_4_tb` reflete a soma de `current_pc_tb + 4` sem atrasos significativos além da propagação combinacional.

Figura 29 – Forma de onda da simulação do Somador PC+4.



Fonte: Simulação realizada pela autora em ModelSim.

A análise da saída do console e das formas de onda confirma que o módulo somador PC+4 funciona conforme o esperado, fornecendo corretamente o endereço da próxima instrução sequencial.

5.2 Simulação da Unidade de Processamento Integrada - Unidade-Processamento

Após a verificação individual dos componentes da unidade de processamento, o módulo integrado `UnidadeProcessamento.v` foi simulado utilizando o testbench `Unidade-Processamento_tb.v`. Conforme descrito anteriormente, este testbench aplica manualmente os sinais de controle que seriam normalmente gerados pela Unidade de Controle, permitindo a verificação do fluxo de dados e da funcionalidade do caminho de dados principal para a execução de instruções representativas.

A Figura 30 apresenta a saída do console gerada durante a simulação da `UnidadeProcessamento`. A sequência de testes incluiu a escrita de valores iniciais em registradores (`x1` e `x2`) através da simulação de instruções `ADDI`, seguida pela execução de uma instrução `ADD` e uma instrução `MUL` utilizando esses registradores como operandos. Finalmente, foram realizadas leituras para verificar os resultados armazenados. Todos os passos da simulação produziram os resultados esperados, conforme indicado na saída do console.

Figura 30 – Saída do console da simulação do testbench da UnidadeProcessamento.

```
# Iniciando Testes da UnidadeProcessamento...
# [36000 ns] ADDI x1,x0,10: instruction=00a00093, core_result_out=      10 (esperado 10).
# [46000 ns] ADDI x2,x0,20: instruction=01400113, core_result_out=      20 (esperado 20).
# [56000 ns] ADD x3,x1,x2: instruction=002081b3, ReadData1=      10, ReadData2=      20, core_result_out=      30, Zero=0
# [66000 ns] MUL x5,x1,x2: instruction=022082b3, ReadData1=      10, ReadData2=      20, core_result_out=     200
# [86000 ns] Leitura Pos-Testes: ReadData1 (x1) =      10 (Esperado 10), ReadData2 (x3) =      30 (Esperado 30)
# [96000 ns] Leitura Pos-Testes: ReadData1 (x5) =     200 (Esperado 200), ReadData2 (x0) =       0 (Esperado 0)
#
# Testes da UnidadeProcessamento finalizados.
# *** Finish *** C:/Users/Thay/OneDrive/Desktop/Projetos/Projetos/Thay/UnidadeProcessamento_tb_v1133
```

Fonte: Simulação realizada pela autora em ModelSim.

A Figura 31 ilustra as formas de onda obtidas durante esta sequência de testes. A análise detalhada da figura permite observar o comportamento dinâmico dos sinais e o fluxo de dados:

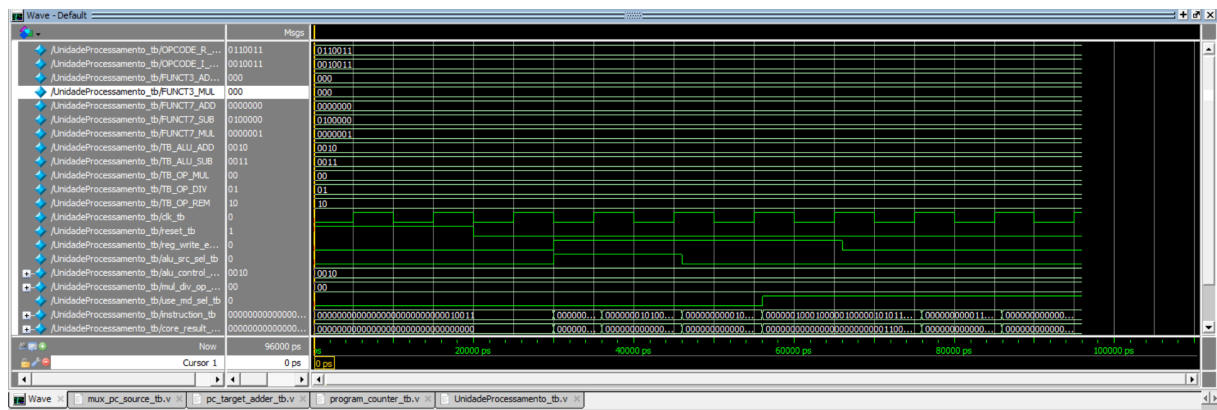
- **Inicialização e Reset (não totalmente visível, mas precede os testes):** O testbench inicia com um período de reset, seguido pela configuração dos sinais de controle para as operações subsequentes.
- **Simulação de ADDI x1, x0, 10 (aproximadamente até 36 ns na saída do console):** O sinal UnidadeProcessamento_tb/instruction_tb é configurado para 0x00A00093. Os sinais de controle relevantes (reg_write_en_tb=1, alu_src_sel_tb=1 para selecionar o imediato, alu_control_op_tb para ADD, use_md_sel_tb=0 para usar a ULA) são aplicados. A forma de onda mostra que a saída /UnidadeProcessamento_tb/core_result_out_tb assume o valor 10 (decimal). Este valor é então escrito no registrador x1 na borda de subida do clock seguinte.
- **Simulação de ADDI x2, x0, 20 (aproximadamente até 46 ns na saída do console):** De forma análoga, instruction_tb é configurada para 0x01400113. Os sinais de controle permanecem configurados para uma operação ADDI. A saída core_result_out_tb mostra 20 (decimal), que é escrito em x2.
- **Simulação de ADD x3, x1, x2 (aproximadamente até 56 ns na saída do console):** A instruction_tb muda para 0x002081B3. Sinais de controle como alu_src_sel_tb mudam para 0 (para selecionar ReadData2), e alu_control_op_tb permanece configurado para ADD. As saídas do banco de registradores, /UnidadeProcessamento_tb/reg_read_data1_tb e /UnidadeProcessamento_tb/reg_read_data2_tb, apresentam os valores lidos de x1 (10) e x2 (20), respectivamente. A ULA processa esses valores, e core_result_out_tb mostra o resultado 30 (decimal), que é escrito em x3. A alu_zero_flag_tb permanece em 0.
- **Simulação de MUL x5, x1, x2 (aproximadamente até 66 ns na saída do console):** A instruction_tb é configurada para 0x022082B3. Os sinais de controle são

ajustados para uma operação MUL: `mul_div_op_sel_tb` é configurado para MUL, e `use_md_sel_tb` muda para 1 (para selecionar a saída da unidade DIV/MULT). As entradas `reg_read_data1_tb` (10) e `reg_read_data2_tb` (20) são processadas pela unidade DIV/MULT. A saída `core_result_out_tb` apresenta o valor 200 (decimal), que é escrito em `x5`.

- **Leituras de Verificação (após 66 ns):** O sinal `reg_write_en_tb` é desabilitado. A `instruction_tb` é modificada para direcionar as leituras aos registradores `x1`, `x3`, `x5` e `x0`. As saídas `reg_read_data1_tb` e `reg_read_data2_tb` confirmam que os valores corretos (10, 30, 200, 0 respectivamente) foram armazenados e podem ser lidos.

É importante notar que os sinais de controle como TB_OPCODE..., TB_FUNCT3..., TB_FUNCT7..., TB_ALU..., TB_OP... (definidos como ‘localparam‘ no testbench) são aplicados às entradas de controle do DUT (alu_control_op_tb, mul_div_op_sel_tb, etc.) e são visíveis na forma de onda, demonstrando como o testbench emula a Unidade de Controle.

Figura 31 – Forma de onda da simulação da UnidadeProcessamento integrada.



Fonte: Simulação realizada pela autora em ModelSim.

A simulação do módulo **UnidadeProcessamento** integrado, mesmo com o controle manual dos sinais de operação, valida com sucesso a interconexão dos seus componentes principais e o fluxo de dados para a execução de instruções aritméticas, lógicas e de multiplicação. Os resultados demonstram que o núcleo de processamento é capaz de ler operandos do banco de registradores, processá-los através da ULA ou da unidade DIV/MULT (com a correta seleção via MUXes), e disponibilizar o resultado para ser escrito de volta no banco de registradores. Esta etapa confirma a correteude funcional da unidade de dados principal do processador.

5.2.0.0.1 Assembler RISC-V para Geração de Código de Máquina:

Durante o processo de validação da unidade de processamento, foi necessário elaborar códigos binários representando instruções RISC-V compatíveis com a arquitetura projetada. Para facilitar esse processo e minimizar erros manuais, desenvolveu-se um *assembler* simples em Python que recebe instruções em formato Assembly RV32I/M e as traduz diretamente para o formato binário de 32 bits correspondente. O programa cobre os principais formatos da ISA, incluindo instruções do tipo R, I, S, B e J, além de suportar instruções estendidas de multiplicação e divisão. O script foi implementado com base na estrutura dos campos da ISA (como `opcode`, `funct3`, `funct7`, registradores e deslocamentos), e seu uso foi fundamental para automatizar a geração dos valores binários inseridos na memória de instruções do processador simulado. Assim, foi possível escrever e testar programas em Assembly de forma prática e confiável, como no exemplo do cálculo da sequência de Fibonacci. O Assembler construído se encontra para consulta em [D](#).

5.3 Verificação em Hardware (FPGA)

Após a verificação funcional do processador RISC-V por meio de simulações unitárias e integradas, a etapa final consistiu na síntese e implementação do projeto completo em uma plataforma FPGA real — a placa DE2-115, baseada no FPGA Altera Cyclone IV. O objetivo era validar o comportamento do sistema sob condições reais de operação, com interação física com periféricos de entrada e saída.

5.3.1 Configuração do Teste

Para realizar os testes em hardware, foi desenvolvido o módulo `toplevel.v`, responsável por instanciar o processador e conectá-lo aos periféricos físicos disponíveis na placa. O módulo implementou a seguinte interface com o usuário:

- 8 switches (SW) e o botão KEY[1] (item) foram utilizados como entrada do sistema, simulando o periférico de entrada mapeado em memória (MMIO Input).
- O botão KEY[0] foi utilizado como sinal de reset.
- O resultado de saída, proveniente do endereço mapeado do display (MMIO Output), foi exibido nos 8 displays de 7 segmentos (HEX0 a HEX7) após conversão do valor binário para BCD.

Dois programas de teste escritos em assembly RISC-V foram montados, convertidos para formato hexadecimal e carregados na memória de instrução do processador:

- Um programa de entrada e saída simples (I/O).

- Um programa de cálculo do número de Fibonacci.

5.3.2 Resultados do Teste de I/O

O primeiro teste foi projetado para validar o caminho de dados de entrada e saída. O programa consistia em realizar a leitura de um valor dos switches, controlada pelo botão "Enter", armazenar esse valor em um registrador por fim escrever o valor diretamente no endereço do display.

Resultado observado: O teste foi bem-sucedido. Ao configurar um valor específico nos switches (exemplo: 123), pressionar o botão "Enter" e em seguida o botão de reset, o valor 00000123 foi corretamente exibido nos displays de 7 segmentos. Isso confirmou que as instruções lw e sw voltadas para os endereços MMIO, bem como os módulos de decodificação e os periféricos conectados, estão operando conforme esperado.

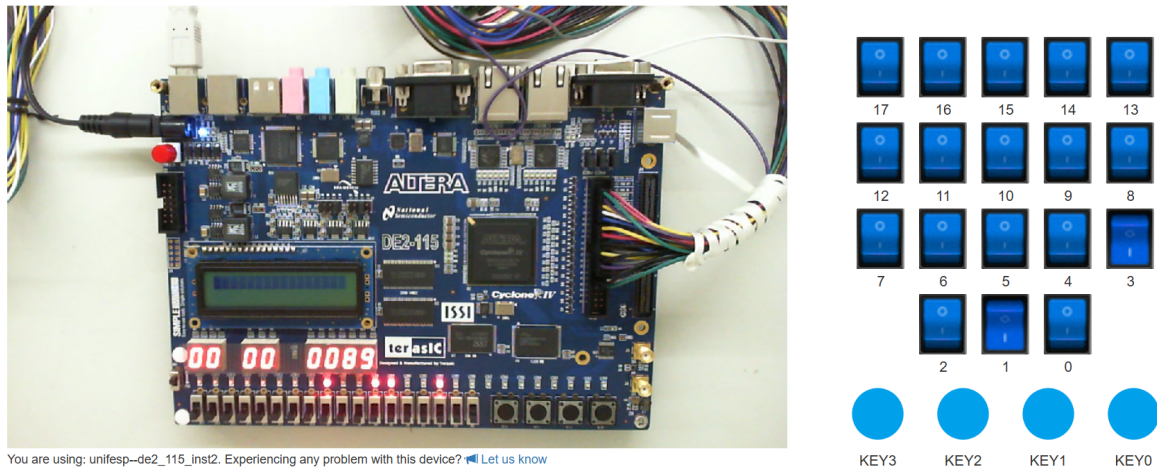
5.3.3 Resultados do Teste de Fibonacci

O segundo teste, mais complexo, visou validar a execução de um algoritmo com laços, dependências de dados e maior atividade da ULA e do banco de registradores. O programa recebia um valor N via switches e executava a versão sequencial do cálculo de Fibonacci até o N-ésimo termo, armazenando o resultado final no registrador de saída.

Resultado observado: Com $N = 10$ configurado via switches, o programa executou corretamente e exibiu o valor 89 nos displays como na imagem 32, isso validou:

- A correta leitura de entrada via lw.
- A execução de laços bne.
- As instruções aritméticas add e addi.
- A consistência de escrita e leitura do banco de registradores.
- A escrita final no endereço MMIO do display.

Figura 32 – FPGA mostrando o final da sequência inputada pelo usuário.



Fonte: Simulação realizada pela autora no FPGA remoto.

5.3.4 Conclusão da Seção

A validação em hardware real por meio da execução de múltiplos programas de teste comprova, de forma prática, que o processador RISC-V RV32IM desenvolvido é funcional, robusto e se comporta conforme a especificação. Os testes em FPGA permitiram verificar a integração entre os módulos de controle, a ULA, o banco de registradores, os periféricos mapeados em memória e a lógica de controle de fluxo, consolidando o sucesso do projeto.

5.4 Análise Geral e Comprovação de Funcionamento

As simulações funcionais realizadas, tanto para os módulos individuais da unidade de processamento quanto para sua integração no módulo `processing_core`, demonstram que a implementação em Verilog se comporta corretamente conforme as especificações da ISA RISC-V RV32IM (simplificada) e o datapath arquitetado no Ponto de Checagem 1. Os resultados obtidos nas formas de onda e nas saídas textuais dos testbenches corresponderam aos valores esperados para uma variedade de casos de teste, cobrindo as diferentes operações e cenários de entrada.

Este trabalho apresentou o projeto, implementação e validação de um processador compatível com o conjunto de instruções RISC-V RV32IM. Foram abordadas todas as etapas, desde a definição da arquitetura, passando pelo projeto do caminho de dados e da unidade de controle, até a tradução para Verilog, simulação e, por fim, implementação em hardware. O sistema completo foi testado e validado com programas reais na plataforma FPGA DE2-115.

Por fim, todos os objetivos do projeto foram cumpridos com sucesso. A arquitetura proposta foi implementada em um modelo monociclo, e sua funcionalidade foi comprovada por meio de simulações modulares, simulações integradas e testes em hardware real em bancada. A capacidade de execução correta de programas como cálculo de Fibonacci e entrada/saída simples demonstra a confiabilidade e solidez da solução desenvolvida.

5.4.1 Verificação Detalhada e Códigos dos Testbenches

A verificação funcional apresentada neste capítulo, tanto para os módulos individuais quanto para a unidade de processamento integrada, foi conduzida utilizando testbenches específicos desenvolvidos em Verilog. Cada testbench foi projetado para cobrir uma gama representativa de cenários de operação, incluindo casos de teste típicos, valores limite e, quando aplicável, tratamento de condições especiais ou erros definidos pela especificação RISC-V.

As saídas de console e as formas de onda exibidas nas seções anteriores são extratos representativos dessas simulações, focando nos aspectos mais relevantes para demonstrar a corretude de cada componente. Para uma análise mais aprofundada e para referência completa dos cenários de teste aplicados a cada módulo, os códigos-fonte completos dos testbenches utilizados para a ULA, Unidade de Multiplicação/Divisão, Extensor de Imediato, Banco de Registradores, os diversos Multiplexadores, o Contador de Programa, os Somadores associados ao PC, e a Unidade de Processamento integrada podem ser consultados no Apêndice C. Esta documentação complementar permite uma auditoria detalhada da metodologia de verificação empregada.

A conclusão bem-sucedida destes testes, conforme detalhado ao longo deste capítulo, fornece um alto grau de confiança na corretude funcional da unidade de processamento implementada, preparando o terreno para as próximas etapas de desenvolvimento do processador RISC-V completo.

6 Considerações Finais

Este relatório documenta o desenvolvimento completo de um processador RISC-V de 32 bits, seguindo a configuração RV32IM, desde sua concepção arquitetural até a verificação prática em hardware. O projeto evoluiu a partir da definição de um subconjunto de instruções, do projeto do *datapath* monociclo e suas interconexões, passando pela implementação em linguagem de descrição de hardware (Verilog), simulação funcional, e terminando na validação em uma plataforma FPGA (DE2-115 com Altera Cyclone IV).

Todos os objetivos específicos delineados ao longo dos Pontos de Checagem foram integralmente alcançados. No PC1, foi definido o conjunto de instruções **RV32I** e **RV32M** (simplificada), foram descritos seus formatos e codificações binárias, os modos de endereçamento utilizados e o esquemático completo da arquitetura do processador. No PC2, os principais módulos da unidade de processamento — como a ULA, Banco de Registradores, Extensor de Imediato, Unidade de Multiplicação/Divisão e multiplexadores — foram implementados em Verilog, integrados no módulo `RISCV_processador` e validados por simulação funcional. Por fim, a etapa final consistiu na síntese e teste do processador em FPGA, onde programas de teste foram corretamente executados, comprovando o funcionamento completo da arquitetura proposta.

O desenvolvimento apresentou desafios relevantes, desde a escolha da arquitetura RISC-V — uma alternativa mais moderna e modular em relação ao MIPS tradicional — até a organização da documentação técnica de apoio. A construção do *datapath*, com suas múltiplas interconexões e sinais de controle, exigiu grande atenção aos detalhes. A transição para a implementação em Verilog representou uma curva de aprendizado significativa, principalmente no que diz respeito à modelagem correta dos módulos, à construção de testbenches robustos e à depuração a partir das formas de onda simuladas. A posterior integração com módulos de controle, memórias e periféricos adicionou novas camadas de complexidade ao projeto.

A fase final de testes em hardware constituiu a etapa mais desafiadora e recompensadora do projeto. Através da integração do processador a uma FPGA real, foi possível testar programas completos que envolviam leitura de switches (MMIO), execução de algoritmos (como o cálculo do número de Fibonacci) e exibição dos resultados nos displays de 7 segmentos da placa. Os testes práticos demonstraram o funcionamento correto da ULA, do banco de registradores, da lógica de controle de fluxo e das interfaces de entrada/saída. O sucesso na execução dos programas de teste no ambiente físico confere ao processador um alto grau de confiabilidade.

O trabalho aqui desenvolvido demonstra, portanto, não apenas a viabilidade da

construção de um processador funcional baseado na ISA RISC-V, como também consolida os conhecimentos em projeto de arquiteturas, descrição de hardware e integração em sistemas embarcados, representando uma experiência completa e enriquecedora em engenharia de computação.

Referências

- 1 WATERMAN, A.; ASANOVIC, K. *The RISC-V Instruction Set Manual – Volume I: User-Level ISA*. [S.l.], 2017. Disponível em: <<https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf>>. Citado 8 vezes nas páginas 9, 13, 14, 15, 16, 18, 21 e 23.
- 2 PATTERSON, D. A.; WATERMAN, A. *Guia prático RISC-V: atlas de uma arquitetura aberta*. 1. ed. Berkeley, CA: Strawberry Canyon LLC, 2019. Tradução da obra original. ISBN 978-0-9992491-1-6. Citado 3 vezes nas páginas 9, 15 e 21.
- 3 PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design: The Hardware/Software Interface*. 5. ed. Waltham/MA, EUA: Morgan Kaufmann, 2007. Citado 4 vezes nas páginas 13, 16, 17 e 18.
- 4 DECOM/UFOP, G. iMobilis O RISC-V. 2022. Acessado em: 5 maio 2025. Disponível em: <<https://www2.decom.ufop.br/imobilis/o-risc-v/>>. Citado 2 vezes nas páginas 14 e 15.
- 5 WEBER, R. F. *Fundamentos de arquitetura de computadores*. 3. ed. Porto Alegre: Bookman, 2010. 408 p. ISBN 978-85-7780-662-7. Citado 3 vezes nas páginas 16, 17 e 18.
- 6 FMUSER. *Diferença entre a arquitetura de Von Neumann e Harvard?* 2018. Acesso em: 27 maio 2025. Disponível em: <<https://pt.fmuser.net/content/?12394.html>>. Citado na página 17.
- 7 PALNITKAR, S. *Verilog HDL: A Guide to Digital Design and Synthesis*. 2. ed. Upper Saddle River, NJ: Prentice Hall, 2003. ISBN 978-0130449115. Citado na página 19.
- 8 ZHU, J. *RISC-V Reference Card V1.0*. Acessado em 2025. <<https://github.com/jameszhu/riscv-card>>. Acesso em: 20 abr. 2025. Licença: Creative Commons Atribuição 4.0 Internacional (CC-BY-4.0). Citado na página 23.
- 9 Ultraembedded (pseudônimo). *riscv: RISC-V CPU Core (RV32IM)*. Acessado em 2025. <<https://github.com/ultraembedded/riscv>>. Acesso em: 15 abr. 2025. Citado na página 25.

Apêndices

APÊNDICE A – Códigos Fibonacci

```

1
2 // Programa Fibonacci!!
3
4     mem[0] = 32'b01111111110000000010000110000011; // lw x3, 2044(x0)
5
6     mem[1] = 32'b0000000000000000000000010010011; // addi x1, x0, 0
7
8     mem[2] = 32'b00000000000100000000000100010011; // addi x2, x0, 1
9
10    mem[3] = 32'b000000000000000011000110001100011; // beq x3, x0, 24
11
12    mem[4] = 32'b00000000001000001000001100110011; // add x6, x1, x2
13
14    mem[5] = 32'b00000000000000001000000010010011; // addi x1, x2, 0
15
16    mem[6] = 32'b000000000000000011000000100010011; // addi x2, x6, 0
17
18    mem[7] = 32'b1111111111100011000000110010011; // addi x3, x3, -1
19
20    mem[8] = 32'b01111110001000000010110000100011; // sw x2, 2040(x0)
21
22    mem[9] = 32'b1111111010011111111000001101111; // jal x0, -24
23
24    mem[10] = 32'b01111110001000000010110000100011; // sw x2, 2040(x0)
25
26    mem[11] = 32'b1111110101011111111000001101111; // jal x0, -44
27 endmodule

```

Listing A.1 – Código escrito em binário para alimentar a memória de instrução

APÊNDICE B – Códigos Verilog do processador

B.1 Código RISCV_processor.v

```

1  module RISCV_processor(
2      input wire clk,
3      input wire reset,
4      input wire [31:0] external_input_i,
5
6      output wire [31:0] display_data_o,
7      output wire      display_we_o,
8
9      // SAIDA PARA DEPURACAO
10     output wire [31:0] pc_out_debug,
11     output wire [31:0] next_pc_debug,
12     output wire      is_reading_input_debug_o,
13     input wire      data_is_ready_from_input,
14
15
16     // Adicione estas saídas temporárias de depuração
17     output wire [31:0] debug_instruction,
18     output wire      debug_MemRead,
19     output wire [31:0] debug_alu_result,
20     output wire      debug_select_input_device
21 );
22
23     localparam DISPLAY_ADDR = 32'd2040; // Endereco do registrador de saída
24     localparam INPUT_ADDR = 32'd2044; // Endereco do registrador de entrada
25
26
27     // --- FIOS INTERNOS (WIRES) ---
28     wire [31:0] pc_out, instruction, pc_plus, pc_target_addr, next_pc;
29     wire [6:0] opcode;
30     wire [2:0] funct3;
31     wire [6:0] funct7;
32     wire [4:0] rs1_addr, rs2_addr, rd_addr;
33     wire [31:0] imm_extended;
34
35     // Sinais de Controle
36     wire      RegWrite_from_control, MemWrite_from_control;
37     wire      RegWrite, AluSrc, MemRead, MemWrite, Branch, Jump, UseMD, Inst;
38     wire [2:0] BranchType;
39     wire [1:0] MulDivOp, Mux3_sel;
40     wire [3:0] AluControl;
41     wire      instruction_squash;
42     wire      MemtoReg;
43
44     // Barramentos de Dados e Flags
45     wire [31:0] read_data1, read_data2;
46     wire [31:0] alu_input2, alu_result;
47     wire      alu_zero_flag, alu_negative_flag, alu_overflow_flag;
48     wire [31:0] md_result, exec_result, reg_write_data;

```

```

49     wire      mem_write_enable, display_write_enable, select_input_device;
50     wire [31:0] data_from_mem, final_read_data;
51     wire      branch_taken;
52
53     wire display_write_internal;
54     // Expandir input de 18 para 32 bits
55     wire [31:0] external_input_32bit;
56     assign external_input_32bit = external_input_i;
57
58     assign next_pc_debug = next_pc;
59     assign pc_out_debug = pc_out;
60
61     assign debug_instruction      = instruction;
62     assign debug_MemRead          = MemRead;
63     assign debug_alu_result       = alu_result;
64     assign debug_select_input_device = select_input_device;
65
66     // Ativa o stall se:
67     // 1. eh uma instrucao de leitura (MemRead)
68     // 2. O endereco eh o do dispositivo de entrada (INPUT_ADDR)
69     // 3. E o dispositivo de entrada NAO esta pronto (!data_is_ready...)
70     assign pc_stall = MemRead && (alu_result == INPUT_ADDR) && !data_is_ready_from_input;
71     assign is_reading_input_debug_o = select_input_device;
72
73     // Lógica de detecção de escrita no display
74     assign display_write_internal = (alu_result == DISPLAY_ADDR) && MemWrite && !pc_stall
75         ;
76
77     // Saidas para o display
78     assign display_data_o = read_data2;
79     assign display_we_o   = display_write_internal;
80
81     // --- DECODIFICAÇÃO DE CAMPOS ---
82     assign opcode = instruction[6:0];
83     assign rd_addr = instruction[11:7];
84     assign funct3 = instruction[14:12];
85     assign rs1_addr = instruction[19:15];
86     assign rs2_addr = instruction[24:20];
87     assign funct7 = instruction[31:25];
88
89     // =====
90     // == INSTANCIACAO DE TODOS OS MODULOS ==
91     // =====
92
93     // --- ETAPA 1: BUSCA DE INSTRUÇÃO (PC e Memória de Instrução) ---
94     pc_plus_4_adder pc_plus_4_adder_inst (
95         .current_pc(pc_out),
96         .pc_plus(pc_plus)
97     );
98
99     pc_target_adder pc_target_adder_inst (
100         .in_pc_out(pc_out),
101         .in_imm_extended(imm_extended),
102         .out_pc_target(pc_target_addr)
103     );
104
105     // MUX4 para selecionar o próximo PC
106     Mux4_pc_source mux4_inst (
107         .in_pc_plus(pc_plus),
108         .in_pc_target(pc_target_addr),

```

```

108     .in_alu_result(alu_result),
109     .sel_jump(Jump),
110     .sel_branch_taken(branch_taken),
111     .opcode_i(opcode),
112     .out_pc_in(next_pc)
113 );
114
115 program_counter program_counter_inst (
116     .clock(clk),
117     .reset(reset),
118     .pc_in(next_pc),
119     .pc_out(pc_out),
120     .enable(!pc_stall) // <-- PC s      avan a se N O houver stall
121 );
122
123 instruction_memory instruction_memory_inst (
124     .address(pc_out),
125     .instruction(instruction)
126 );
127
128 // --- ETAPA 2: DECODIFICACAO ---
129 Unidade_de_controle unidade_de_controle_inst (
130     .opcode(opcode),
131     .funct3(funct3),
132     .funct7(funct7),
133     .RegWrite(RegWrite_from_control),
134     .ALUSrc(AluSrc),
135     .ALUControl(AluControl),
136     .MemRead(MemRead),
137     .MemWrite(MemWrite_from_control),
138     .MemtoReg(MemtoReg),
139     .Branch(Branch),
140     .Jump(Jump),
141     .UseMD(UseMD),
142     .MulDivOp(MulDivOp),
143     .BranchType(BranchType),
144     .Inst(Inst)
145 );
146
147 extensor extensor_inst (
148     .instr(instruction),
149     .imm_ext(imm_extended)
150 );
151
152 // --- LOGICA DE ANULACAO (SQUASH) ---
153 assign instruction_squash = branch_taken && !reset;
154 assign RegWrite = RegWrite_from_control && !instruction_squash;
155 assign MemWrite = MemWrite_from_control && !instruction_squash;
156
157 bancoderegistradores bancoderegistradores_inst (
158     .clock(clk),
159     .reset(reset),
160     .reg_write(RegWrite),
161     .read_addr1(rs1_addr),
162     .read_addr2(rs2_addr),
163     .write_addr(rd_addr),
164     .write_data(reg_write_data),
165     .read_data1(read_data1),
166     .read_data2(read_data2)
167 );

```

```

168
169 // --- ETAPA 3: EXECUCAO ---
170 Mux1 mux1_inst (
171     .in_read_data2(read_data2),
172     .in_imm_extended(imm_extended),
173     .sel_alu_src(AluSrc),
174     .out_alu_operand_b(alu_input2)
175 );
176
177 ula ula_inst (
178     .operand_a(read_data1),
179     .operand_b(alu_input2),
180     .alu_control(AluControl),
181     .alu_result(alu_result),
182     .zero_flag(alu_zero_flag),
183     .negative_flag(alu_negative_flag),
184     .overflow_flag(alu_overflow_flag)
185 );
186
187 div_mult div_mult_inst (
188     .operand1(read_data1),
189     .operand2(read_data2),
190     .mul_div_op(MulDivOp),
191     .md_result(md_result)
192 );
193
194 // Unidade de Branch
195 branch branch_inst (
196     .Branch_i(Branch),
197     .BranchType_i(BranchType),
198     .Zero_i(alu_zero_flag),
199     .Negative_i(alu_negative_flag),
200     .BranchTaken_o(branch_taken)
201 );
202
203 // MUX2 para selecionar o resultado da execucao
204 Mux2 mux2_inst (
205     .in_alu_result(alu_result),
206     .in_md_result(md_result),
207     .sel_use_md(UseMD),
208     .out_conta_final(exec_result)
209 );
210
211 // --- ETAPA 4: ACESSO A MEMORIA E I/O ---
212 decoder_mmio_output dec_out_inst (
213     .MemWrite_i(MemWrite),
214     .Address_i(alu_result),
215     .Display_WriteEnable_o(display_write_enable),
216     .MemDados_WriteEnable_o(mem_write_enable)
217 );
218
219 data_memory data_mem_inst (
220     .clk_read(clk),
221     .clk_write(clk),
222     .MemWrite(mem_write_enable),
223     .MemRead(MemRead),
224     .address(alu_result),
225     .write_data(read_data2),
226     .read_data(data_from_mem)
227 );

```



```

228
229     decoder_mmio_input dec_in_inst (
230         .MemRead_i(MemRead),
231         .Address_i(alu_result),
232         .select_input_device_o(select_input_device)
233     );
234
235     // MUX5 para selecionar entre memoria principal e MMIO de entrada
236     assign final_read_data = (select_input_device) ? external_input_32bit : data_from_mem
        ;
237
238     // --- ETAPA 5: WRITE-BACK ---
239     assign Mux3_sel = {Inst, MemtoReg};
240     Mux3 mux3_inst (
241         .in_exec_result(exec_result),
242         .in_mem_data(final_read_data),
243         .in_pc_plus(pc_plus),
244         .sel(Mux3_sel),
245         .out_data(reg_write_data)
246     );
247
248 endmodule
249 endmodule

```

Listing B.1 – Código do processador para unir todos os módulos internos

B.2 Código toplevel.v

```

1 module toplevel (
2     input  wire      CLOCK_50,
3     input  wire [17:0] SW,
4     input  wire [3:0] KEY,
5
6     output wire [6:0]  HEX0, HEX1, HEX2, HEX3,
7     output wire [6:0]  HEX4, HEX5, HEX6, HEX7,
8     output wire [9:0]  LEDR
9 );
10
11     wire [31:0] input_device_data;
12     wire [31:0] current_pc_from_cpu;
13     wire [31:0] debug_next_pc;
14     wire        cpu_is_reading_input;
15
16     wire        reset_key_pressed = ~KEY[0];
17     wire        enter_key_pressed = ~KEY[1];
18
19     wire        reset_signal;
20     wire        enter_signal;
21
22     wire        tick_1hz;
23
24     wire        input_data_ready_signal;
25
26     // --- Sinais que conectam a CPU ao Display ---
27     wire        we_signal_from_cpu;
28     wire [31:0] data_from_cpu_for_output;
29     reg  [31:0] display_value_reg;

```

```

30
31
32
33     clock_divider #(
34         .DIVISOR(10_000_000)//50_000_000
35     ) clk_div_1hz (
36         .clk(CLOCK_50),
37         .reset(reset_signal),
38         .tick(tick_1hz)
39     );
40
41     // Debouncers para os botoes
42     debouncer reset_debouncer (
43         .clk(CLOCK_50),
44         .button_in(reset_key_pressed),
45         .button_out(reset_signal)
46     );
47
48     debouncer enter_debouncer (
49         .clk(CLOCK_50),
50         .button_in(enter_key_pressed),
51         .button_out(enter_signal)
52     );
53
54
55
56     input_device input_unit (
57         .clk(CLOCK_50),
58         .reset(reset_signal),
59         .physical_switches_i({14'b0, SW[17:0]}),
60         .enter_button_i(enter_signal), //mudei aqui pra simula o
61         .cpu_read_en(cpu_is_reading_input), // Informa ao controlador quando a CPU est
            lendo
62         .data_for_cpu_o(input_device_data),
63         .data_ready(input_data_ready_signal)
64     );
65
66     RISCv_processor cpu (
67         .clk(tick_1hz),
68         .reset(reset_signal),
69         .external_input_i(input_device_data),
70         .display_data_o(data_from_cpu_for_output),
71         .display_we_o(we_signal_from_cpu),
72         .pc_out_debug(current_pc_from_cpu),
73         .is_reading_input_debug_o(cpu_is_reading_input),
74         .data_is_ready_from_input(input_data_ready_signal),
75         .next_pc_debug(debug_next_pc),
76         .debug_instruction(w_debug_instruction),
77         .debug_MemRead(w_debug_MemRead),
78         .debug_alu_result(w_debug_alu_result),
79         .debug_select_input_device(w_debug_select_input_device)
80     );
81
82     // --- LOGICA DO DISPLAY COM REGISTRADOR DE SAIDA DEDICADO ---
83     always @(posedge CLOCK_50 or posedge reset_signal) begin
84         if (reset_signal) begin
85             display_value_reg <= 32'd0; // Valor inicial para teste
86         end else if (we_signal_from_cpu) begin
87             display_value_reg <= data_from_cpu_for_output;
88         // end else if (enter_signal) begin // DEBUG: Use SW[17] para testar manualmente

```

```

89         //      display_value_reg <= {15'b0, SW[16:0]}; // Mostra valor dos switches
90     end
91 end
92
93 wire [31:0] bcd_result;
94 binary_to_bcd bcd_converter (
95     .binary_in(display_value_reg),
96     .bcd_out(bcd_result)
97 );
98
99 // Funcao de conversao BCD para 7 segmentos
100 function [6:0] bcd_to_7seg;
101     input [3:0] digit;
102     case (digit)
103         4'h0: bcd_to_7seg = 7'b1000000; // 0
104         4'h1: bcd_to_7seg = 7'b1111001; // 1
105         4'h2: bcd_to_7seg = 7'b0100100; // 2
106         4'h3: bcd_to_7seg = 7'b0110000; // 3
107         4'h4: bcd_to_7seg = 7'b0011001; // 4
108         4'h5: bcd_to_7seg = 7'b0010010; // 5
109         4'h6: bcd_to_7seg = 7'b0000010; // 6
110         4'h7: bcd_to_7seg = 7'b1111000; // 7
111         4'h8: bcd_to_7seg = 7'b0000000; // 8
112         4'h9: bcd_to_7seg = 7'b0010000; // 9
113         default: bcd_to_7seg = 7'b1111111; // Apagado
114     endcase
115 endfunction
116
117
118 assign HEX0 = bcd_to_7seg(bcd_result[3:0]); // Unidades
119 assign HEX1 = bcd_to_7seg(bcd_result[7:4]); // Dezenas
120 assign HEX2 = bcd_to_7seg(bcd_result[11:8]); // Centenas
121 assign HEX3 = bcd_to_7seg(bcd_result[15:12]); // Milhares
122 assign HEX4 = bcd_to_7seg(bcd_result[19:16]);
123 assign HEX5 = bcd_to_7seg(bcd_result[23:20]);
124 assign HEX6 = bcd_to_7seg(bcd_result[27:24]);
125 assign HEX7 = bcd_to_7seg(bcd_result[31:28]); // D gito mais significativo
126
127 // Clock de teste
128 reg [24:0] clock_divider_reg;
129 always @(posedge CLOCK_50 or posedge reset_signal) begin
130     if (reset_signal) begin
131         clock_divider_reg <= 0;
132     end else begin
133         clock_divider_reg <= clock_divider_reg + 1;
134     end
135 end
136
137 // Debug LEDs - MAIS DETALHADOS
138 assign LEDR[0] = clock_divider_reg[24]; // pisca 1 vez por segundo
139 assign LEDR[1] = we_signal_from_cpu; // CRITICO: deve acender quando CPU escreve
140 assign LEDR[2] = |display_value_reg; // indica se ha valor no display
141 assign LEDR[3] = enter_signal; // botao enter pressionado
142 assign LEDR[4] = reset_signal; // reset ativo
143 assign LEDR[5] = (current_pc_from_cpu != 32'd0); // PC nao est no inicio
144 assign LEDR[6] = |bcd_result; // conversao BCD funcionando
145 assign LEDR[7] = tick_1hz; // clock de 1Hz
146 assign LEDR[8] = |data_from_cpu_for_output; // dados vindos da CPU
147 assign LEDR[9] = |input_device_data; // dados do dispositivo de entrada
148

```

149 `endmodule`

Listing B.2 – Código TopLevel para unir todos os módulos

B.3 Códigos dos módulos separadamente

```

1  `timescale 1ns / 1ps
2
3  module program_counter (
4      input  wire      clock,          // Sinal de clock global
5      input  wire      reset,          // Sinal de reset (ativo alto)
6      input  wire [31:0] pc_in,        // Proximo valor do PC (vindo do MUX4
7      output reg [31:0] pc_out,        // Valor atual do PC (para Memoria
        de Instru  o e l g i c a PC+4/Target)
8      input  wire      enable
9  );
10
11     // Logica sincrona para o registrador PC
12     always @(posedge clock or posedge reset) begin
13         if (reset) begin
14             pc_out <= 32'h00000000; // Endere o inicial padrao no reset
15         end else if (enable) begin
16             pc_out <= pc_in;        // Carrega o proximo valor do PC
17         end
18     end
19
20 endmodule

```

Listing B.3 – Código Verilog do Contador de Programa (program_counter.v)

```

1
2  `timescale 1ns / 1ps
3
4  module pc\_plus\_4\_adder (
5      input  wire [31:0] current_pc,    // Saida do modulo program_counter
6      output wire [31:0] pc\_plus\_4    // Resultado (current\_pc + 4)
7  );
8
9      // Logica combinacional para somar 4
10     assign pc\_plu\s_4 = current\_pc + 32'd4;
11
12 endmodule

```

Listing B.4 – Código Verilog do Somador PC+4 (pc_plus_4_adder.v)

1

```

2 `timescale 1ns / 1ps
3
4 // Calcula PC_out + ImmExt
5 module pc_target_adder (
6     input wire [31:0] in_pc_out,          // Valor atual do PC (saida do
        program_counter)
7     input wire [31:0] in_imm_extended,    // Imediato estendido (offset
        para B-type ou J-type)
8     output wire [31:0] out_pc_target      // Endereco de destino
        calculado (PC_out + ImmExt)
9 );
10
11 // Logica combinacional para a soma
12 assign out_pc_target = in_pc_out + in_imm_extended;
13
14 endmodule

```

Listing B.5 – Código Verilog do Somador PC Target (pc_target_adder.v)

```

1
2 `timescale 1ns / 1ps
3
4 module mux_pc_source (
5     input wire [31:0] in_pc_plus_4,        // Entrada 0: PC + 4
6     input wire [31:0] in_pc_target_branch_jal, // Entrada 1: Endereco
        de desvio/JAL (PC + Imm_B/J)
7     input wire [31:0] in_alu_result_jalr,   // Entrada 2: Endereco de
        JALR (rs1 + Imm_I)
8     input wire [1:0] sel_pc_src,           // Sinal de selecao [1:0]
9                                           // 00: PC+4
10                                           // 01: PC_Target_Branch/
        JAL
11                                           // 10: Alu_Result_JALR
12                                           // 11: (Nao usado /
        default para PC+4)
13     output reg [31:0] out_pc_in           // Saida selecionada para
        PC_In
14 );
15
16 always @(*) begin
17     case (sel_pc_src)
18         2'b00: out_pc_in = in_pc_plus_4;
19         2'b01: out_pc_in = in_pc_target_branch_jal;
20         2'b10: out_pc_in = in_alu_result_jalr;
21         default: out_pc_in = in_pc_plus_4; // Comportamento seguro
        para selecao nao usada
22     endcase
23 end

```

```
24
25 endmodule
```

Listing B.6 – Código Verilog do MUX4 (PcSource) (mux_pc_source.v)

```
1 module ula (
2     input wire [31:0] operand_a,      // entrada A (ex: ReadData1)
3     input wire [31:0] operand_b,      // entrada B (ex: saida do MUX1)
4     input wire [3:0] alu_control,      // sinal de controle da Unidade de Controle
5     output reg [31:0] alu_result,      // resultado da operacao
6     output reg zero_flag               // Flag Zero (1 se resultado for 0)
7     // Adicionar outras flags se necessario (ex: negative_flag, overflow_flag)
8 );
9
10 // ula_op:
11 // 0000 -> AND
12 // 0001 -> OR
13 // 0010 -> ADD
14 // 0011 -> SUB
15 // 0100 -> XOR
16 // 0101 -> SLT
17 // 0110 -> SLL
18 // 0111 -> SRL
19 // 1000 -> SRA
20
21 always @(*) begin
22
23     alu_result = 32'b0;
24     zero_flag  = 1'b0;
25
26     case (alu_control)
27         4'b0000: alu_result = operand_a & operand_b;           // AND
28         4'b0001: alu_result = operand_a | operand_b;           // OR
29         4'b0010: alu_result = operand_a + operand_b;           // ADD
30         4'b0011: alu_result = operand_a - operand_b;           // SUB
31         4'b0100: alu_result = operand_a ^ operand_b;           // XOR
32         4'b0101: alu_result = ($signed(operand_a) < $signed(operand_b)) ? 32'd1 : 32'd0; // SLT
33         4'b0110: alu_result = operand_a << operand_b[4:0];      // SLL
34         4'b0111: alu_result = operand_a >> operand_b[4:0];      // SRL
35         4'b1000: alu_result = $signed(operand_a) >>> operand_b[4:0]; // SRA
36
37         default: alu_result = 32'b0; // comportamento padrao ou erro
38     endcase
39
40     if (alu_result == 32'b0) begin
41         zero_flag = 1'b1;
42     end else begin
43         zero_flag = 1'b0;
44     end
45 end
46
47 endmodule
```

Listing B.7 – Código Verilog da ULA (ula.v)

```
1 module div_mult (
2     input wire signed [31:0] operand1,    // rs1
3     input wire signed [31:0] operand2,    // rs2
```

```

4     input wire [1:0]      mul_div_op,    // Sinal de controle da
        Unidade de Controle
5     output reg signed [31:0] md_result
6 );
7
8 // mul_div_op:
9 // 00 -> MUL
10 // 01 -> DIV
11 // 10 -> REM
12 // 11 -> (reservado / nao utilizado)
13
14
15 //sinais internos para facilitar a leitura e o calculo de 64 bits
    para mul
16 reg signed [63:0] mul_temp_result;
17
18 always @(*) begin
19     md_result = 32'b0;
20     mul_temp_result = 64'b0;
21     //evitar latches
22     case (mul_div_op)
23         // --- MUL (Multiplicacao Signed) ---
24         // RV32I 'mul' rd, rs1, rs2: rd = (rs1 * rs2)[31:0]
25         2'b00: begin
26             mul_temp_result = operand1 * operand2; // multiplicacao
                signed de 32x32 -> 64 bits
27             md_result      = mul_temp_result[31:0]; // pega os 32
                bits inferiores
28         end
29
30         // --- DIV (Divisao Signed) ---
31         // RV32I 'div' rd, rs1, rs2
32         2'b01: begin
33             if (operand2 == 32'd0) begin // divisao por zero
34                 md_result = 32'hFFFFFFFF; // Todos os bits '1' (-1
                    signed)
35             end else if (operand1 == 32'h80000000 && operand2 == 32'
                hFFFFFFFF) begin // Overflow: INT_MIN / -1
36                 md_result = 32'h80000000; // resultado eh INT_MIN (o
                    dividendo)
37             end else begin
38                 md_result = operand1 / operand2;
39             end
40         end
41
42         // --- REM (Resto da Divisao Signed) ---
43         // RV32I 'rem' rd, rs1, rs2

```

```

44         2'b10: begin
45             if (operand2 == 32'd0) begin // resto da divisao por
                zero
46                 md_result = operand1;      //resultado eh o dividendo
47             end else if (operand1 == 32'h80000000 && operand2 == 32'
                hFFFFFFF) begin // Overflow na divisao: INT_MIN / -1
48                 md_result = 32'd0;          //resto eh 0
49             end else begin
50                 md_result = operand1 % operand2;
51             end
52         end
53
54         default: md_result = 32'b0; //comportamento padrao ou erro
55     endcase
56 end
57
58 endmodule

```

Listing B.8 – Código Verilog da Unidade de Multiplicação/Divisão (div_mult.v)

```

1 module extensor (
2     input wire [31:0] instruction, // A instrucao completa de 32 bits
3     input wire [2:0] imm_type,      // Sinal da Unidade de Controle
        para tipo de imediato
4     output reg [31:0] imm_extended // Imediato estendido para 32 bits
5 );
6
7 // Definicao dos tipos de imediato
8 localparam IMM_TYPE_I = 3'b000;
9 localparam IMM_TYPE_S = 3'b001;
10 localparam IMM_TYPE_B = 3'b010;
11 localparam IMM_TYPE_J = 3'b011;
12 // localparam IMM_TYPE_U = 3'b100 para uso futuro
13
14 // Logica combinacional
15 always @(*) begin
16     case (imm_type)
17         IMM_TYPE_I: begin
18             // Imediato de 12 bits: inst[31:20]
19             // Extensao de sinal do bit inst[31]
20             imm_extended = {{20{instruction[31]}}, instruction
                [31:20]};
21         end
22
23         IMM_TYPE_S: begin
24             // Imediato de 12 bits: inst[31:25] (imm[11:5]) e inst
                [11:7] (imm[4:0])
25             // Extensao de sinal do bit inst[31]

```



```

26         imm_extended = {{20{instruction[31]}}}, instruction
           [31:25], instruction[11:7]};
27     end
28
29     IMM_TYPE_B: begin
30         // Imediato de 13 bits (bit 0 implícito como 0):
31         // imm[12]    = inst[31]
32         // imm[10:5]  = inst[30:25]
33         // imm[4:1]   = inst[11:8]
34         // imm[11]    = inst[7]
35         // Extensão de sinal do bit inst[31] (imm[12])
36         imm_extended = {{19{instruction[31]}}}, instruction[31],
           instruction[7], instruction[30:25], instruction
           [11:8], 1'b0};
37     end
38
39     IMM_TYPE_J: begin
40         // Imediato de 21 bits (bit 0 implícito como 0):
41         // imm[20]    = inst[31]
42         // imm[10:1]  = inst[30:21]
43         // imm[11]    = inst[20]
44         // imm[19:12] = inst[19:12]
45         // Extensão de sinal do bit inst[31] (imm[20])
46         imm_extended = {{11{instruction[31]}}}, instruction[31],
           instruction[19:12], instruction[20], instruction
           [30:21], 1'b0};
47     end
48
49
50     default: begin
51         imm_extended = 32'bx; //valor indefinido para tipo
           desconhecido
52     end
53 endcase
54 end
55
56 endmodule

```

Listing B.9 – Código Verilog do Extensor de Imediato (extensor.v)

```

1 module bancoderegistradores (
2     input wire    clock,
3     input wire    reset,          // Sinal de reset (ativo alto)
4     input wire    reg_write,      // Habilita escrita no
           registrador (da Unidade de Controle)
5     input wire [4:0] read_addr1,  // Endereco do registrador rs1
6     input wire [4:0] read_addr2,  // Endereco do registrador rs2
7     input wire [4:0] write_addr,  // Endereco do registrador rd (

```

```

destino)
8   input wire [31:0] write_data,      // Dado a ser escrito no
    registrador (do MUX3)
9
10  output wire [31:0] read_data1,     // Dado lido de rs1
11  output wire [31:0] read_data2     // Dado lido de rs2
12 );
13
14  //(flip-flops)
15  reg [31:0] rf_array [0:31];
16
17  // logica de Leitura Combinacional. O registrador x0 (endereço 0)
    sempre retorna 0.
18  assign read_data1 = (read_addr1 == 5'b0) ? 32'b0 : rf_array[
    read_addr1];
19  assign read_data2 = (read_addr2 == 5'b0) ? 32'b0 : rf_array[
    read_addr2];
20
21  //logica de escrita sincrona
22  // A escrita ocorre apenas na borda de subida do clock E se
    reg_write estiver ativo e se o endereço de escrita não for x0.
23  integer i;
24  always @(posedge clock or posedge reset) begin
25      if (reg_write && (write_addr != 5'b0)) begin
26          // Condicao de escrita:
27          // 1. reg_write deve ser '1' (habilitado pela Unidade de
                Controle).
28          // 2. write_addr não deve ser 5'b0, não tentar escrever no
                registrador x0.
29          rf_array[write_addr] <= write_data; // Atribuicao n o
                bloqueante, valor eh atualizado na borda do clock.
30      end
31      // Se as condicoes acima não forem atendidas, os valores nos
        rf_array permanecem inalterados
32  end
33
34 endmodule

```

Listing B.10 – Código Verilog do Banco de Registradores (bancoderegistradores.v)

```

1  // MUX para selecionar o segundo operando da ULA (operand_b)
2  module Mux1 (
3      input wire [31:0] in_read_data2, // Vindo do Banco de Registradores
        (rs2)
4      input wire [31:0] in_imm_extended, // Vindo do Extensor de Imediato
5      input wire        sel_alu_src,    // Sinal de controle AluSrc (0 =
        ReadData2, 1 = ImmExt)
6      output wire [31:0] out_alu_operand_b

```

```

7 );
8
9 // logica combinacional para selecao
10 assign out_alu_operand_b = sel_alu_src ? in_imm_extended :
    in_read_data2;
11 // se sel_alu_src = 0, seleciona in_read_data2
12 // se sel_alu_src = 1, seleciona in_imm_extended
13
14 endmodule

```

Listing B.11 – Código Verilog do MUX para o segundo operando da ULA (Mux1.v)

```

1 // MUX para selecionar entre o resultado da ULA e da unidade DIV/MULT
2 module Mux2 (
3     input wire [31:0] in_alu_result,    // Vindo da ULA
4     input wire [31:0] in_md_result,     // Vindo da Unidade DIV/MULT
5     input wire        sel_use_md,       // Sinal de controle UseMD (0 =
        AluResult, 1 = MDResult)
6     output wire [31:0] out_conta_final
7 );
8
9 // Logica combinacional para selecao
10 assign out_conta_final = sel_use_md ? in_md_result : in_alu_result;
11 // Se sel_use_md = 0, seleciona in_alu_result
12 // Se sel_use_md = 1, seleciona in_md_result
13
14 endmodule

```

Listing B.12 – Código Verilog do MUX para a fonte de escrita (ULA/MD) (Mux2.v)

```

1
2 module input_device(
3     input wire        clk,              // Clock recebido (50MHz)
4     input wire        reset,
5     // Sinais Físicos
6     input wire [31:0] physical_switches_i,
7     input wire        enter_button_i,
8     // Interface com a CPU (MMIO)
9     input wire        cpu_read_en,      // Sinal indicando que a CPU está lendo deste
        dispositivo
10    output wire [31:0] data_for_cpu_o,   // Dado + Status para a CPU
11    output reg        data_ready
12 );
13
14 reg [31:0] captured_data; // Registra o valor dos switches
15
16 // Lógica para capturar os dados e levantar a bandeira
17 always @(posedge clk or posedge reset) begin
18     if (reset) begin
19         data_ready <= 1'b0;
20         captured_data <= 32'b0;
21     end else if (enter_button_i) begin // No pulso do Enter...
22         captured_data <= physical_switches_i; // ...captura o valor
23         data_ready <= 1'b1;                  // ...e levanta a bandeira.

```

```

24     end else if (cpu_read_en) begin // Se a CPU est lendo...
25         data_ready <= 1'b0;        // ...abaixa a bandeira. NAO COMENTAR
26     end
27 end
28
29
30 assign data_for_cpu_o = captured_data;
31
32
33 endmodule

```

Listing B.13 – Código Verilog do Dispositivo de Entrada com botão "Enter".

```

1 // Debouncers para os botões
2 debouncer reset_debouncer (
3     .clk(CLOCK_50),
4     .button_in(reset_key_pressed),
5     .button_out(reset_signal)
6 );
7
8 debouncer enter_debouncer (
9     .clk(CLOCK_50),
10    .button_in(enter_key_pressed),
11    .button_out(enter_signal)
12 );

```

Listing B.14 – Código Verilog do Debouncer de Nível para o sinal de Reset.

```

1 clock_divider #(
2     .DIVISOR(50_000_000)
3 ) clk_div_1hz (
4     .clk(CLOCK_50),
5     .reset(reset_signal),
6     .tick(tick_1hz)
7 );

```

Listing B.15 – Código Verilog do Divisor de Clock para gerar um sinal de habilitação de 1 Hz.

```

1 module binary_to_bcd (
2     input wire [31:0] binary_in,
3     output reg [31:0] bcd_out
4 );
5
6 // Variáveis para os 8 dígitos BCD (4 bits cada)
7 reg [3:0] digit_7, digit_6, digit_5, digit_4;
8 reg [3:0] digit_3, digit_2, digit_1, digit_0;
9 reg [31:0] binary_temp;
10
11 integer i;
12
13 // Algoritmo Double Dabble para conversão binário para BCD
14 always @(*) begin
15     // Inicializa o
16     digit_7 = 4'b0000;
17     digit_6 = 4'b0000;
18     digit_5 = 4'b0000;
19     digit_4 = 4'b0000;
20     digit_3 = 4'b0000;

```

```

21     digit_2 = 4'b0000;
22     digit_1 = 4'b0000;
23     digit_0 = 4'b0000;
24     binary_temp = binary_in;
25
26     // Processa cada bit do número binário
27     for (i = 0; i < 32; i = i + 1) begin
28         // Adiciona 3 se o dígito for >= 5 (antes do shift)
29         if (digit_7 >= 5) digit_7 = digit_7 + 3;
30         if (digit_6 >= 5) digit_6 = digit_6 + 3;
31         if (digit_5 >= 5) digit_5 = digit_5 + 3;
32         if (digit_4 >= 5) digit_4 = digit_4 + 3;
33         if (digit_3 >= 5) digit_3 = digit_3 + 3;
34         if (digit_2 >= 5) digit_2 = digit_2 + 3;
35         if (digit_1 >= 5) digit_1 = digit_1 + 3;
36         if (digit_0 >= 5) digit_0 = digit_0 + 3;
37
38         // Shift left: move o MSB do número binário para o LSB do primeiro dígito
39         digit_7 = {digit_7[2:0], digit_6[3]};
40         digit_6 = {digit_6[2:0], digit_5[3]};
41         digit_5 = {digit_5[2:0], digit_4[3]};
42         digit_4 = {digit_4[2:0], digit_3[3]};
43         digit_3 = {digit_3[2:0], digit_2[3]};
44         digit_2 = {digit_2[2:0], digit_1[3]};
45         digit_1 = {digit_1[2:0], digit_0[3]};
46         digit_0 = {digit_0[2:0], binary_temp[31]};
47
48         // Shift do número binário
49         binary_temp = binary_temp << 1;
50     end
51
52     // Monta a saída final
53     bcd_out = {digit_7, digit_6, digit_5, digit_4,
54               digit_3, digit_2, digit_1, digit_0};
55 end
56
57 endmodule

```

Listing B.16 – Código Verilog do Conversor Binário para BCD.

```

1 module data_memory (
2     input wire      clk_read,
3     input wire      clk_write,
4     input wire      MemWrite, // Sinal da Unidade de Controle para habilitar a
        escrita
5     input wire      MemRead,  // Sinal da Unidade de Controle para habilitar a
        leitura
6     input wire [31:0] address, // Endereço vindo da ULA
7     input wire [31:0] write_data, // Dado a ser escrito, vindo do ReadData2
8     output reg  [31:0] read_data // Dado lido que vai para o MUX final
9 );
10
11     reg [31:0] address_latch;
12
13     // Memória de Dados com 256 palavras de 32 bits (1KB). -> mudar pra 2^16 (maior)
14     reg [31:0] mem [0:255];
15
16
17     // --- LÓGICA DE ESCRITAS SÍNCRONA ---
18     // A escrita acontece apenas na borda de subida do clock se MemWrite estiver ativo.

```

```

19  always @(posedge clk_read) begin
20      address_latch <= address;
21  end
22
23      always@(posedge clk_write)begin
24          if (MemWrite) begin
25              mem[address[9:2]] <= write_data; // Usando os bits do endereço que
                correspondem ao tamanho da memória
26          end
27      end
28
29  // Leitura Sincrona com 1 ciclo de latência
30  always @(posedge clk_read) begin
31      if (MemRead) begin
32          read_data <= mem[address[9:2]];
33      end
34  end
35
36  endmodule

```

Listing B.17 – Código Verilog da memória de Dados.

```

1  module instruction_memory (
2      input  wire [31:0] address,
3      output wire [31:0] instruction
4  );
5      // Memória para 256 instruções de 32 bits
6      reg [31:0] mem [0:255];
7
8      // Carrega o programa do arquivo 'program.hex' no início da simulação
9      //initial begin
10         // $readmemh("tb/jumpcerto.b", mem);
11     //end
12
13
14  initial begin
15      // load e store!
16      // mem[0] = 32'b0000001111011000000000001010010011; // addi x5, x0, 123
17
18      // mem[1] = 32'b0000000000101001000100000000100011; // sw x5, 0(x4)
19
20      // mem[2] = 32'b00000000000000001000100011000000011; // lw x6, 0(x4)
21
22      // mem[3] = 32'b01111110011000100010110000100011; // sw x6, 2040(x4)
23
24      // mem[4] = 32'b000000000000000000000000001100011; // beq x0, x0, 0
25
26      // // load INPUT!
27      // mem[0] = 32'b0111111111000000000100010100000011; // lw x5, 2044(x0)
28
29      // mem[1] = 32'b011111100101000000010110000100011; // sw x5, 2040(x0)
30      // mem[2] = 32'b000000000000000000000000001100011; // beq x0, x0, 0
31
32      // // addi simples com input!
33      // mem[0] = 32'b0111111111000000000100010100000011; // lw x5, 2044(x0)
34
35      // mem[1] = 32'b000000000010100101000001100010011; // addi x6, x5, 5
36
37      // mem[2] = 32'b011111100110000000010110000100011; // sw x6, 2040(x0)
38

```



```

99     mem[5] = 32'b000000000000000010000000010010011; // addi x1, x2, 0
100
101     mem[6] = 32'b0000000000000000110000000100010011; // addi x2, x6, 0
102
103     mem[7] = 32'b111111111111100011000000110010011; // addi x3, x3, -1
104
105     mem[8] = 32'b011111110001000000010110000100011; // sw x2, 2040(x0)
106
107     mem[9] = 32'b11111110100111111110000011011111; // jal x0, -24
108
109     mem[10] = 32'b01111110001000000010110000100011; // sw x2, 2040(x0)
110
111     mem[11] = 32'b11111110101011111111000001101111; // jal x0, -44
112
113
114
115     end
116
117     // A leitura combinacional. tem que mudar aqui pra posedge clk -> n o usar
118     assign
119     assign instruction = mem[address[9:2]];
120
121 endmodule

```

Listing B.18 – Código Verilog da memória de Instruções e seus testes

```

1 module branch (
2     input wire      Branch_i,          // 1 se a instrução é um branch
3     input wire [2:0] BranchType_i,    // Tipo de branch (beq, bne, blt, bge)
4
5     // Entradas das Flags da ULA
6     input wire      Zero_i,           // 1 se o resultado da ULA foi zero
7     input wire      Negative_i,       // 1 se o resultado da ULA foi negativo
8
9     // Saída para o MUX4
10    output wire      BranchTaken_o
11 );
12
13
14    localparam BEQ = 3'b000; // Branch on Equal
15    localparam BNE = 3'b001; // Branch on Not Equal
16    localparam BLT = 3'b100; // Branch on Less Than
17    localparam BGE = 3'b101; // Branch on Greater Than or Equal
18
19    reg condition_met;
20
21    // Lógica combinacional para verificar a condição do branch
22    always @(*) begin
23        case (BranchType_i)
24            BEQ: condition_met = Zero_i;           // Desvia se (rs1 - rs2) == 0
25            BNE: condition_met = ~Zero_i;          // Desvia se (rs1 - rs2) != 0
26            BLT: condition_met = Negative_i;        // Desvia se (rs1 - rs2) < 0
27            BGE: condition_met = ~Negative_i;       // Desvia se (rs1 - rs2) >= 0
28            default: condition_met = 1'b0;
29        endcase
30    end
31
32    assign BranchTaken_o = Branch_i && condition_met;
33

```



```
34 endmodule
```

Listing B.19 – Código Verilog do módulo branch.

```
1  
2 endmodule
```

Listing B.20 – Código Verilog do decoder mmio de entrada.

```
1 module decoder_mmio_input (
2     input wire      MemRead_i,    // Sinal original da Unidade de Controle
3     input wire [31:0] Address_i,  // Endere o vindo da ULA
4
5     //vai para o pino de sele o do MUX 5
6     output wire     select_input_device_o
7 );
8
9     // Endere o do dispositivo de entrada
10    localparam INPUT_ADDR = 32'd 2044; // Limitado -2048-2047 por causa do tamanho do
        registrador de 32 bits
11
12    // O MUX 5 deve selecionar o dispositivo de entrada se MemRead estiver ativo E o
        endere o for o do dispositivo.
13    assign select_input_device_o = (Address_i == INPUT_ADDR) && MemRead_i;
14
15 endmodule
16 endmodule
```

Listing B.21 – Código Verilog do decoder mmio de entrada.

```
1 module decoder_mmio_output (
2     input wire      MemWrite_i,    // Sinal original da Unidade de Controle
3     input wire [31:0] Address_i,  // Endere o vindo da ULA
4
5     output wire     Display_WriteEnable_o, // Habilita a escrita no display
6     output wire     MemDados_WriteEnable_o // Habilita a escrita na mem ria de dados
7 );
8
9     // Endere o do seu display, conforme o diagrama
10    localparam DISPLAY_ADDR = 32'd2040;
11
12    // A escrita no display habilitada se MemWrite estiver ativo E o endere o for o
        do display.
13    assign Display_WriteEnable_o = (Address_i == DISPLAY_ADDR) && MemWrite_i;
14
15    // A escrita na mem ria de dados habilitada se MemWrite estiver ativo E o
        endere o N O for o do display.
16    assign MemDados_WriteEnable_o = (Address_i != DISPLAY_ADDR) && MemWrite_i;
17
18 endmodule
19 endmodule
```

Listing B.22 – Código Verilog do decoder mmio de entrada.

```
1 // --- SAIDA ---
2 module display_device (
3     input wire clk,
4     input wire reset,
5     input wire display_write_enable, // Vem do decoder_mmio_output
6     input wire [31:0] data_in        // Vem de ReadData2
```

```
7 );
8
9     always @(posedge clk) begin
10         if (display_write_enable) begin
11             $display("DISPLAY @ t=%0t ns: Recebeu valor para exibir: %d (%h)", $time,
12                     $signed(data_in), data_in);
13         end
14     end
15 endmodule
16 endmodule
```

Listing B.23 – Código Verilog do *display device*.

APÊNDICE C – Códigos Verilog dos Testbenches

C.1 Testbench da ULA (ula_riscv_comprehensive_tb.v)

```

1  `timescale 1ns / 1ps
2
3  module ula_tb;
4
5      reg [31:0] operand_a, operand_b;
6      reg [3:0] alu_control;
7      wire [31:0] alu_result;
8      wire zero_flag;
9
10     ula DUT (
11         .operand_a(operand_a),
12         .operand_b(operand_b),
13         .alu_control(alu_control),
14         .alu_result(alu_result),
15         .zero_flag(zero_flag)
16     );
17
18     initial begin
19         $display("Iniciando Testes da ULA...");
20         $monitor("Time: %0t | A: %h | B: %h | Ctrl: %b | Result: %h | Zero: %b",
21             $time, operand_a, operand_b, alu_control, alu_result, zero_flag);
22
23         // Operacao ADD
24         alu_control = 4'b0000; // ADD
25         operand_a = 5; operand_b = 10; #10;
26         operand_a = 0; operand_b = 0; #10;
27         operand_a = 32'hFFFFFFFB; operand_b = 10; #10;
28         operand_a = 32'hFFFFFFFB; operand_b = 32'hFFFFFFF6; #10;
29         operand_a = 32'h7FFFFFFF; operand_b = 1; #10;
30         operand_a = 32'h80000000; operand_b = 32'hFFFFFFF; #10;
31
32         // Operacao SUB
33         alu_control = 4'b0001; // SUB
34         operand_a = 10; operand_b = 5; #10;
35         operand_a = 5; operand_b = 10; #10;
36         operand_a = 7; operand_b = 7; #10;
37         operand_a = 32'hFFFFFFFB; operand_b = 10; #10;
38         operand_a = 32'h80000000; operand_b = 1; #10;
39
40         // Operacao AND
41         alu_control = 4'b0010; // AND
42         operand_a = 32'hFFFF0000; operand_b = 32'h00FFFF00; #10;
43         operand_a = 32'hAAAAAAAA; operand_b = 32'h55555555; #10;
44
45         // Operacao OR
46         alu_control = 4'b0011; // OR
47         operand_a = 32'hF0F0F0F0; operand_b = 32'h0F0F0F0F; #10;

```

```

48     operand_a = 32'h00000000; operand_b = 32'h00000000; #10;
49
50     // Operacao XOR
51     alu_control = 4'b0100; // XOR
52     operand_a = 32'hA5A5A5A5; operand_b = 32'hA5A5A5A5; #10;
53     operand_a = 32'hA5A5A5A5; operand_b = 32'hA5A5A5A5; #10;
54
55     // Operacao SLT (signed)
56     alu_control = 4'b0101; // SLT
57     operand_a = 5; operand_b = 10; #10;
58     operand_a = 10; operand_b = 5; #10;
59     operand_a = 5; operand_b = 5; #10;
60     operand_a = 32'hFFFFFFF6; operand_b = 32'hFFFFFFFB; #10;
61     operand_a = 32'hFFFFFFFB; operand_b = 32'hFFFFFFF6; #10;
62     operand_a = 32'hFFFFFFFB; operand_b = 5; #10;
63     operand_a = 5; operand_b = 32'hFFFFFFFB; #10;
64
65     // Operacao SLL
66     alu_control = 4'b0110; // SLL
67     operand_a = 32'h0000000F; operand_b = 32'd2; #10;
68     operand_a = 32'h80000000; operand_b = 32'd1; #10;
69     operand_a = 32'hFFFFFFF; operand_b = 32'd0; #10;
70
71     // Operacao SRL
72     alu_control = 4'b0111; // SRL
73     operand_a = 32'h000000F0; operand_b = 32'd4; #10;
74     operand_a = 32'hF0000000; operand_b = 32'd4; #10;
75     operand_a = 32'h00000001; operand_b = 32'd1; #10;
76
77     // Operacao SRA
78     alu_control = 4'b1000; // SRA
79     operand_a = 32'h000000F0; operand_b = 32'd4; #10;
80     operand_a = 32'hF0000000; operand_b = 32'd4; #10;
81     operand_a = 32'hFFFFFFFE; operand_b = 32'd1; #10;
82
83     // Operacao invalida/default
84     alu_control = 4'b1111;
85     operand_a = 32'h12345678; operand_b = 32'h9ABCDEF0; #10;
86
87     $display("Testes finalizados.");
88     $stop;
89 end
90 endmodule

```

Listing C.1 – Testbench completo para a ULA.

C.2 Testbench da Unidade de Multiplicação/Divisão (div_mult_tb.v)

```

1  `timescale 1ns / 1ps
2
3  module div_mult_tb; // Nome do modulo de testbench
4
5      // Entradas para o DUT (Device Under Test)
6      reg signed [31:0] tb_operand1;
7      reg signed [31:0] tb_operand2;
8      reg [1:0]         tb_mul_div_op;
9

```

```

10 // Saida do DUT
11 wire signed [31:0] tb_md_result;
12
13 // Instancia o modulo 'div_mult' (o seu DUT)
14 // Certifique-se de que o nome 'div_mult' corresponde ao nome do seu modulo.
15 div_mult uut (
16     .operand1(tb_operand1),
17     .operand2(tb_operand2),
18     .mul_div_op(tb_mul_div_op),
19     .md_result(tb_md_result)
20 );
21
22 // Constantes Uteis para Testes
23 localparam MAX_INT_S = 32'h7FFFFFFF;
24 localparam MIN_INT_S = 32'h80000000;
25 localparam NEG_ONE_S = 32'hFFFFFFF;
26
27 // Mapeamento mul_div_op (conforme seu modulo)
28 localparam OP_MUL = 2'b00; // Multiplica o
29 localparam OP_DIV = 2'b01; // Divis o
30 localparam OP_REM = 2'b10; // Resto
31 localparam OP_DEFAULT = 2'b11; // Default/Invalido
32
33 integer error_count = 0;
34 integer test_id = 0;
35
36 // Task para verificar resultados
37 task check;
38     input signed [31:0] expected;
39     input [80*8-1:0] description; // String para descri o do teste (aumentei o
        tamanho)
40     begin
41         test_id = test_id + 1;
42         #0; // Garante avalia o no mesmo instante de tempo, apos propaga o
43         if ($signed(tb_md_result) != expected) begin
44             $display("[%03d] FALHA: %s. Operands: %d (%h), %d (%h). Op: %b. Esperado:
                %d (%h), Obtido: %d (%h)",
45                 test_id, description,
46                 $signed(tb_operand1), tb_operand1, $signed(tb_operand2),
47                 tb_operand2, tb_mul_div_op,
48                 expected, expected, $signed(tb_md_result), tb_md_result);
49             error_count = error_count + 1;
50         end else begin
51             $display("[%03d] SUCESSO: %s. Operands: %d (%h), %d (%h). Op: %b.
                Resultado: %d (%h)",
52                 test_id, description,
53                 $signed(tb_operand1), tb_operand1, $signed(tb_operand2),
54                 tb_operand2, tb_mul_div_op,
55                 $signed(tb_md_result), tb_md_result);
56         end
57     endtask
58
59 // Procedimento de teste
60 initial begin
61     $dumpfile("div_mult_tb.vcd");
62     $dumpvars(0, div_mult_tb); // Dump vars do modulo de testbench e seu DUT
63
64     $display("Iniciando Testes da Unidade div_mult\n");
65     error_count = 0;

```

```

65     test_id = 0;
66
67     // --- Testes MUL (mul_div_op = OP_MUL) ---
68     tb_mul_div_op = OP_MUL;
69     $display("\n--- TESTES DE MULTIPLICACAO (MUL) ---");
70     // Teste 1.1
71     tb_operand1 = 5; tb_operand2 = 10; #10; check(50, "MUL: 5 * 10");
72     // Teste 1.2
73     tb_operand1 = 7; tb_operand2 = -4; #10; check(-28, "MUL: 7 * -4");
74     // Teste 1.3
75     tb_operand1 = -6; tb_operand2 = 8; #10; check(-48, "MUL: -6 * 8");
76     // Teste 1.4
77     tb_operand1 = -3; tb_operand2 = -9; #10; check(27, "MUL: -3 * -9");
78     // Teste 1.5
79     tb_operand1 = 123; tb_operand2 = 0; #10; check(0, "MUL: 123 * 0");
80     // Teste 1.6
81     tb_operand1 = -123; tb_operand2 = 0; #10; check(0, "MUL: -123 * 0");
82     // Teste 1.7
83     tb_operand1 = 456; tb_operand2 = 1; #10; check(456, "MUL: 456 * 1");
84     // Teste 1.8
85     tb_operand1 = 789; tb_operand2 = NEG_ONE_S; #10; check(-789, "MUL: 789 * -1");
86     // Teste 1.9 (Overflow - mul retorna 32 bits inferiores)
87     tb_operand1 = 65536; tb_operand2 = 65536; #10; check(32'd0, "MUL: 65536 * 65536 (
        low 32b)");
88     // Teste 1.10 (Overflow - resultado grande positivo)
89     tb_operand1 = MAX_INT_S; tb_operand2 = 2; #10; check(32'hFFFFFFFE, "MUL:
        MAX_INT_S * 2 (low 32b)");
90     // Teste 1.11 (Overflow - resultado grande negativo)
91     tb_operand1 = MIN_INT_S; tb_operand2 = 2; #10; check(32'h00000000, "MUL:
        MIN_INT_S * 2 (low 32b)");
92
93     // --- Testes DIV (mul_div_op = OP_DIV) ---
94     tb_mul_div_op = OP_DIV;
95     $display("\n--- TESTES DE DIVISAO (DIV) ---");
96     // Teste 2.1
97     tb_operand1 = 20; tb_operand2 = 4; #10; check(5, "DIV: 20 / 4");
98     // Teste 2.2
99     tb_operand1 = 10; tb_operand2 = 3; #10; check(3, "DIV: 10 / 3");
100    // Teste 2.3
101    tb_operand1 = -10; tb_operand2 = 3; #10; check(-3, "DIV: -10 / 3");
102    // Teste 2.4
103    tb_operand1 = 10; tb_operand2 = -3; #10; check(-3, "DIV: 10 / -3");
104    // Teste 2.5
105    tb_operand1 = -10; tb_operand2 = -3; #10; check(3, "DIV: -10 / -3");
106    // Teste 2.6
107    tb_operand1 = 0; tb_operand2 = 7; #10; check(0, "DIV: 0 / 7");
108    // Teste 2.7
109    tb_operand1 = 123; tb_operand2 = 1; #10; check(123, "DIV: 123 / 1");
110    // Teste 2.8
111    tb_operand1 = 123; tb_operand2 = NEG_ONE_S; #10; check(-123, "DIV: 123 / -1");
112    // Teste 2.9 (Divisao por Zero - RISC-V Spec)
113    tb_operand1 = 7; tb_operand2 = 0; #10; check(NEG_ONE_S, "DIV: 7 / 0");
114    // Teste 2.10 (Divisao por Zero, dividendo negativo - RISC-V Spec)
115    tb_operand1 = -7; tb_operand2 = 0; #10; check(NEG_ONE_S, "DIV: -7 / 0");
116    // Teste 2.11 (Overflow da Divisao - MIN_INT_S / -1 - RISC-V Spec)
117    tb_operand1 = MIN_INT_S; tb_operand2 = NEG_ONE_S; #10; check(MIN_INT_S, "DIV:
        MIN_INT_S / -1");
118
119    // --- Testes REM (mul_div_op = OP_REM) ---
120    tb_mul_div_op = OP_REM;

```

```

121     $display("\n--- TESTES DE RESTO (REM) ---");
122     // Teste 3.1
123     tb_operand1 = 10; tb_operand2 = 3; #10; check(1, "REM: 10 % 3");
124     // Teste 3.2
125     tb_operand1 = -10; tb_operand2 = 3; #10; check(-1, "REM: -10 % 3");
126     // Teste 3.3
127     tb_operand1 = 10; tb_operand2 = -3; #10; check(1, "REM: 10 % -3");
128     // Teste 3.4
129     tb_operand1 = -10; tb_operand2 = -3; #10; check(-1, "REM: -10 % -3");
130     // Teste 3.5
131     tb_operand1 = 0; tb_operand2 = 7; #10; check(0, "REM: 0 % 7");
132     // Teste 3.6
133     tb_operand1 = 123; tb_operand2 = 1; #10; check(0, "REM: 123 % 1");
134     // Teste 3.7
135     tb_operand1 = 123; tb_operand2 = NEG_ONE_S; #10; check(0, "REM: 123 % -1");
136     // Teste 3.8 (Resto por Zero - RISC-V Spec)
137     tb_operand1 = 7; tb_operand2 = 0; #10; check(7, "REM: 7 % 0");
138     // Teste 3.9 (Resto por Zero, dividendo negativo - RISC-V Spec)
139     tb_operand1 = -7; tb_operand2 = 0; #10; check(-7, "REM: -7 % 0");
140     // Teste 3.10 (Overflow da Divis o , caso para REM - RISC-V Spec)
141     tb_operand1 = MIN_INT_S; tb_operand2 = NEG_ONE_S; #10; check(0, "REM: MIN_INT_S %
        -1");
142
143     // --- Teste Operacao Default/Invalida ---
144     tb_mul_div_op = OP_DEFAULT;
145     $display("\n--- TESTE DE OPERACAO DEFAULT/INVALIDA ---");
146     // Teste 4.1
147     tb_operand1 = 12345; tb_operand2 = 67890; #10; check(32'd0, "DEFAULT OP: 12345,
        67890");
148
149     $display("\nTestes finalizados.");
150     if (error_count == 0) begin
151         $display("TODOS OS %0d TESTES DA div_mult PASSARAM!", test_id);
152     end else begin
153         $display("%0d de %0d TESTES DA div_mult FALHARAM!", error_count, test_id);
154     end
155     $finish;
156 end
157
158 endmodule

```

Listing C.2 – Testbench completo para a Unidade DIV/MULT.

C.3 Testbench da Unidade de Extensão (extensor_tb.v)

```

1  `timescale 1ns / 1ps
2
3  module extensor_tb;
4
5      // Entrada para o DUT (extensor)
6      reg [31:0] tb_instr; // Renomeado para corresponder ao 'instr' do seu DUT
7
8      // Sa da do DUT
9      wire [31:0] tb_imm_ext; // Renomeado para corresponder ao 'imm_ext' do seu DUT
10
11     // Instancia o m dulo 'extensor'
12     extensor uut (

```

```

13     .instr(tb_instr),           // Conecta tb_instr porta instr do DUT
14     .imm_ext(tb_imm_ext)       // Conecta tb_imm_ext porta imm_ext do DUT
15 );
16
17 // Opcodes (conforme seu m dulo extensor e a especifica o RISC-V)
18 localparam OPCODE_IMM_ARITH = 7'b0010011; // ADDI, SLTI, XORI, ORI, ANDI
19 localparam OPCODE_LOAD      = 7'b0000011; // LW
20 localparam OPCODE_JALR      = 7'b1100111; // JALR
21 localparam OPCODE_STORE     = 7'b0100011; // SW
22 localparam OPCODE_BRANCH    = 7'b1100011; // BEQ, BNE, BLT, BGE
23 localparam OPCODE_JAL       = 7'b1101111; // JAL
24 localparam OPCODE_R_TYPE    = 7'b0110011; // Exemplo de opcode que n o usa imediato
    (para default)
25
26 integer error_count = 0;
27 integer test_id = 0;
28
29 // Task para verificar resultados
30 task check;
31     input [31:0] expected;
32     input [80*8-1:0] description;
33     begin
34         test_id = test_id + 1;
35         #1;
36         if (tb_imm_ext != expected) begin
37             $display("[%03d] FALHA: %s. Instru o: %h. Esperado: %h (%d), Obtido: %h (%d)",
38                 test_id, description, tb_instr,
39                 expected, $signed(expected), tb_imm_ext, $signed(tb_imm_ext));
40             error_count = error_count + 1;
41         end else begin
42             $display("[%03d] SUCESSO: %s. Instru o: %h -> Extendido: %h (%d)",
43                 test_id, description, tb_instr, tb_imm_ext, $signed(tb_imm_ext))
44         end
45     end
46 endtask
47
48 // Procedimento de teste
49 initial begin
50     $dumpfile("extensor_tb.vcd");
51     $dumpvars(0, extensor_tb);
52
53     $display("Iniciando Testes da Unidade Extensor de Imediato (baseado no opcode)\n"
54 );
55     error_count = 0;
56     test_id = 0;
57
58     // --- TESTES TIPO I (usando opcodes relevantes) ---
59     $display("\n--- TESTES TIPO I ---");
60     // Teste I-1: ADDI x1, x0, 5 (imm = 5)
61     // imm[11:0]=5, rs1=x0, funct3=000(ADDI), rd=x1, opcode=OPCODE_IMM_ARITH
62     tb_instr = {12'd5, 5'b00000, 3'b000, 5'b00001, OPCODE_IMM_ARITH};
63     #10; check(32'd5, "Tipo I (ADDI): Imediato +5");
64
65     // Teste I-2: LW x1, -4(x2) (imm = -4)
66     // imm[11:0]=-4, rs1=x2, funct3=010(LW), rd=x1, opcode=OPCODE_LOAD
67     tb_instr = {12'hFFC, 5'b00010, 3'b010, 5'b00001, OPCODE_LOAD};
68     #10; check(32'hFFFFFFFC, "Tipo I (LW): Imediato -4");

```



```

69 // Teste I-3: JALR x0, x1, 2047 (imm = 2047)
70 // imm[11:0]=2047, rs1=x1, funct3=000, rd=x0, opcode=OPCODE_JALR
71 tb_instr = {12'h7FF, 5'b00001, 3'b000, 5'b00000, OPCODE_JALR};
72 #10; check(32'd2047, "Tipo I (JALR): Imediato +2047");
73
74 // Teste I-4: JALR x0, x1, -2048 (imm = -2048)
75 tb_instr = {12'h800, 5'b00001, 3'b000, 5'b00000, OPCODE_JALR};
76 #10; check(32'hFFFFFF800, "Tipo I (JALR): Imediato -2048");
77
78 // --- TESTES TIPO S ---
79 $display("\n--- TESTES TIPO S ---");
80 // Teste S-1: SW x5, 8(x2) (imm = 8)
81 // imm[11:5]=0, imm[4:0]=8, funct3=010(SW), rs1=x2, rs2=x5, opcode=OPCODE_STORE
82 tb_instr = {7'b0000000, 5'b00101, 5'b00010, 3'b010, 5'b01000, OPCODE_STORE};
83 #10; check(32'd8, "Tipo S (SW): Imediato +8");
84
85 // Teste S-2: SW x5, -8(x2) (imm = -8 = 0xFF8)
86 // imm[11:5]=1111111(0x7F), imm[4:0]=11000(0x18)
87 tb_instr = {7'b1111111, 5'b00101, 5'b00010, 3'b010, 5'b11000, OPCODE_STORE};
88 #10; check(32'hFFFFFFF8, "Tipo S (SW): Imediato -8");
89
90 // --- TESTES TIPO B ---
91 $display("\n--- TESTES TIPO B ---");
92
93 // Teste B-1: BEQ x1, x2, +20 (offset = +20)
94 // Para offset +20: 20 = 0b10100
95 // 0 imediato B tem: imm[12:1] (12 bits) + imm[0]=0 sempre
96 // 20 >> 1 = 10 = 0b1010 (removemos o LSB que sempre 0)
97 // Distribui o: imm[12]=0, imm[11]=0, imm[10:5]=000000, imm[4:1]=1010
98 // Mapeamento: [31]=imm[12]=0, [7]=imm[11]=0, [30:25]=imm[10:5]=000000, [11:8]=
99 //               imm[4:1]=1010
100 tb_instr = {1'b0, // instr[31] (imm[12])
101             6'b000000, // instr[30:25] (imm[10:5]) - CORRIGIDO
102             5'b00010, // rs2 (exemplo)
103             5'b00001, // rs1 (exemplo)
104             3'b000, // funct3 (BEQ)
105             4'b1010, // instr[11:8] (imm[4:1])
106             1'b0, // instr[7] (imm[11])
107             OPCODE_BRANCH};
108 #10; check(32'd20, "Tipo B (BEQ): Offset +20");
109
110 // Teste B-2: BGE x1, x2, -20 (offset = -20)
111 // Para offset -20: -20 em complemento de 2
112 // -20 >> 1 = -10, em complemento de 2 (12 bits): 1 1111 1110 1100
113 // imm[12]=1, imm[11]=1, imm[10:5]=11111, imm[4:1]=0110, imm[0]=0
114 tb_instr = {1'b1, // instr[31] (imm[12])
115             6'b111111, // instr[30:25] (imm[10:5]) - CORRIGIDO
116             5'b00010, // rs2
117             5'b00001, // rs1
118             3'b101, // funct3 (BGE)
119             4'b0110, // instr[11:8] (imm[4:1]) - CORRIGIDO
120             1'b1, // instr[7] (imm[11])
121             OPCODE_BRANCH};
122 #10; check(32'hFFFFFFEC, "Tipo B (BGE): Offset -20");
123
124
125 // --- TESTES TIPO J ---
126 $display("\n--- TESTES TIPO J ---");
127

```

```

128 // [ 9] Tipo J (JAL): Offset +1024
129 // Instru o: 0x400000EF (JAL x1, +1024)
130 // Imediato (21 bits) = +1024      extend para 32 bits = 0x00000400
131 tb_instr = {
132     1'b0,           // imm[20] = 0
133     10'b1000000000, // imm[10:1] = 1000000000 (bin)
134     1'b0,           // imm[11] = 0
135     8'b00000000,    // imm[19:12] = 00000000
136     5'b00001,       // rd = x1
137     OPCODE_JAL      // 7'b1101111
138 };
139 #10;
140 check(32'h00000400, "Tipo J (JAL): Offset +1024");
141
142 // [ 10] Tipo J (JAL): Offset -1024
143 // Instrucao: 0xC01FF0EF (JAL x1, -1024)
144 // Imediato (21 bits) = 1024      extend para 32 bits = 0xFFFFFC00
145 tb_instr = {
146     1'b1,           // imm[20] = 1
147     10'b1000000000, // imm[10:1] = 1000000000 (bin)
148     1'b1,           // imm[11] = 1
149     8'b11111111,    // imm[19:12] = 11111111
150     5'b00001,       // rd = x1
151     OPCODE_JAL      // 7'b1101111
152 };
153 #10;
154 check(32'hFFFFFFC00, "Tipo J (JAL): Offset -1024");
155 $display("\nTestes do Extensor de Imediato finalizados.");
156 if (error_count == 0) begin
157     $display("TODOS OS %0d TESTES DO EXTENSOR PASSARAM!", test_id);
158 end else begin
159     $display("%0d de %0d TESTES DO EXTENSOR FALHARAM!", error_count, test_id);
160 end
161 $finish;
162 end
163
164 endmodule

```

Listing C.3 – Testbench completo para a Unidade de Extensão de bit.

C.4 Testbench da Unidade de Banco de Registradores (bancoderegistradores)

```

1
2 `timescale 1ns / 1ps
3
4 module bancoderegistradores_tb;
5
6     // Sinais do Testbench
7     reg        tb_clock;
8     reg        tb_reset;
9     reg        tb_reg_write;
10    reg [4:0]   tb_read_addr1;
11    reg [4:0]   tb_read_addr2;
12    reg [4:0]   tb_write_addr;
13    reg [31:0]  tb_write_data;
14
15    wire [31:0] tb_read_data1;

```

```

16     wire [31:0] tb_read_data2;
17
18     integer      error_count = 0;
19     integer      test_id = 0;
20
21     // Instancia o do DUT (Device Under Test)
22     bancoderegistradores uut (
23         .clock(tb_clock),
24         .reset(tb_reset),
25         .reg_write(tb_reg_write),
26         .read_addr1(tb_read_addr1),
27         .read_addr2(tb_read_addr2),
28         .write_addr(tb_write_addr),
29         .write_data(tb_write_data),
30         .read_data1(tb_read_data1),
31         .read_data2(tb_read_data2)
32     );
33
34     // Gera o de Clock
35     always #5 tb_clock = ~tb_clock; // Período de 10ns
36
37     // Task para verificar leitura
38     task check_read;
39         input [4:0] addr1;
40         input [4:0] addr2;
41         input [31:0] expected_data1;
42         input [31:0] expected_data2;
43         input [80*8-1:0] description;
44         begin
45             test_id = test_id + 1;
46             tb_read_addr1 = addr1;
47             tb_read_addr2 = addr2;
48             #1; // Espera para propaga o combinacional da leitura
49             if (tb_read_data1 != expected_data1 || tb_read_data2 != expected_data2)
50                 begin
51                     $display("[%0t ns][%03d] FALHA Leitura: %s. Addr1=%d, Addr2=%d.", $time,
52                         test_id, description, addr1, addr2);
53                     $display("                Esperado D1=%h, D2=%h. Obtido D1=%h, D2
54                         =%h",
55                         expected_data1, expected_data2, tb_read_data1, tb_read_data2);
56                     error_count = error_count + 1;
57                 end else begin
58                     $display("[%0t ns][%03d] SUCESSO Leitura: %s. Addr1=%d->D1=%h, Addr2=%d->
59                         D2=%h",
60                         $time, test_id, description, addr1, tb_read_data1, addr2,
61                         tb_read_data2);
62                 end
63             endtask
64
65     // Procedimento de teste
66     initial begin
67         $dumpfile("bancoderegistradores_tb.vcd");
68         $dumpvars(0, bancoderegistradores_tb); // Dump de todas as variáveis do
69             testbench e DUT
70
71         $display("Iniciando Testbench para bancoderegistradores...");
72         error_count = 0;
73         test_id = 0;
74         tb_clock = 0;

```

```

70
71 // 1. Aplicar Reset
72 $display("\n--- Teste de Reset ---");
73 tb_reset = 1;
74 tb_reg_write = 0; // Nenhuma escrita durante o reset
75 tb_write_addr = 5'b0;
76 tb_write_data = 32'hDEADBEEF;
77 #20; // Manter reset por alguns ciclos de clock
78 tb_reset = 0;
79 #1; // Liberar reset
80 check_read(5'd1, 5'd2, 32'd0, 32'd0, "Leitura ap s Reset (esperado regs zerados)");
81 check_read(5'd0, 5'd15, 32'd0, 32'd0, "Leitura de x0 e outro reg ap s Reset");
82
83 // 2. Escrita Simples e Leitura
84 $display("\n--- Teste de Escrita e Leitura Simples ---");
85 // Escrever em x1
86 tb_reg_write = 1;
87 tb_write_addr = 5'd1; // x1
88 tb_write_data = 32'hAAAAAAAA;
89 @(posedge tb_clock); // Espera a borda do clock para a escrita
90 #1; tb_reg_write = 0; // Desabilita escrita ap s o ciclo
91 check_read(5'd1, 5'd2, 32'hAAAAAAAA, 32'd0, "Ler x1 ap s escrita, x2 n o
    alterado");
92
93 // Escrever em x2
94 tb_reg_write = 1;
95 tb_write_addr = 5'd2; // x2
96 tb_write_data = 32'hBBBBBBBB;
97 @(posedge tb_clock);
98 #1; tb_reg_write = 0;
99 check_read(5'd1, 5'd2, 32'hAAAAAAAA, 32'hBBBBBBBB, "Ler x1 e x2 ap s escrita em
    x2");
100
101 // 3. Leitura de x0
102 $display("\n--- Teste de Leitura de x0 ---");
103 check_read(5'd0, 5'd1, 32'd0, 32'hAAAAAAAA, "Ler x0 (deve ser 0) e x1");
104
105 // 4. Tentativa de Escrita em x0
106 $display("\n--- Teste de Tentativa de Escrita em x0 ---");
107 tb_reg_write = 1;
108 tb_write_addr = 5'd0; // x0
109 tb_write_data = 32'h12345678; // Dado que n o deveria ser escrito
110 @(posedge tb_clock);
111 #1; tb_reg_write = 0;
112 check_read(5'd0, 5'd1, 32'd0, 32'hAAAAAAAA, "Ler x0 apos tentativa de escrita (
    deve ser 0)");
113
114 // 5. Escrita Desabilitada (RegWrite = 0)
115 $display("\n--- Teste com RegWrite Desabilitado ---");
116 tb_reg_write = 0; // RegWrite desabilitado
117 tb_write_addr = 5'd3; // x3
118 tb_write_data = 32'hCCCCCCCC;
119 @(posedge tb_clock);
120 #1;
121 check_read(5'd3, 5'd1, 32'd0, 32'hAAAAAAAA, "Ler x3 apos tentativa de escrita com
    RegWrite=0 (x3 deve ser 0)");
122
123 // 6. Sobrescrever Registrador
124 $display("\n--- Teste de Sobrescrita de Registrador ---");

```

```

125     tb_reg_write = 1;
126     tb_write_addr = 5'd1; // x1 (que continha AAAAAAAA)
127     tb_write_data = 32'h11112222;
128     @(posedge tb_clock);
129     #1; tb_reg_write = 0;
130     check_read(5'd1, 5'd2, 32'h11112222, 32'hBBBBBBBB, "Ler x1 sobrescrito e x2");
131
132     // 7. Leitura de dois registradores diferentes simultaneamente
133     $display("\n--- Teste de Leitura Simultanea ---");
134     // x1 j    tem 11112222, x2 j    tem BBBBBBBB
135     // Escrever em x5 e x10 para ter mais valores
136     tb_reg_write = 1;
137     tb_write_addr = 5'd5; tb_write_data = 32'h05050505; @(posedge tb_clock); #1;
138     tb_write_addr = 5'd10; tb_write_data = 32'h10101010; @(posedge tb_clock); #1;
139     tb_reg_write = 0;
140     check_read(5'd5, 5'd10, 32'h05050505, 32'h10101010, "Leitura de x5 e x10");
141     check_read(5'd1, 5'd10, 32'h11112222, 32'h10101010, "Leitura de x1 e x10");
142
143
144     $display("\nTestes do Banco de Registradores finalizados.");
145     if (error_count == 0) begin
146         $display("TODOS OS %0d TESTES DO BANCO DE REGISTRADORES PASSARAM!", test_id);
147     end else begin
148         $display("%0d de %0d TESTES DO BANCO DE REGISTRADORES FALHARAM!", error_count
149             , test_id);
150     end
151     $finish;
152 end
153 endmodule

```

Listing C.4 – Testbench completo para a Unidade Banco de Registradores.

C.5 Testbench da Unidade de Mux1 (mux1_tb.v)

```

1
2 `timescale 1ns / 1ps
3
4 module mux1_tb;
5
6     // Entradas para o DUT
7     reg [31:0] tb_in_read_data2;
8     reg [31:0] tb_in_imm_extended;
9     reg        tb_sel_alu_src;
10
11     // Saída do DUT
12     wire [31:0] tb_out_alu_operand_b;
13
14     // Instancia o DUT
15     Mux1 uut (
16         .in_read_data2(tb_in_read_data2),
17         .in_imm_extended(tb_in_imm_extended),
18         .sel_alu_src(tb_sel_alu_src),
19         .out_alu_operand_b(tb_out_alu_operand_b)
20     );
21
22     // Procedimento de teste

```

```

23  initial begin
24      $dumpfile("mux1_tb.vcd");
25      $dumpvars(0, mux1_tb);
26
27      $display("Iniciando Testbench para mux1...");
28
29      // Teste 1: Selecionar in_read_data2
30      tb_in_read_data2    = 32'hAAAAAAAA;
31      tb_in_imm_extended = 32'hBBBBBBBB;
32      tb_sel_alu_src      = 1'b0; // Seleciona in_read_data2
33      #10;
34      if (tb_out_alu_operand_b === 32'hAAAAAAAA) begin
35          $display("[%0t ns] SUCESSO: sel_alu_src=0, Saida=%h (Esperado AAAAAAAA)",
36                  $time, tb_out_alu_operand_b);
37      end else begin
38          $error("[%0t ns] FALHA: sel_alu_src=0, Saida=%h (Esperado AAAAAAAA)", $time,
39                 tb_out_alu_operand_b);
40      end
41
42      // Teste 2: Selecionar in_imm_extended
43      tb_in_read_data2    = 32'hCCCCCCCC; // Mudar valores para garantir que n o o
44      anterior
45      tb_in_imm_extended = 32'hDDDDDDDD;
46      tb_sel_alu_src      = 1'b1; // Seleciona in_imm_extended
47      #10;
48      if (tb_out_alu_operand_b === 32'hDDDDDDDD) begin
49          $display("[%0t ns] SUCESSO: sel_alu_src=1, Saida=%h (Esperado DDDDDDDD)",
50                  $time, tb_out_alu_operand_b);
51      end else begin
52          $error("[%0t ns] FALHA: sel_alu_src=1, Saida=%h (Esperado DDDDDDDD)", $time,
53                 tb_out_alu_operand_b);
54      end
55
56      // Teste 3: Mudar entradas com mesma sele o (opcional, para ver se a sa da
57      muda)
58      tb_in_read_data2    = 32'h11111111;
59      tb_in_imm_extended = 32'h22222222;
60      tb_sel_alu_src      = 1'b1; // Mant m selecionando in_imm_extended
61      #10;
62      if (tb_out_alu_operand_b === 32'h22222222) begin
63          $display("[%0t ns] SUCESSO: sel_alu_src=1 (novas entradas), Saida=%h (
64                  Esperado 22222222)", $time, tb_out_alu_operand_b);
65      end else begin
66          $error("[%0t ns] FALHA: sel_alu_src=1 (novas entradas), Saida=%h (Esperado
67                  22222222)", $time, tb_out_alu_operand_b);
68      end
69
70      $display("Testes do mux_alu_operand_b finalizados.");
71      $finish;
72  end
73 endmodule

```

Listing C.5 – Testbench completo para a Unidade Multiplexadora 1.

C.6 Testbench da Unidade de Mux2 (mux2.v)

```

1
2 `timescale 1ns / 1ps
3
4 module mux2_tb;
5
6     // Entradas para o DUT
7     reg [31:0] tb_in_alu_result;
8     reg [31:0] tb_in_md_result;
9     reg          tb_sel_use_md;
10
11     // Saída do DUT
12     wire [31:0] tb_out_conta_final;
13
14     // Instancia o DUT
15     Mux2 uut (
16         .in_alu_result(tb_in_alu_result),
17         .in_md_result(tb_in_md_result),
18         .sel_use_md(tb_sel_use_md),
19         .out_conta_final(tb_out_conta_final)
20     );
21
22     // Procedimento de teste
23     initial begin
24         $dumpfile("mux2_tb.vcd");
25         $dumpvars(0, mux2_tb);
26
27         $display("Iniciando Testbench para mux2...");
28
29         // Teste 1: Selecionar in_alu_result
30         tb_in_alu_result = 32'h11223344;
31         tb_in_md_result  = 32'h55667788;
32         tb_sel_use_md    = 1'b0; // Seleciona in_alu_result
33         #10;
34         if (tb_out_conta_final === 32'h11223344) begin
35             $display("[%0t ns] SUCESSO: sel_use_md=0, Saida=%h (Esperado 11223344)",
36                 $time, tb_out_conta_final);
37         end else begin
38             $error("[%0t ns] FALHA: sel_use_md=0, Saida=%h (Esperado 11223344)", $time,
39                 tb_out_conta_final);
40         end
41
42         // Teste 2: Selecionar in_md_result
43         tb_in_alu_result = 32'h99AABBCC;
44         tb_in_md_result  = 32'hDDEEFF00;
45         tb_sel_use_md    = 1'b1; // Seleciona in_md_result
46         #10;
47         if (tb_out_conta_final === 32'hDDEEFF00) begin
48             $display("[%0t ns] SUCESSO: sel_use_md=1, Saida=%h (Esperado DDEEFF00)",
49                 $time, tb_out_conta_final);
50         end else begin
51             $error("[%0t ns] FALHA: sel_use_md=1, Saida=%h (Esperado DDEEFF00)", $time,
52                 tb_out_conta_final);
53         end
54
55         $display("Testes do mux_writeback_source1 finalizados.");
56         $finish;
57     end
58 end

```

55 `endmodule`

Listing C.6 – Testbench completo para a Unidade Multiplexadora 2.

C.7 Testbench da Unidade de PC (program_counter_tb.v)

```

1
2 `timescale 1ns / 1ps
3
4 module mux2_tb;
5
6     // Entradas para o DUT
7     reg [31:0] tb_in_alu_result;
8     reg [31:0] tb_in_md_result;
9     reg         tb_sel_use_md;
10
11     // Saída do DUT
12     wire [31:0] tb_out_conta_final;
13
14     // Instancia o DUT
15     Mux2 uut (
16         .in_alu_result(tb_in_alu_result),
17         .in_md_result(tb_in_md_result),
18         .sel_use_md(tb_sel_use_md),
19         .out_conta_final(tb_out_conta_final)
20     );
21
22     // Procedimento de teste
23     initial begin
24         $dumpfile("mux2_tb.vcd");
25         $dumpvars(0, mux2_tb);
26
27         $display("Iniciando Testbench para mux2...");
28
29         // Teste 1: Selecionar in_alu_result
30         tb_in_alu_result = 32'h11223344;
31         tb_in_md_result  = 32'h55667788;
32         tb_sel_use_md    = 1'b0; // Seleciona in_alu_result
33         #10;
34         if (tb_out_conta_final == 32'h11223344) begin
35             $display("[%0t ns] SUCESSO: sel_use_md=0, Saida=%h (Esperado 11223344)",
36                 $time, tb_out_conta_final);
37         end else begin
38             $error("[%0t ns] FALHA: sel_use_md=0, Saida=%h (Esperado 11223344)", $time,
39                 tb_out_conta_final);
40         end
41
42         // Teste 2: Selecionar in_md_result
43         tb_in_alu_result = 32'h99AABBCC;
44         tb_in_md_result  = 32'hDDEEFF00;
45         tb_sel_use_md    = 1'b1; // Seleciona in_md_result
46         #10;
47         if (tb_out_conta_final == 32'hDDEEFF00) begin
48             $display("[%0t ns] SUCESSO: sel_use_md=1, Saida=%h (Esperado DDEEFF00)",
49                 $time, tb_out_conta_final);
50         end else begin
51             $error("[%0t ns] FALHA: sel_use_md=1, Saida=%h (Esperado DDEEFF00)", $time,
52                 tb_out_conta_final);
53         end
54     end
55 endmodule

```



```

49     end
50
51     $display("Testes do mux_writeback_source1 finalizados.");
52     $finish;
53 end
54
55 endmodule

```

Listing C.7 – Testbench completo para a Unidade de Program Counter.

C.8 Testbench da Unidade de PC+4 (*pc_plus_4adder_tb.v*)

```

1
2 `timescale 1ns / 1ps
3
4 module pc_plus_4_adder_tb;
5     reg [31:0] current_pc_tb;
6     wire [31:0] pc_plus_4_tb;
7
8     pc_plus_4_adder uut (
9         .current_pc(current_pc_tb), .pc_plus_4(pc_plus_4_tb)
10    );
11
12    initial begin
13        $dumpfile("pc_plus_4_tb.vcd"); $dumpvars(0, pc_plus_4_adder_tb);
14        $display("Iniciando Testbench para PC_Plus_4 Adder...");
15
16        current_pc_tb = 32'h00000000; #10;
17        $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000004)", $time, current_pc_tb,
18            pc_plus_4_tb);
19        if(pc_plus_4_tb !== 32'h00000004) $error("Falha PC=0!");
20
21        current_pc_tb = 32'h00000004; #10;
22        $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000008)", $time, current_pc_tb,
23            pc_plus_4_tb);
24        if(pc_plus_4_tb !== 32'h00000008) $error("Falha PC=4!");
25
26        current_pc_tb = 32'hFFFFFFF8; #10; // Perto do limite
27        $display("[%0t ns] PC=%h, PC+4=%h (Esperado FFFFFFFC)", $time, current_pc_tb,
28            pc_plus_4_tb);
29        if(pc_plus_4_tb !== 32'hFFFFFFFC) $error("Falha PC=FFFFFFF8!");
30
31        current_pc_tb = 32'hFFFFFFFC; #10; // ultimo endereco alinhado antes do overflow
32        $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000000 com wrap-around)", $time,
33            current_pc_tb, pc_plus_4_tb);
34        if(pc_plus_4_tb !== 32'h00000000) $error("Falha PC=FFFFFFFC (wrap-around)!");
35
36        $display("Testes do PC_Plus_4 Adder finalizados.");
37        $finish;
38    end
39 endmodule

```

Listing C.8 – Testbench completo para a Unidade Program Counter +4.

C.9 Testbench da Unidade de PC Target Adder (pc_target_adder_tb.v)

```

1  `timescale 1ns / 1ps
2
3
4  module pc_plus_4_adder_tb;
5      reg [31:0] current_pc_tb;
6      wire [31:0] pc_plus_4_tb;
7
8      pc_plus_4_adder uut (
9          .current_pc(current_pc_tb), .pc_plus_4(pc_plus_4_tb)
10     );
11
12     initial begin
13         $dumpfile("pc_plus_4_tb.vcd"); $dumpvars(0, pc_plus_4_adder_tb);
14         $display("Iniciando Testbench para PC_Plus_4 Adder...");
15
16         current_pc_tb = 32'h00000000; #10;
17         $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000004)", $time, current_pc_tb,
18             pc_plus_4_tb);
19         if(pc_plus_4_tb !== 32'h00000004) $error("Falha PC=0!");
20
21         current_pc_tb = 32'h00000004; #10;
22         $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000008)", $time, current_pc_tb,
23             pc_plus_4_tb);
24         if(pc_plus_4_tb !== 32'h00000008) $error("Falha PC=4!");
25
26         current_pc_tb = 32'hFFFFFFF8; #10; // Perto do limite
27         $display("[%0t ns] PC=%h, PC+4=%h (Esperado FFFFFFFC)", $time, current_pc_tb,
28             pc_plus_4_tb);
29         if(pc_plus_4_tb !== 32'hFFFFFFFC) $error("Falha PC=FFFFFFF8!");
30
31         current_pc_tb = 32'hFFFFFFFC; #10; // ultimo endereco alinhado antes do overflow
32         $display("[%0t ns] PC=%h, PC+4=%h (Esperado 00000000 com wrap-around)", $time,
33             current_pc_tb, pc_plus_4_tb);
34         if(pc_plus_4_tb !== 32'h00000000) $error("Falha PC=FFFFFFFC (wrap-around)!");
35
36         $display("Testes do PC_Plus_4 Adder finalizados.");
37         $finish;
38     end
39 endmodule

```

Listing C.9 – Testbench completo para a Unidade PC Target Adder.

C.10 Testbench da Unidade de MUX4 (mux_pc_source_tb.v)

```

1  `timescale 1ns / 1ps
2
3
4  module mux_pc_source_tb;
5
6      // Entradas para o DUT
7      reg [31:0] tb_pc_plus_4;
8      reg [31:0] tb_pc_target_branch_jal;
9      reg [31:0] tb_alu_result_jalr;
10     reg [1:0]  tb_sel_pc_src;

```

```

11
12 // Saida do DUT
13 wire [31:0] tb_out_pc_in;
14
15 // Instancia o DUT
16 mux_pc_source uut (
17     .in_pc_plus_4(tb_pc_plus_4),
18     .in_pc_target_branch_jal(tb_pc_target_branch_jal),
19     .in_alu_result_jalr(tb_alu_result_jalr),
20     .sel_pc_src(tb_sel_pc_src),
21     .out_pc_in(tb_out_pc_in)
22 );
23
24 integer error_count = 0;
25 integer test_id = 0;
26
27 // Task para verificar
28 task check;
29     input [31:0] expected;
30     input [60*8-1:0] description;
31     begin
32         test_id = test_id + 1;
33         #1;
34         if (tb_out_pc_in != expected) begin
35             $display("[%0t ns][%02d] FALHA: %s. Sel=%b. Esperado: %h, Obtido: %h",
36                 $time, test_id, description, tb_sel_pc_src, expected,
37                 tb_out_pc_in);
38             error_count = error_count + 1;
39         end else begin
40             $display("[%0t ns][%02d] SUCESSO: %s. Sel=%b -> Saida: %h",
41                 $time, test_id, description, tb_sel_pc_src, tb_out_pc_in);
42         end
43     endtask
44
45 initial begin
46     $dumpfile("mux_pc_source_tb.vcd");
47     $dumpvars(0, mux_pc_source_tb);
48
49     $display("Iniciando Testbench para mux_pc_source (MUX4)...");
50     error_count = 0;
51     test_id = 0;
52
53     tb_pc_plus_4          = 32'h0000_1004;
54     tb_pc_target_branch_jal = 32'h0000_2020;
55     tb_alu_result_jalr    = 32'h0000_3030;
56
57     // Teste 1: Selecionar PC+4
58     tb_sel_pc_src = 2'b00;
59     #10; check(tb_pc_plus_4, "Seleciona PC+4");
60
61     // Teste 2: Selecionar PC_Target_Branch/JAL
62     tb_sel_pc_src = 2'b01;
63     #10; check(tb_pc_target_branch_jal, "Seleciona PC_Target_Branch/JAL");
64
65     // Teste 3: Selecionar Alu_Result_JALR
66     tb_sel_pc_src = 2'b10;
67     #10; check(tb_alu_result_jalr, "Seleciona Alu_Result_JALR");
68
69     // Teste 4: Mudar entradas com mesma sele o (ex: PC+4)

```

```

70     tb_pc_plus_4 = 32'hAAAA_BBBB;
71     tb_sel_pc_src = 2'b00;
72     #10; check(tb_pc_plus_4, "Seleciona PC+4 (novo valor)");
73
74     // Teste 5: Seleção Default (não usada)
75     tb_sel_pc_src = 2'b11;
76     #10; check(tb_pc_plus_4, "Seleciona Default (deve ser PC+4)");
77
78
79     $display("\nTestes do mux_pc_source finalizados.");
80     if (error_count == 0) begin
81         $display("TODOS OS %0d TESTES DO MUX_PC_SOURCE PASSARAM!", test_id);
82     end else begin
83         $display("%0d de %0d TESTES DO MUX_PC_SOURCE FALHARAM!", error_count, test_id);
84     end
85     $finish;
86 end
87
88 endmodule

```

Listing C.10 – Testbench completo para a Unidade Multiplexação do PC.

C.11 Testbench da Unidade de Processamento (UnidadeProcessamento_tb.v)

```

1
2 `timescale 1ns / 1ps
3
4 module UnidadeProcessamento_tb; // Testbench para UnidadeProcessamento
5
6     // Sinais do Testbench
7     reg        clk_tb;
8     reg        reset_tb;
9     reg        reg_write_en_tb;
10
11     reg        alu_src_sel_tb;
12     reg [3:0]  alu_control_op_tb;
13     reg [1:0]  mul_div_op_sel_tb;
14     reg        use_md_sel_tb;
15     reg [31:0] instruction_tb;
16
17     wire [31:0] core_result_out_tb;
18     wire [31:0] reg_read_data1_tb;
19     wire [31:0] reg_read_data2_tb;
20     wire        alu_zero_flag_tb;
21
22     // Instanciacao do DUT (Device Under Test)
23
24     UnidadeProcessamento dut (
25         .clk            (clk_tb),
26         .reset          (reset_tb),
27         .reg_write_en   (reg_write_en_tb),
28         // .imm_type_sel  (imm_type_sel_tb), // REMOVIDO
29         .alu_src_sel     (alu_src_sel_tb),
30         .alu_control_op  (alu_control_op_tb),
31         .mul_div_op_sel  (mul_div_op_sel_tb),
32         .use_md_sel      (use_md_sel_tb),

```

```

33     .instruction      (instruction_tb),
34     .core_result_out (core_result_out_tb),
35     .reg_read_data1  (reg_read_data1_tb),
36     .reg_read_data2  (reg_read_data2_tb),
37     .alu_zero_flag   (alu_zero_flag_tb)
38 );
39
40 // Geracao de Clock
41 always #5 clk_tb = ~clk_tb; // Período de 10ns
42
43
44 // Opcodes (para montar instruction_tb)
45 localparam OPCODE_R_TYPE   = 7'b0110011;
46 localparam OPCODE_I_ARITH  = 7'b0010011; // ADDI, SLTI, etc.
47 // localparam OPCODE_LOAD  = 7'b0000011; // LW (se fosse testar aqui)
48 // localparam OPCODE_STORE = 7'b0100011; // SW (se fosse testar aqui)
49 // localparam OPCODE_BRANCH = 7'b1100011; // BEQ, etc. (se fosse testar aqui)
50 // localparam OPCODE_JALR  = 7'b1100111; // JALR (se fosse testar aqui)
51 // localparam OPCODE_JAL   = 7'b1101111; // JAL (se fosse testar aqui)
52
53 // Funct3 para instrucoes Tipo R e I (exemplos)
54 localparam FUNCT3_ADD_SUB = 3'b000;
55 localparam FUNCT3_MUL     = 3'b000; // Para extens\o M
56 // ... adicione outros funct3
57
58 // Funct7 para instrucoes Tipo R (exemplos)
59 localparam FUNCT7_ADD = 7'b0000000;
60 localparam FUNCT7_SUB = 7'b0100000;
61 localparam FUNCT7_MUL = 7'b0000001; // Para extensao M
62 // ... adicione outros funct7
63
64 // Operacoes da ULA (mapeamento para alu_control_op_tb)
65 localparam TB_ALU_ADD = 4'b0010;
66 localparam TB_ALU_SUB = 4'b0011;
67
68 // Operacoes da Unidade DIV/MULT (mapeamento para mul_div_op_sel_tb)
69 localparam TB_OP_MUL = 2'b00;
70 localparam TB_OP_DIV = 2'b01;
71 localparam TB_OP_REM = 2'b10;
72
73 // Procedimento de Teste
74 initial begin
75     $dumpfile("UnidadeProcessamento_tb.vcd");
76     $dumpvars(0, UnidadeProcessamento_tb);
77
78     clk_tb = 0;
79     reset_tb = 1;
80     // Inicializar sinais de controle
81     reg_write_en_tb = 0;
82     //imm_type_sel_tb = 3'b000; // REMOVIDO
83     alu_src_sel_tb = 0;
84     alu_control_op_tb = TB_ALU_ADD; // Default
85     mul_div_op_sel_tb = TB_OP_MUL; // Default
86     use_md_sel_tb = 0;
87     instruction_tb = {12'd0, 5'b0, 3'b0, 5'b0, OPCODE_I_ARITH}; // NOP (addi x0,
                        x0, 0)
88     #20; // Manter reset
89
90     reset_tb = 0;
91     $display("Iniciando Testes da UnidadeProcessamento...");

```

```

92     #10;
93
94     // --- Teste 1: Escrever 10 em x1 (ADDI x1, x0, 10) ---
95     // opcode=OPCODE_I_ARITH, rd=x1(1), funct3=FUNCT3_ADD_SUB(000), rs1=x0(0), imm=10
96     instruction_tb    = {12'd10, 5'b00000, FUNCT3_ADD_SUB, 5'b00001, OPCODE_I_ARITH};
97     //imm_type_sel_tb  = TB_IMM_TYPE_I; // Nao eh mais necessario se o extensor usa
        opcode
98     alu_src_sel_tb    = 1; // Seleciona imediato para ULA
99     alu_control_op_tb = TB_ALU_ADD;
100    reg_write_en_tb   = 1;
101    use_md_sel_tb     = 0; // Resultado da ULA
102    @(posedge clk_tb); #1;
103    $display("[%0t ns] ADDI x1,x0,10: instruction=%h, core_result_out=%d (esperado
        10).", $time, instruction_tb, $signed(core_result_out_tb));
104    if ($signed(core_result_out_tb) != 10) $error("Falha ADDI x1");
105
106    // --- Teste 2: Escrever 20 em x2 (ADDI x2, x0, 20) ---
107    instruction_tb    = {12'd20, 5'b00000, FUNCT3_ADD_SUB, 5'b00010, OPCODE_I_ARITH};
108    // Sinais de controle para ADDI permanecem os mesmos
109    @(posedge clk_tb); #1;
110    $display("[%0t ns] ADDI x2,x0,20: instruction=%h, core_result_out=%d (esperado
        20).", $time, instruction_tb, $signed(core_result_out_tb));
111    if ($signed(core_result_out_tb) != 20) $error("Falha ADDI x2");
112
113    // --- Teste 3: ADD x3, x1, x2 (x1=10, x2=20 -> x3=30) ---
114    // opcode=OPCODE_R_TYPE, rd=x3(3), funct3=FUNCT3_ADD_SUB(000), rs1=x1(1), rs2=x2
        (2), funct7=FUNCT7_ADD
115    instruction_tb    = {FUNCT7_ADD, 5'b00010, 5'b00001, FUNCT3_ADD_SUB, 5'b00011,
        OPCODE_R_TYPE};
116    alu_src_sel_tb    = 0; // Seleciona ReadData2 para ULA
117    alu_control_op_tb = TB_ALU_ADD;
118    reg_write_en_tb   = 1;
119    use_md_sel_tb     = 0; // Resultado da ULA
120    @(posedge clk_tb); #1;
121    $display("[%0t ns] ADD x3,x1,x2: instruction=%h, ReadData1=%d, ReadData2=%d,
        core_result_out=%d, Zero=%b",
122            $time, instruction_tb, $signed(reg_read_data1_tb), $signed(
        reg_read_data2_tb), $signed(core_result_out_tb), alu_zero_flag_tb);
123    if ($signed(core_result_out_tb) != 30 || $signed(reg_read_data1_tb) != 10 ||
        $signed(reg_read_data2_tb) != 20) $error("Falha ADD x3");
124
125
126    // --- Teste 4: MUL x5, x1, x2 (x1=10, x2=20 -> x5=200) ---
127    // opcode=OPCODE_R_TYPE, rd=x5(5), funct3=FUNCT3_MUL(000), rs1=x1(1), rs2=x2(2),
        funct7=FUNCT7_MUL
128    instruction_tb    = {FUNCT7_MUL, 5'b00010, 5'b00001, FUNCT3_MUL, 5'b00101,
        OPCODE_R_TYPE};
129    mul_div_op_sel_tb = TB_OP_MUL;
130    reg_write_en_tb   = 1;
131    use_md_sel_tb     = 1; // Seleciona resultado da unidade DIV/MULT
132    // alu_src_sel_tb e alu_control_op_tb nao sao criticos para MUL, mas podem ser
        mantidos ou zerados
133    @(posedge clk_tb); #1;
134    $display("[%0t ns] MUL x5,x1,x2: instruction=%h, ReadData1=%d, ReadData2=%d,
        core_result_out=%d",
135            $time, instruction_tb, $signed(reg_read_data1_tb), $signed(
        reg_read_data2_tb), $signed(core_result_out_tb));
136    if ($signed(core_result_out_tb) != 200 || $signed(reg_read_data1_tb) != 10 ||
        $signed(reg_read_data2_tb) != 20) $error("Falha MUL x5");
137

```

```

138
139 // Desabilitar escrita para os proximos ciclos de leitura apenas
140 reg_write_en_tb = 0;
141 #10; // Espera para garantir que a escrita anterior tenha terminado e os sinais
      de leitura propaguem
142
143 // Ler x1 e x3 para verificar
144 // A instrucao em si nao importa muito aqui, desde que nao escreva nos
      registradores.
145 // 0 que importa sao os enderecos rs1 e rs2 da instrucao para ler banco de
      registradores.
146 // Usando uma instrucao NOP conceitual, mas apenas configurando os campos de
      leitura:
147 instruction_tb = 32'b0; // Limpa instrucao
148 instruction_tb[19:15] = 5'd1; // rs1 = x1
149 instruction_tb[24:20] = 5'd3; // rs2 = x3
150 #10; // Deixar os sinais de leitura propagarem
151 $display("[%0t ns] Leitura Pos-Testes: ReadData1 (x1) = %d (Esperado 10),
      ReadData2 (x3) = %d (Esperado 30)",
152           $time, $signed(reg_read_data1_tb), $signed(reg_read_data2_tb));
153 if ($signed(reg_read_data1_tb) != 10 || $signed(reg_read_data2_tb) != 30)
      $error("Falha na leitura x1/x3");
154
155 // Ler x5 e x0
156 instruction_tb = 32'b0; // Limpa instrucao
157 instruction_tb[19:15] = 5'd5; // rs1 = x5
158 instruction_tb[24:20] = 5'd0; // rs2 = x0
159 #10;
160 $display("[%0t ns] Leitura Pos-Testes: ReadData1 (x5) = %d (Esperado 200),
      ReadData2 (x0) = %d (Esperado 0)",
161           $time, $signed(reg_read_data1_tb), $signed(reg_read_data2_tb));
162 if ($signed(reg_read_data1_tb) != 200 || $signed(reg_read_data2_tb) != 0)
      $error("Falha na leitura x5/x0");
163
164
165 $display("\nTestes da UnidadeProcessamento finalizados.");
166 $finish;
167 end
168
169 endmodule

```

Listing C.11 – Testbench completo para a Unidade Completa de Processamento.

APÊNDICE D – Assembler

```

1
2 import re
3
4 class RiscVAssembler:
5     """
6     Um Assembler simples para instrucoes RV32I/M para gerar codigo de maquina.
7     """
8     def __init__(self):
9         # Mapeamento de registradores numeros
10        self.reg_map = {f'x{i}': i for i in range(32)}
11        self.reg_map.update({
12            'zero': 0, 'ra': 1, 'sp': 2, 'gp': 3, 'tp': 4,
13            't0': 5, 't1': 6, 't2': 7, 's0': 8, 'fp': 8, 's1': 9,
14            'a0': 10, 'a1': 11, 'a2': 12, 'a3': 13, 'a4': 14, 'a5': 15,
15            'a6': 16, 'a7': 17, 's2': 18, 's3': 19, 's4': 20, 's5': 21,
16            's6': 22, 's7': 23, 's8': 24, 's9': 25, 's10': 26, 's11': 27,
17            't3': 28, 't4': 29, 't5': 30, 't6': 31,
18        })
19
20        # Definicoess das instrucoes com base nos formatos
21        self.instructions = {
22            # R-Type
23            'add': {'type': 'R', 'opcode': 0x33, 'funct3': 0x0, 'funct7': 0x00},
24            'sub': {'type': 'R', 'opcode': 0x33, 'funct3': 0x0, 'funct7': 0x20},
25            'sll': {'type': 'R', 'opcode': 0x33, 'funct3': 0x1, 'funct7': 0x00},
26            'slt': {'type': 'R', 'opcode': 0x33, 'funct3': 0x2, 'funct7': 0x00},
27            'xor': {'type': 'R', 'opcode': 0x33, 'funct3': 0x4, 'funct7': 0x00},
28            'srl': {'type': 'R', 'opcode': 0x33, 'funct3': 0x5, 'funct7': 0x00},
29            'sra': {'type': 'R', 'opcode': 0x33, 'funct3': 0x5, 'funct7': 0x20},
30            'or': {'type': 'R', 'opcode': 0x33, 'funct3': 0x6, 'funct7': 0x00},
31            'and': {'type': 'R', 'opcode': 0x33, 'funct3': 0x7, 'funct7': 0x00},
32            'mul': {'type': 'R', 'opcode': 0x33, 'funct3': 0x0, 'funct7': 0x01},
33            'div': {'type': 'R', 'opcode': 0x33, 'funct3': 0x4, 'funct7': 0x01},
34            'rem': {'type': 'R', 'opcode': 0x33, 'funct3': 0x6, 'funct7': 0x01},
35
36            # I-Type
37            'addi': {'type': 'I', 'opcode': 0x13, 'funct3': 0x0},
38            'slti': {'type': 'I', 'opcode': 0x13, 'funct3': 0x2},
39            'xori': {'type': 'I', 'opcode': 0x13, 'funct3': 0x4},
40            'ori': {'type': 'I', 'opcode': 0x13, 'funct3': 0x6},
41            'andi': {'type': 'I', 'opcode': 0x13, 'funct3': 0x7},
42            'lw': {'type': 'I-load', 'opcode': 0x03, 'funct3': 0x2},
43            'jalr': {'type': 'I-jalr', 'opcode': 0x67, 'funct3': 0x0},
44
45            # S-Type
46            'sw': {'type': 'S', 'opcode': 0x23, 'funct3': 0x2},
47
48            # B-Type
49            'beq': {'type': 'B', 'opcode': 0x63, 'funct3': 0x0},
50            'bne': {'type': 'B', 'opcode': 0x63, 'funct3': 0x1},
51            'blt': {'type': 'B', 'opcode': 0x63, 'funct3': 0x4},
52            'bge': {'type': 'B', 'opcode': 0x63, 'funct3': 0x5},
53
54            # J-Type

```

```

55         'jal': {'type': 'J', 'opcode': 0x6F},
56     }
57
58     def _parse_reg(self, reg_str):
59         return self.reg_map[reg_str.lower()]
60
61     def _to_twos_comp(self, val, bits):
62         """Converte um valor para complemento de dois com 'bits' de largura."""
63         if val < 0:
64             val = (1 << bits) + val
65         return val
66
67     def encode(self, asm):
68         """Funcao principal que recebe uma string em Assembly e retorna o codigo de
69         maquina."""
70         asm = asm.lower().replace(' ', '')
71         parts = asm.split()
72         mnemonic = parts[0]
73         operands = parts[1:]
74
75         if mnemonic not in self.instructions:
76             raise ValueError(f"Instrucao desconhecida: {mnemonic}")
77
78         instr_info = self.instructions[mnemonic]
79         instr_type = instr_info['type']
80
81         # --- Logica de montagem para cada tipo de instrucao ---
82
83         if instr_type == 'R':
84             rd, rs1, rs2 = map(self._parse_reg, operands)
85             opcode = instr_info['opcode']
86             funct3 = instr_info['funct3']
87             funct7 = instr_info['funct7']
88             return (funct7 << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) | (rd <<
89                     7) | opcode
90
91         elif instr_type in ['I', 'I-jalr']:
92             rd = self._parse_reg(operands[0])
93             rs1 = self._parse_reg(operands[1])
94             imm = int(operands[2])
95             imm = self._to_twos_comp(imm, 12)
96
97             opcode = instr_info['opcode']
98             funct3 = instr_info['funct3']
99             return (imm << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode
100
101         elif instr_type == 'I-load':
102             rd = self._parse_reg(operands[0])
103             match = re.match(r'(-?\d+)\((.+)\)', operands[1]) # Padrao offset(base)
104             imm = int(match.group(1))
105             rs1 = self._parse_reg(match.group(2))
106             imm = self._to_twos_comp(imm, 12)
107
108             opcode = instr_info['opcode']
109             funct3 = instr_info['funct3']
110             return (imm << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode
111
112         elif instr_type == 'S':
113             rs2 = self._parse_reg(operands[0])
114             match = re.match(r'(-?\d+)\((.+)\)', operands[1]) # Padrao offset(base)

```

```

113         imm = int(match.group(1))
114         rs1 = self._parse_reg(match.group(2))
115         imm = self._to_twos_comp(imm, 12)
116
117         imm11_5 = (imm >> 5) & 0x7F # Pega os 7 bits mais significativos
118         imm4_0 = imm & 0x1F          # Pega os 5 bits menos significativos
119
120         opcode = instr_info['opcode']
121         funct3 = instr_info['funct3']
122         return (imm11_5 << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) | (imm4_0
            << 7) | opcode
123
124     elif instr_type == 'B':
125         rs1 = self._parse_reg(operands[0])
126         rs2 = self._parse_reg(operands[1])
127         imm = int(operands[2])
128         imm = self._to_twos_comp(imm, 13)
129
130         imm12 = (imm >> 12) & 0x1
131         imm10_5 = (imm >> 5) & 0x3F
132         imm4_1 = (imm >> 1) & 0xF
133         imm11 = (imm >> 11) & 0x1
134
135         opcode = instr_info['opcode']
136         funct3 = instr_info['funct3']
137
138         field1 = (imm12 << 6) | imm10_5 # [12|10:5]
139         field2 = (imm4_1 << 1) | imm11 # [4:1|11]
140
141         return (field1 << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) | (field2
            << 7) | opcode
142
143     elif instr_type == 'J':
144         rd = self._parse_reg(operands[0])
145         imm = int(operands[1])
146         imm = self._to_twos_comp(imm, 21)
147
148         imm20 = (imm >> 20) & 0x1
149         imm10_1 = (imm >> 1) & 0x3FF
150         imm11 = (imm >> 11) & 0x1
151         imm19_12 = (imm >> 12) & 0xFF
152
153         imm_field = (imm20 << 19) | (imm10_1 << 9) | (imm11 << 8) | imm19_12
154
155         opcode = instr_info['opcode']
156         return (imm_field << 12) | (rd << 7) | opcode
157
158     return None
159
160 # --- Exemplo de Uso ---
161 if __name__ == "__main__":
162     asm = RiscVAssembler()
163
164     # Suas instrucoes de teste
165     instructions_to_test = [
166         "lw x3, 2044(x0)",
167
168         "addi x1, x0, 0",
169         "addi x2, x0, 1",
170         "sub x3, x3, x2",

```

```

171     "beq x3, x0, 24",
172     "add x6, x1, x2",
173     "addi x1, x2, 0",
174     "addi x2, x6, 0",
175     "addi x3, x3, -1",
176     "sw x1, 2040(x0)",
177     "jal x0, -24",
178
179     "sw x1, 2040(x0)",
180     "jal x0, -48"
181 ]
182
183 print("--- Gerador de Instrucoes RISC-V ---")
184
185 for i, instr_str in enumerate(instructions_to_test):
186     try:
187         machine_code = asm.encode(instr_str)
188
189         # Formata para Verilog Hexadecimal (8 digitos)
190         hex_code = f"32'h{machine_code:08X}"
191
192         # Formata para Verilog Binario (32 bits)
193         bin_code = f"32'b{machine_code:032b}"
194
195         print(f"\nmem[{i}] = {bin_code}; // {instr_str}")
196     # print(f"mem[{i}] = {hex_code}; // Binario: {bin_code}")
197
198     except (ValueError, AttributeError) as e:
199         print(f"\nErro ao processar '{instr_str}': {e}")

```

Listing D.1 – Código python do *Assembler*.

Anexos

